PCCST 501 COMPUTER NETWORKS

Ms. ANJU MARKOSE,
Asst. Professor & Dept. Secretary,
Dept. of Computer Science and Design,
VJCET, Vazhakulam.
Ph No: 9946004657.



SEMESTER S5

COMPUTER NETWORKS

(Common to CS/CD/CM/CR/CA/AD/AI/CB/CN/CU/CI)

Course Code	**PCCST501**	**CIE Marks**	40
Teaching Hours/Week (L: T:P: R)	3:1:0:0	**ESE Marks**	60
Credits	4	**Exam Hours**	2 Hrs. 30 Min.
Prerequisites (if any)	None	**Course Type**	Theory

Course Objectives:

1. To introduce the core concepts of computer networking.
2. To develop a big picture of the internetworking implementation on Linux-based systems.
3. To impart an overview of network management concepts.

MODULE 2

2	Transport Layer: Services, Protocols, UDP, TCP (Book 1 Ch 3). ***Hands-on: Sockets Introduction, Elementary TCP Sockets, TCP Client/Server Example, I/O Multiplexing: The select and poll Functions (Book 2 Ch 3 to 6), Elementary UDP Sockets (Book 2 Ch 8), Advanced I/O Functions (Book 2 Ch 14)*** Network Layer: Introduction, Network-layer protocols, Unicast routing, Multicast routing - Multicasting Basics, Intra domain and inter-domain routing, Next generation IP (Book 1 Ch 4), Quality of Service (Book 1 Ch 8) ***Hands-on: Linux Kernel Implementation of Routing Table and Caches, Routing Cache Implementation Overview, Adding new entry in the Routing Table using ip command (Book 3 Ch 14)***	**18**

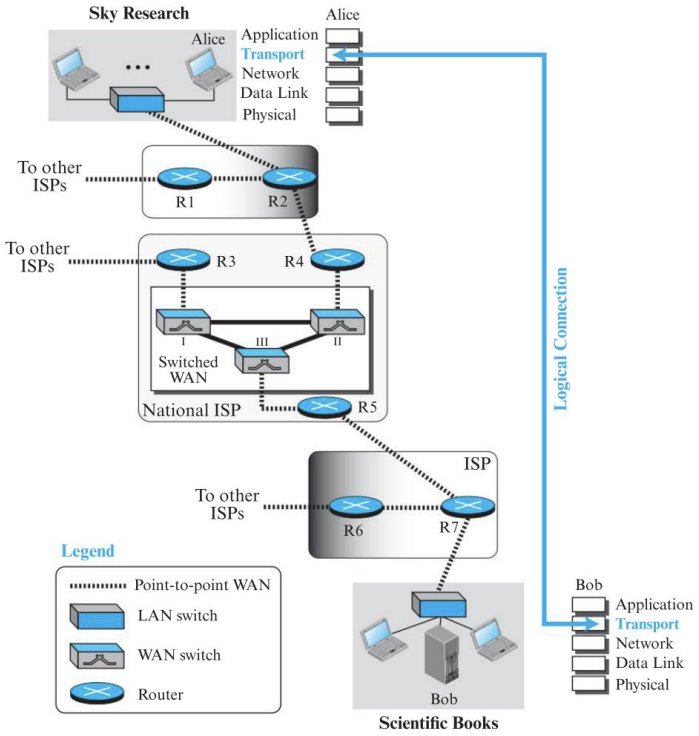
Transport Layer

- The transport layer in the TCP/IP suite is located between the application layer and the network layer.
- It provides services to the application layer and receives services from the network layer.
- The transport layer acts as a liaison between a client program and a server program, a process-to-process connection.
- The transport layer is the heart of the TCP/IP protocol suite; it is the end-to-end logical vehicle for transferring data from one point to another in the Internet.

INTRODUCTION

- The transport layer is located between the application layer and the network layer.
- It provides a process-to-process communication between two application layers, one at the local host and the other at the remote host.
- Communication is provided using a logical connection, which means that the two application layers, which can be located in different parts of the globe, assume that there is an imaginary direct connection through which they can send and receive messages.

Figure 3.1 *Logical connection at the transport layer*



- This diagram shows how two end users — **Alice** (at Sky Research) and **Bob** (at Scientific Books) — are logically connected at the **Transport Layer**, even though their physical paths go through multiple networks.
- Data travels through a chain of **routers (R1–R7)**, a **National ISP** (with a Switched WAN), and local **ISPs**.
- The actual path uses **Point-to-point WANs**, **LAN switches**, **WAN switches**, and **Routers**.
- Despite this complex physical path, the **Transport Layer** creates a direct logical connection between Alice and Bob (shown by the blue line on the right).
- Both Alice and Bob use the standard 5-layer model: Application, **Transport**, Network, Data Link, and Physical.

The Transport layer provides end-to-end communication between two hosts, regardless of the underlying network infrastructure in between.

Transport Layer Services

- The **Transport Layer** is located between the **Application Layer** and the **Network Layer**. It provides services to application processes and receives services from the network layer
- The primary responsibility of the transport layer is **Process-to-Process Communication**.

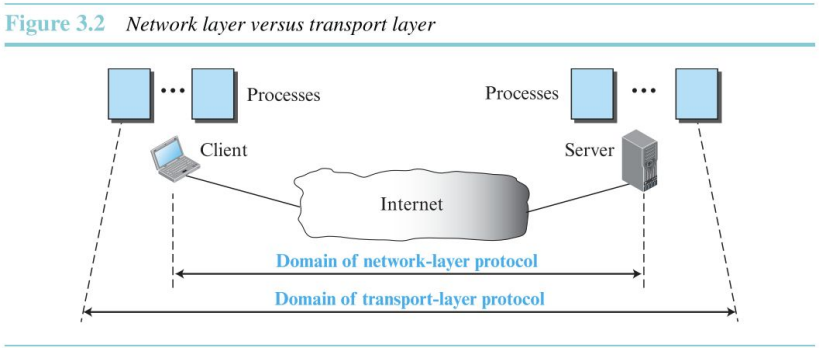


Figure 3.2 *Network layer versus transport layer*

Network Layer

- Provides **Host-to-Host Communication**
- Delivers data to the destination computer

Transport Layer

- Provides **Process-to-Process Communication**
- Delivers data to the correct process/application on that computer

Process-to-Process Communication

- A **process** is a running application program that uses transport layer services.

Host-to-Host vs Process-to-Process Communication

- The network layer can identify the destination host using an IP address, but it cannot determine which application should receive the data.
- The transport layer solves this problem.

Client–Server Communication

Most process-to-process communication follows the **Client–Server Model**.

Client

- Requests services.
- Usually runs on the local host.
- Uses a temporary port number.

Server

- Provides services.
- Usually runs on a remote host.
- Uses a fixed, well-known port number.

Example:

- Daytime Client → Requests current date and time.
- Daytime Server → Provides date and time service.

Port Numbers

To identify processes, the transport layer uses **Port Numbers**.

Definition

A port number is a **16-bit integer** ranging from:**0 to 65,535**

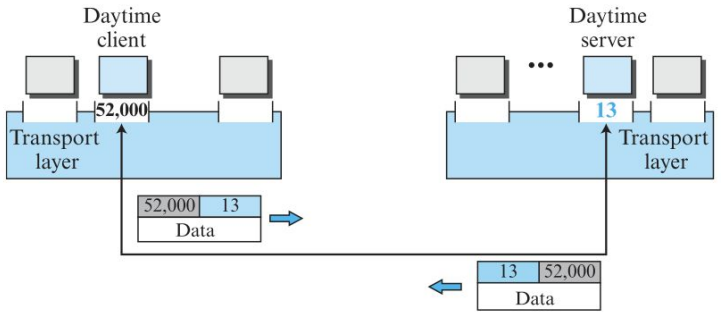
Purpose

- **IP Address** identifies the destination host.
- **Port Number** identifies the destination process on that host.

Thus, communication requires both:

(IP Address, Port Number)

Figure 3.3 *Port numbers*



Types of Port Numbers

1. Ephemeral Port Number

- Used by client processes.
- Temporary (short-lived).
- Chosen dynamically by the operating system.
- Usually greater than **1023**.

Example: 52000

2. Well-Known Port Number

- Used by server processes.
- Fixed and universally recognized.
- Allows clients to locate services easily.

Example:

- Daytime Server → Port **13**

CANN Port Number Ranges

The **ICANN (Internet Corporation for Assigned Names and Numbers)** classifies port numbers into three categories:

Range	Type	Description
0 – 1023	Well-Known Ports	Assigned and controlled by ICANN
1024 – 49151	Registered Ports	Registered with ICANN to avoid duplication
49152 – 65535	Dynamic/Private Ports	Temporary ports used by clients

- *Transport layer provides process-to-process communication.*
- *A process is a running application program.*
- *Network layer handles host-to-host communication.*
- *Transport layer delivers data to the correct process using port numbers.*
- *Clients typically use ephemeral ports.*
- *Servers use well-known ports.*
- *Port numbers range from 0 to 65,535.*
- *ICANN divides ports into Well-Known, Registered, and Dynamic ranges.*

IP Address → Identifies the Host

Port Number → Identifies the Process

IP Address + Port Number → Complete Destination Identification.

1. Socket Address

A **Socket Address** is the combination of:

Socket Address = IP Address + Port Number

- **IP Address** identifies the host (computer).
- **Port Number** identifies the process (application) running on that host.

Types of Socket Addresses

- **Client Socket Address** → Uniquely identifies the client process.
- **Server Socket Address** → Uniquely identifies the server process.

Communication Requirement

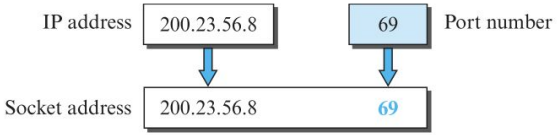
For communication over the Internet, two socket addresses are required:

1. Client Socket Address
2. Server Socket Address

Thus, a connection is identified by:

(Client IP, Client Port, Server IP, Server Port)

Figure 3.6 *Socket address*

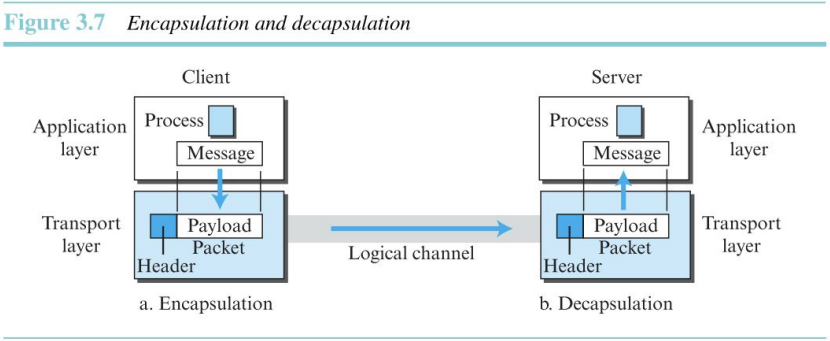


2. Encapsulation and Decapsulation

Transport layer communication involves adding and removing headers.

Encapsulation (Sender Side)

1. The application process creates a message.
2. The message is passed to the transport layer.
3. The transport layer adds a transport header containing port information.
4. The packet is forwarded to the network layer.



Identifies source and destination processes and enables reliable delivery of data.

Decapsulation (Receiver Side)

1. The transport layer receives the packet.
2. The transport header is removed.
3. The original message is extracted.
4. The message is delivered to the correct application process.

The sender's socket address is also passed to the receiving process so that it can send a reply if required.

3. Multiplexing and Demultiplexing

Multiplexing (Many-to-One)

Definition:
Combining data from multiple application processes into packets before transmission.

Performed At

Sender Transport Layer

Example

- Process P1 sends data.
- Process P2 sends data.
- Process P3 sends data.

The transport layer collects all messages and creates packets for transmission.

Demultiplexing (One-to-Many)

Definition:
Delivering incoming packets to the correct destination process.

Performed At

Receiver Transport Layer

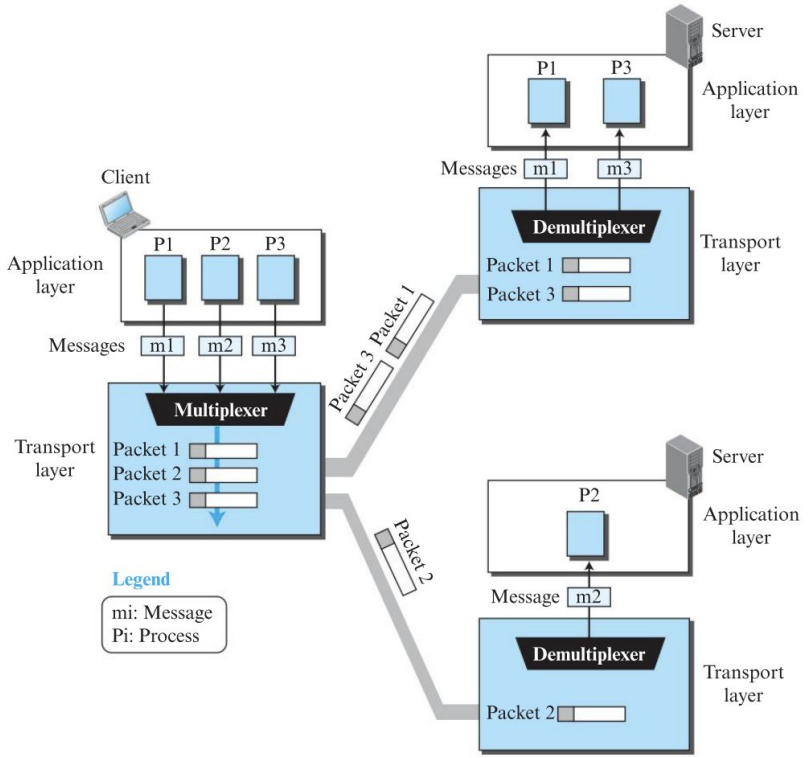
Example

- Packet 1 → Process P1
- Packet 2 → Process P2
- Packet 3 → Process P3

The transport layer examines port numbers and forwards each packet to the appropriate process.

Multiplexing	Demultiplexing
Many processes → One transport layer	One transport layer → Many processes
Sender side operation	Receiver side operation

Figure 3.8 *Multiplexing and demultiplexing*



4. Flow Control

Definition

Flow control ensures that the sender does not transmit data faster than the receiver can process it.

Need for Flow Control

Without flow control:

- Receiver may become overloaded.
- Packets may be lost.
- Communication becomes inefficient.

Goal

Maintain a balance between:

Production Rate = Consumption Rate

5. Pushing and Pulling

Pushing

The sender sends data whenever it is produced.

Characteristics

- No prior request from receiver.
- Receiver may become overloaded.
- Flow control is required.

Example

Email delivery.

Pulling

The receiver requests data when needed.

Characteristics

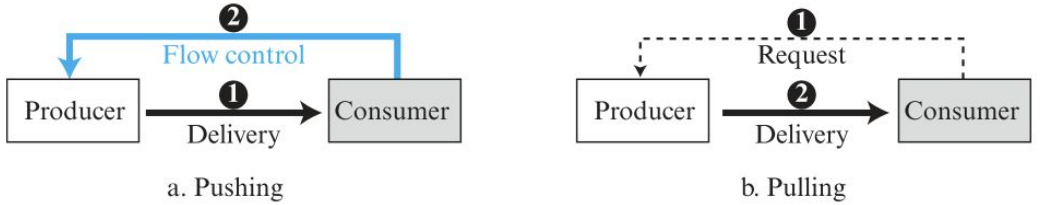
- Receiver controls data transfer.
- No risk of overload.
- Flow control is generally unnecessary.

Example

Downloading a webpage after requesting it.

Pushing	Pulling
Sender initiates transfer	Receiver initiates transfer
Flow control required	Flow control usually not required
Receiver may be overwhelmed	Receiver controls data rate

Figure 3.9 *Pushing or pulling*



6.Flow Control at the Transport Layer

- Flow control is a mechanism used by the transport layer to ensure that the sender does not transmit data faster than the receiver can process it.
- It helps prevent packet loss, buffer overflow, and network congestion.

Entities Involved in Flow Control

Transport-layer communication involves four entities:

1. Sender Process

- The sender process is an application running on the source host.
- It generates data or messages and passes them to the sender transport layer for transmission.

2. Sender Transport Layer

- The sender transport layer receives messages from the sender process and stores them temporarily in a buffer.
- It encapsulates the messages into packets by adding the transport-layer header and forwards them to the receiver transport layer.
- It also responds to flow-control signals from the receiver.

3. Receiver Transport Layer

- The receiver transport layer accepts packets arriving from the network.
- It stores the packets in a receiving buffer, removes the transport-layer header (decapsulation), and delivers the original message to the appropriate receiver process.
- It monitors buffer availability and sends flow-control information back to the sender.

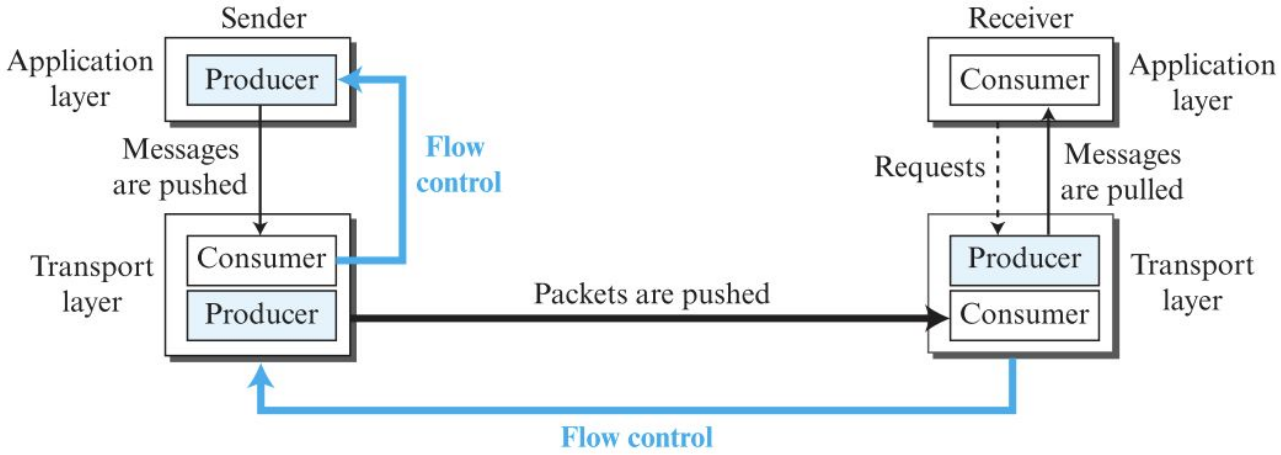
4. Receiver Process

- The receiver process is the destination application that consumes the received data.
- It retrieves messages from the receiver transport layer and processes them according to the application requirements.

Working of Flow Control

- The sender continuously sends packets only when the receiver has sufficient space available in its buffer.
- If the receiver's buffer becomes full, it informs the sender to temporarily stop transmission. Once buffer space becomes available, the receiver notifies the sender to resume transmission.
- This coordination ensures reliable and efficient communication.

Figure 3.10 *Flow control at the transport layer*



7. Buffers in Flow Control

A buffer is a temporary storage area in memory used to hold data packets before they are processed or transmitted. Buffers play a vital role in implementing flow control by handling differences in the data production and consumption rates.

Types of Buffers

1. Sending Buffer

The sending buffer is located at the sender transport layer. It temporarily stores outgoing packets before they are transmitted across the network. If the buffer becomes full, the sender transport layer instructs the application process to pause sending additional data until space becomes available.

2. Receiving Buffer

The receiving buffer is located at the receiver transport layer. It temporarily stores incoming packets until they can be processed and delivered to the receiver process. If the receiving buffer becomes full, the receiver transport layer sends a signal to the sender to stop transmission. When free space becomes available, the receiver allows the sender to continue sending packets.

Importance of Buffers

- Prevent packet loss due to receiver overload.
- Handle differences in sender and receiver speeds.
- Improve communication efficiency.
- Support reliable flow-control mechanisms.

Flow control ensures a balance between data transmission and data reception. Buffers at both the sender and receiver sides temporarily store packets and help regulate the flow of data, preventing congestion and loss of information.

8. Error Control

- The Internet Protocol (IP) at the network layer is **unreliable**, meaning that packets may be lost, corrupted, duplicated, or delivered out of order.
- Therefore, if an application requires reliable communication, the transport layer must provide **error control mechanisms**.
- Error control ensures that data is delivered accurately and completely from the sender to the receiver.

Functions of Error Control

The transport layer performs the following error-control functions:

1. Detection and Discarding of Corrupted Packets

During transmission, packets may become corrupted due to noise or other network problems. The receiver detects these corrupted packets and discards them.

2. Recovery of Lost Packets

Some packets may be lost during transmission. The receiver informs the sender about the missing packets, and the sender retransmits them.

3. Detection and Removal of Duplicate Packets

- A packet may be received more than once because of retransmissions. Error control identifies duplicate packets and discards the extra copies.

4. Handling Out-of-Order Packets

- Packets may not always arrive in the same order in which they were sent.
- The receiver temporarily stores out-of-order packets and waits for the missing packets before delivering the data to the application.

Working of Error Control

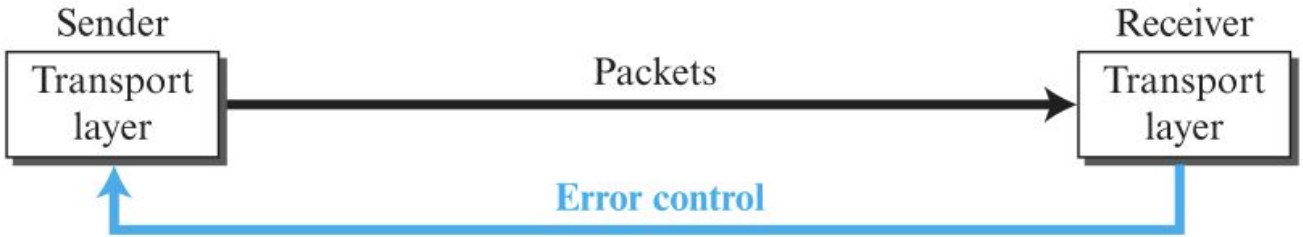
Error control operates between the:

- Sender Transport Layer
- Receiver Transport Layer

The receiving transport layer continuously monitors incoming packets and informs the sender whenever a packet is lost, corrupted, duplicated, or received out of order.

Unlike flow control, error control does not involve the application processes directly.

Figure 3.11 *Error control at the transport layer*



9. Sequence Numbers

- A **Sequence Number** is a unique number assigned to each packet before transmission.
- It helps the transport layer identify lost, duplicate, and out-of-order packets.

Purpose of Sequence Numbers

Sequence numbers help in:

- Detecting lost packets
- Identifying duplicate packets
- Recognizing out-of-order packets
- Requesting retransmission of missing packets

Packets are numbered sequentially before transmission.

Example:

Packet 1 → Sequence Number 0
Packet 2 → Sequence Number 1
Packet 3 → Sequence Number 2
Packet 4 → Sequence Number 3

If Packet 2 is lost, the receiver notices the gap in sequence numbers and can request retransmission.

10. Acknowledgment (ACK)

- An **Acknowledgment (ACK)** is a control message sent by the receiver to inform the sender that a packet has been received successfully.
- ACKs are the most commonly used positive feedback mechanism in transport-layer error control.

Step 1: Packet Transmission

The sender transmits a packet to the receiver.

Step 2: ACK Generation

If the packet arrives correctly, the receiver sends an acknowledgment (ACK) back to the sender.

Step 3: Confirmation

The sender receives the ACK and knows that the packet was delivered successfully.

Sliding Window

The **Sliding Window** is a mechanism used for flow control and error control. Since sequence numbers are assigned modulo 2m2m, they can be represented as a circle ranging from **0 to 2m−12m−1**.

A portion of this sequence number space is called the **sliding window**. The window represents the packets that can be sent before receiving acknowledgments.

Working of Sliding Window

- When a packet is sent, the corresponding position (slice) in the window is marked.
- If all slices are marked, the sender's buffer is full and no more packets can be accepted from the application layer.
- When an acknowledgment (ACK) is received, the corresponding slice is unmarked.
- If consecutive slices at the beginning of the window become unmarked, the window moves forward (slides), creating space for new packets.

Example

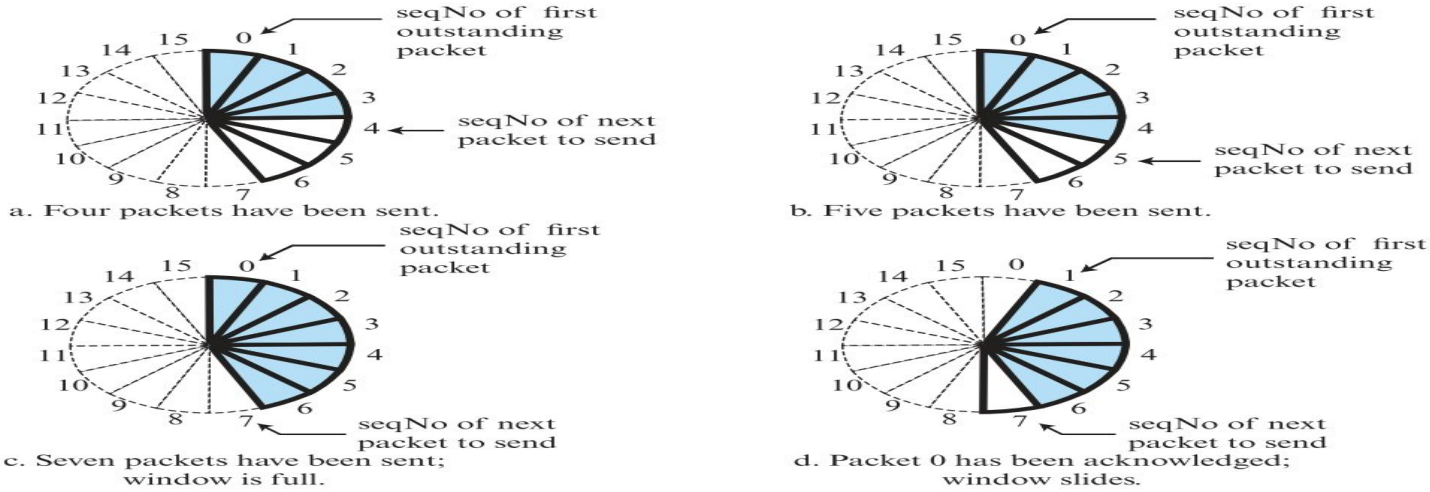
If:

- m=4m=4
- Sequence numbers = 0 to 15 (Modulo 16)
- Window size = 7

The sender can transmit seven packets before waiting for acknowledgments. As ACKs arrive, the window slides forward, allowing transmission of additional packets.

Note: The sliding window is an abstraction. In practice, computer variables are used to keep track of the next packet to be sent and the last packet sent.

Figure 3.12 *Sliding window in circular format*



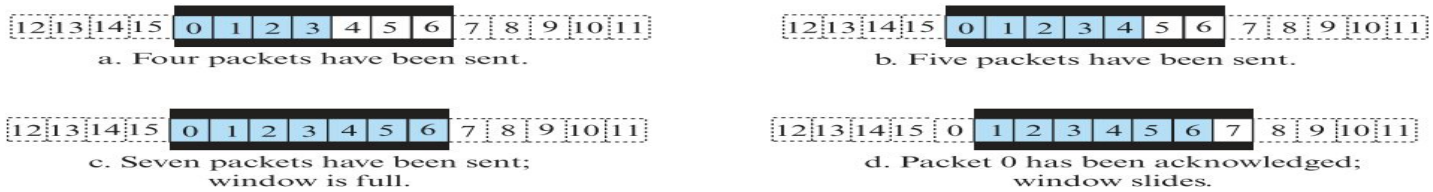
a. Four packets have been sent.

b. Five packets have been sent.

c. Seven packets have been sent; window is full.

d. Packet 0 has been acknowledged; window slides.

Figure 3.13 *Sliding window in linear format*



a. Four packets have been sent.

b. Five packets have been sent.

c. Seven packets have been sent; window is full.

d. Packet 0 has been acknowledged; window slides.

Congestion Control

- **Congestion** is an important issue in packet-switched networks such as the Internet. It occurs when the **network load**(number of packets sent to the network) exceeds the **network capacity** (number of packets the network can handle).
- **Congestion Control** refers to the mechanisms and techniques used to control congestion and keep the network load below its capacity.

Causes of Congestion

- Congestion occurs in systems where waiting is involved. Similar to traffic congestion on a highway, any abnormality in network traffic can create blockages and delays.
- In a network, congestion occurs because routers and switches use **queues (buffers)** to temporarily store packets before and after processing.

Each router contains:

- Input queues
- Output queues

If packets arrive faster than the router can process and forward them, the queues become overloaded, resulting in congestion.

Effects of Congestion

- Increased packet delay
- Packet loss due to buffer overflow
- Reduced network performance
- Increased retransmissions

Congestion at the Transport Layer

- Congestion observed at the transport layer is actually a result of congestion occurring at the network layer. When routers and switches become overloaded, the effects are reflected in transport-layer communication.
- TCP assumes that there is no congestion-control mechanism at the network layer and therefore implements its own congestion-control techniques to avoid network overload.

Connectionless and Connection-Oriented Services

A transport-layer protocol can provide two types of services:

1. **Connectionless Service**
2. **Connection-Oriented Service**

At the transport layer, **connectionless service means independence between packets**, while **connection-oriented service means dependency and coordination between packets**.

Connectionless Service

- In a connectionless service, the source process divides its message into chunks and delivers them to the transport layer one by one.
- The transport layer treats each chunk as an independent unit, encapsulates it into a packet, and sends it without establishing a connection.

Characteristics

- No logical connection is established before data transfer.
- Each packet is treated independently.
- Packets may arrive out of order.
- Lost packets cannot be easily detected.
- The receiving transport layer does not know when the first packet will arrive or when all packets have arrived.
- There is no coordination between the sending and receiving transport layers.

Limitation :-Flow control, error control, and congestion control cannot be

effectively implemented in a connectionless service.

Figure 3.14 *Connectionless service*

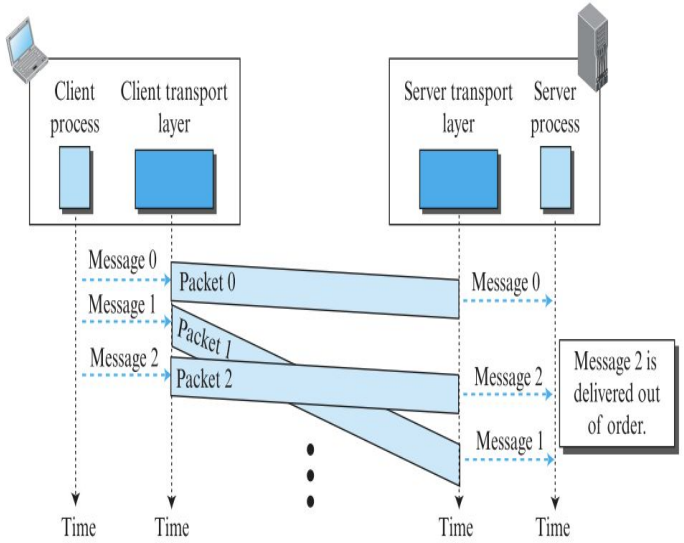
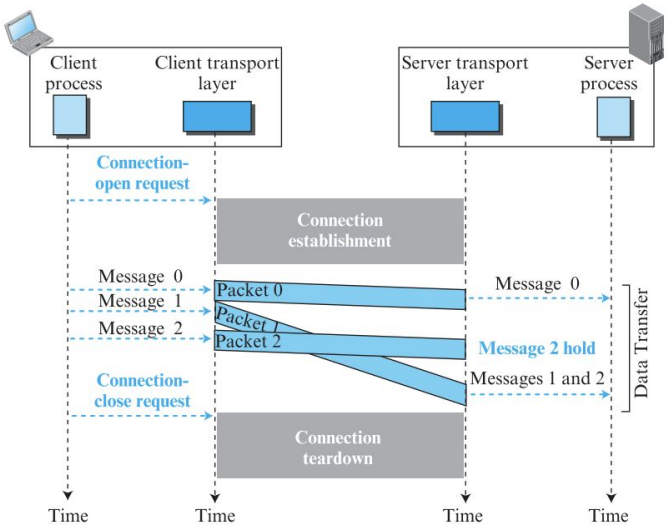


Figure 3.15 *Connection-oriented service*



Connection-Oriented Service

In a connection-oriented service, the client and server must first establish a logical connection before exchanging data. After data transfer is completed, the connection is terminated.

Phases of Communication

1. **Connection Establishment**
2. **Data Transfer**
3. **Connection Termination (Tear-down)**

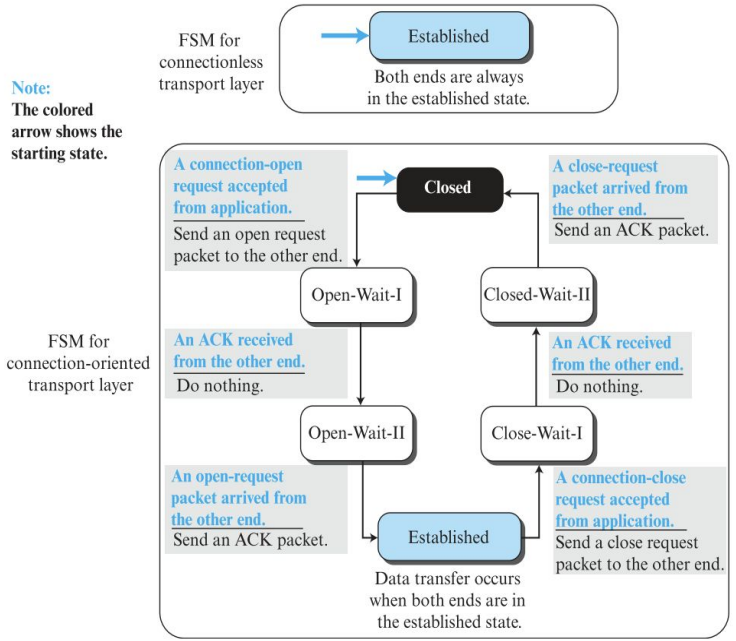
Characteristics

- A logical connection exists between sender and receiver.
- Data transfer occurs only after connection establishment.
- Coordination exists between the two transport layers.
- Supports reliable communication through flow control, error control, and congestion control.

Network Layer	Transport Layer
Involves end hosts and intermediate routers	Involves only the two end hosts
Connection is established across the network path	Connection is end-to-end between hosts
Requires coordination with routers	No coordination with intermediate routers

- A connection-oriented transport protocol can operate over either a connectionless or connection-oriented network-layer protocol.
- **Connectionless Service:** Packets are independent; no connection setup.
- **Connection-Oriented Service:** Packets are related; connection establishmen data transfer, and connection termination are required.

Figure 3.16 *Connectionless and connection-oriented service represented as FSMs*



This diagram compares two transport layer services using **Finite State Machines (FSMs)**.

1. Connectionless Transport (UDP-like):

- Has only **one state — Established**.
- Both ends are always considered connected; no setup or teardown needed.

2. Connection-Oriented Transport (TCP-like):

- Starts in the **Closed** state.
- **Opening a connection:** Closed → Open-Wait-I → Open-Wait-II → **Established** (data transfer happens here).
- **Closing a connection:** Established → Close-Wait-I → Closed-Wait-II → **Closed**.
- Each transition is triggered by events like receiving an ACK, open-request, or close-request packet.
- Connectionless service is always "ready," while connection-oriented service goes through a formal handshake to open and close a connection before and after data transfer.

Transport-Layer Protocols

- Transport-layer protocols are created by combining different transport-layer services such as connection control, flow control, and error control.
- To understand their operation, protocols are usually studied from simple to complex forms.
- Initially, they are considered as **unidirectional (simplex) protocols**, where data flows only in one direction.
- These protocols can later be extended to **bidirectional (full-duplex) protocols**, where data flows in both directions.

3.2.1 Simple Protocol

The **Simple Protocol** is the simplest transport-layer protocol.

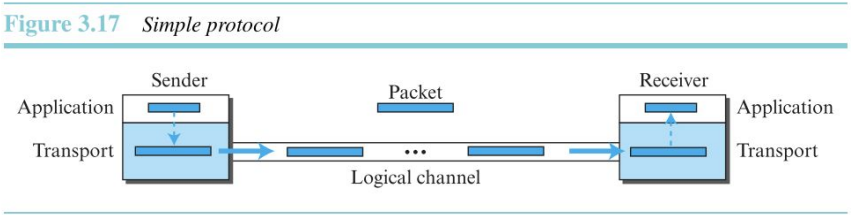


Figure 3.17 *Simple protocol*

Features

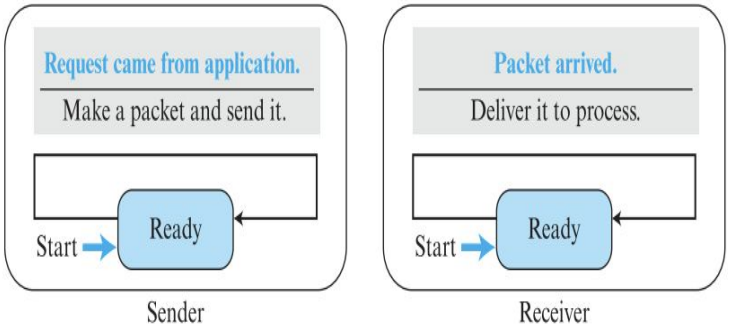
- Connectionless protocol.
- No flow control.
- No error control.
- Assumes the receiver can process incoming packets immediately.
- The receiver is never overwhelmed by incoming packets.

Working

Sender Side

1. The transport layer receives a message from the application layer.
2. It encapsulates the message into a packet.
3. The packet is sent to the receiver.

Figure 3.18 *FSMs for the simple protocol*



Receiver Side

1. The transport layer receives the packet from the network layer.
2. It extracts the message from the packet.
3. The message is delivered to the application layer.

Two FSMs govern sender and receiver, each with just **one state — Ready**.

- **Sender:** Waits in Ready state → gets a message from the app → wraps it in a packet → sends it.
- **Receiver:** Waits in Ready state → packet arrives → unwraps it → delivers message to the app.
- Both machines always stay in the **Ready state**; they simply react to events (message from app / packet arrival).
- UDP is a slight modification of this simple protocol.

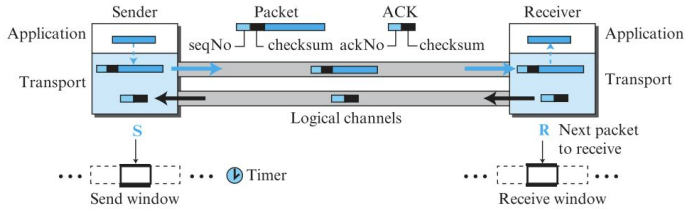
Stop-and-Wait Protocol

- The **Stop-and-Wait Protocol** is a **connection-oriented protocol** that provides both **flow control** and **error control**.

Characteristics

- Uses a **sliding window of size 1** at both sender and receiver.
- The sender sends **one packet at a time**.
- The sender waits for an acknowledgment (ACK) before sending the next packet.
- Only **one data packet and one ACK** can be in transit at any time.

Figure 3.20 *Stop-and-Wait protocol*



Working of Stop-and-Wait Protocol

1. The sender sends a packet and starts a timer.
2. The receiver checks the packet using a **checksum**.
3. If the packet is error-free, it is delivered to the application layer and an ACK is sent.
4. If the packet is corrupted, it is discarded.
5. If the ACK arrives before the timer expires, the sender sends the next packet.
6. If the timer expires before receiving an ACK, the sender retransmits the same packet.
7. The sender keeps a copy of the transmitted packet until its ACK is received.

Error Control

Error control is achieved using:

- **Checksum** → Detects corrupted packets.
- **Acknowledgments (ACKs)** → Confirm successful reception.
- **Timer** → Detects lost packets or lost ACKs.
- **Retransmission** → Resends lost or corrupted packets.

Sequence Numbers

Sequence numbers are used to identify packets and prevent duplicate packet delivery.

Why Sequence Numbers Are Needed

Three situations may occur after sending a packet:

1. Packet and ACK are received successfully.
2. Packet is lost or corrupted, so the sender retransmits it after timeout.
3. Packet is received correctly, but the ACK is lost or corrupted. The sender retransmits the packet, creating a duplicate.

To distinguish a new packet from a duplicate packet, sequence numbers are required.

Sequence Number Range

Only two sequence numbers are needed:

0 and 1

Therefore, the protocol uses **Modulo-2 Arithmetic**.

Sequence:

0, 1, 0, 1, 0, 1, ...

Acknowledgment Numbers

The acknowledgment number always indicates the **sequence number of the next packet expected** by the receiver.

Examples

- If packet **0** is received correctly, the receiver sends **ACK 1**, indicating that packet 1 is expected next.
- If packet **1** is received correctly, the receiver sends **ACK 0**, indicating that packet 0 is expected next.

Control Variables

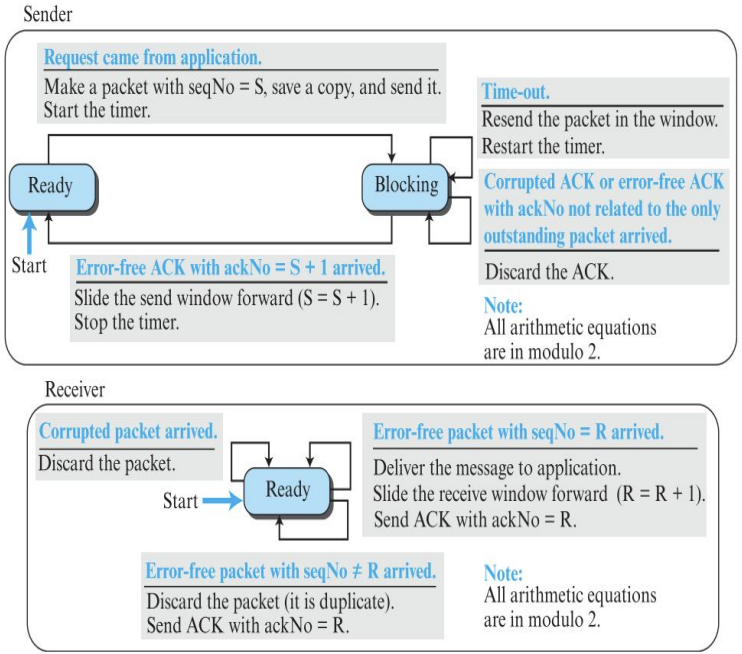
Sender Variable (S)

- Points to the only slot in the sender window.
- Indicates the sequence number of the packet to be sent.

Receiver Variable (R)

- Points to the only slot in the receiver window.
- Indicates the sequence number of the packet expected by the receiver.

Figure 3.21 *FSM for the Stop-and-Wait protocol*



Sender has 2 states — Ready and Blocking:

- **Ready** → gets a message → sends packet, starts timer → moves to **Blocking**.
- **Correct ACK received** → moves back to **Ready**.
- **Timeout** → resends the packet, restarts timer, stays **Blocking**.
- **Wrong/corrupted ACK** → discards it, stays **Blocking**.

Receiver has 1 state — Ready (always):

- **Good packet (correct seqNo)** → delivers to app, sends ACK.
- **Duplicate packet (wrong seqNo)** → discards it, resends ACK.
- **Corrupted packet** → simply discards it.
- Sender sends one packet and *waits* (blocks) for an ACK before sending the next. If no ACK comes, it retransmits. Sequence numbers (in modulo 2) help detect duplicates.

Go-Back-N (GBN) Protocol

The **Go-Back-N (GBN) Protocol** is a **connection-oriented protocol** that improves transmission efficiency by allowing **multiple packets** to be sent before receiving acknowledgments.

Unlike the Stop-and-Wait protocol, several packets can be in transit simultaneously, keeping the communication channel busy.

- Multiple packets can be sent without waiting for individual ACKs.
- The sender keeps a copy of all transmitted packets until they are acknowledged.
- The receiver can buffer **only one packet**.
- Several data packets and ACKs can be in the channel at the same time.

Sequence Numbers

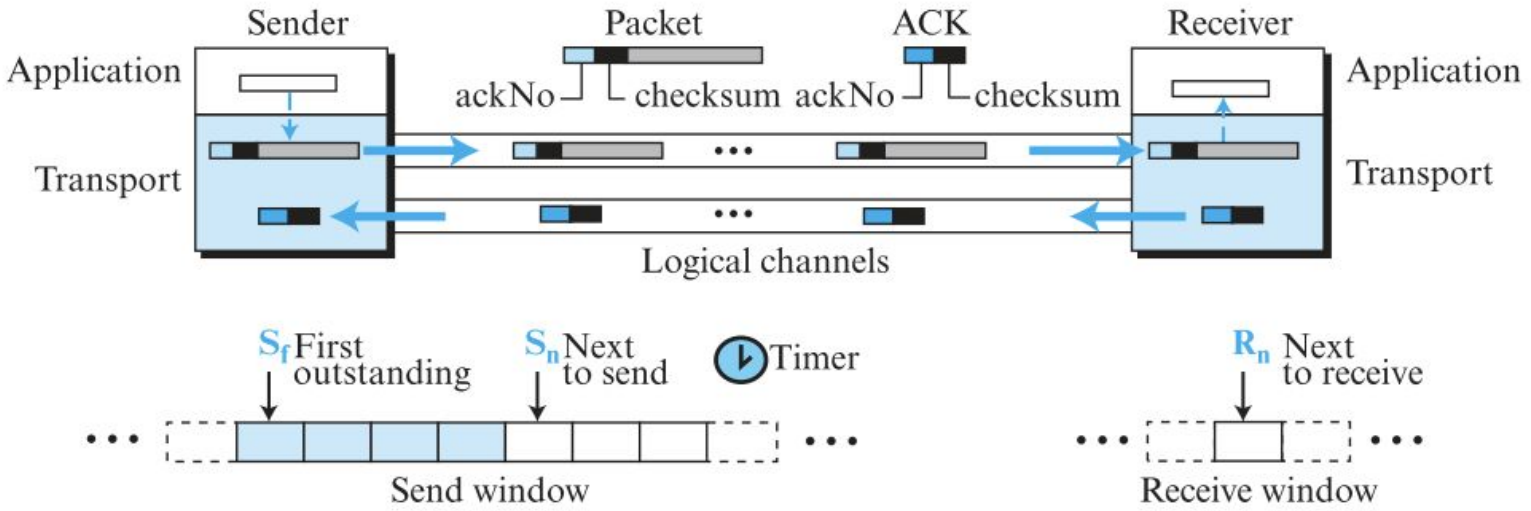
Sequence numbers are assigned using **Modulo 2m2m** arithmetic, where **m** is the number of bits in the sequence number field.

Acknowledgment Numbers

The acknowledgment number is **cumulative** and indicates the **next packet expected** by the receiver.

Example: ACK = 7 means that all packets with sequence numbers **0 to 6** have been received successfully, and the receiver is expecting packet **7**.

Figure 3.23 *Go-Back-N protocol*



In the Go-Back-*N* protocol, the acknowledgment number is cumulative and defines the sequence number of the next packet expected to arrive.

Send Window

The **send window** is an imaginary window that contains the sequence numbers of packets that can be sent or are waiting for acknowledgment.

Maximum Window Size

Maximum Send Window Size = 2^m − 1

Regions of the Send Window

The send window divides the sequence numbers into four regions:

1. **Acknowledged Packets** – Packets already acknowledged; no copies are kept.
2. **Outstanding Packets** – Packets sent but not yet acknowledged.
3. **Packets Ready to be Sent** – Packets that can be transmitted.
4. **Packets Outside the Window** – Cannot be sent until the window slides.

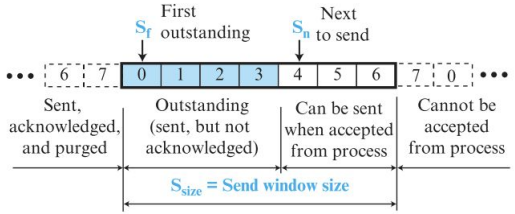
Send Window Variables

The send window is represented using three variables:

- **Sf** – Sequence number of the **first outstanding packet**.
- **Sn** – Sequence number of the **next packet to be sent**.
- **Ssize** – Size of the send window.

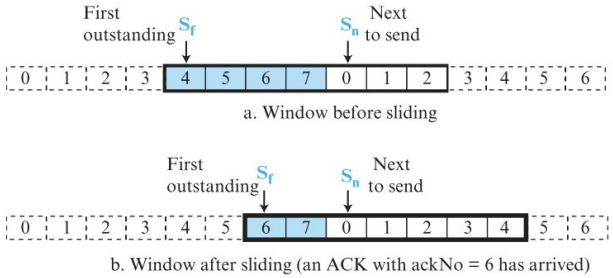
The send window slides to the right when cumulative acknowledgments are received.

Figure 3.24 *Send window for Go-Back-N*



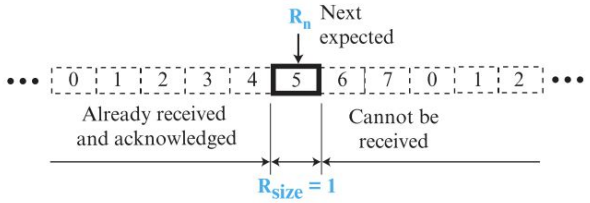
The send window is an abstract concept defining an imaginary box of maximum size = $2^m - 1$ with three variables: S_f, S_n, and S_{size}.

Figure 3.25 *Sliding the send window*



The send window can slide one or more slots when an error-free ACK with ackNo greater than or equal S_f and less than S_n (in modular arithmetic) arrives.

Figure 3.26 *Receive window for Go-Back-N*



The receive window is an abstract concept defining an imaginary box of size 1 with one single variable R_n. The window slides when a correct packet has arrived; sliding occurs one slot at a time.

Receive Window

The **receive window** ensures that packets are received in the correct order.

Characteristics

- Receive window size is always **1**.
- The receiver accepts **only the expected packet**.
- Out-of-order packets are discarded.
- Only one variable, **Rn**, is used.

Receive Window Variable

- **Rn** – Sequence number of the **next packet expected**.

When the expected packet is received correctly:

Rn = (Rn + 1) mod 2^m

The receive window slides **one position** at a time.

Timer

- Although a separate timer can be used for each packet, the Go-Back-N protocol uses **only one timer**, which is associated with the **first outstanding packet**.

If this timer expires, all outstanding packets are retransmitted.

Resending Packets

- When a timeout occurs, the sender **goes back** to the first unacknowledged packet and retransmits **all outstanding packets**.

Example:
If:

- **Sf = 3**
- **Sn = 7**

then packets **3, 4, 5, and 6** are retransmitted.

- This retransmission of all outstanding packets gives the protocol its name **Go-Back-N**.

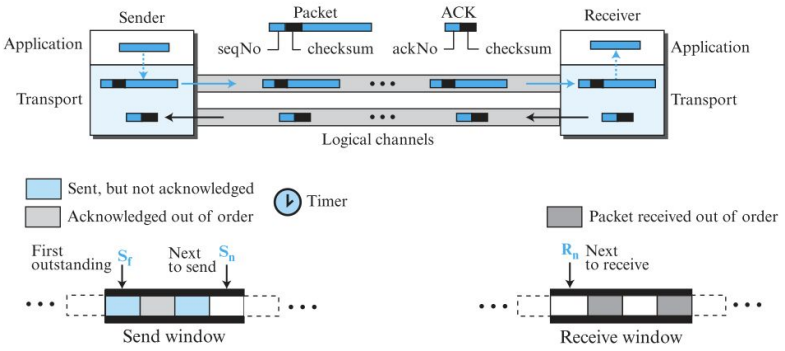
Key Points

- *Connection-oriented protocol.*
- *Allows multiple outstanding packets.*
- *Uses **cumulative acknowledgments**.*
- *Sequence numbers use **Modulo 2m2m** arithmetic.*
- *Maximum send window size = **2m−12m−1**.*
- *Receive window size = **1**.*
- *Uses one timer for the first outstanding packet.*
- *On timeout, all outstanding packets are retransmitted (Go-Back-N).*

Selective-Repeat (SR) Protocol

- In **Go-Back-N (GBN)**, if one packet is lost or corrupted, the sender retransmits **all outstanding packets**, even if some have already been received correctly.
- This causes **unnecessary retransmissions**, wastes bandwidth, increases congestion, and may lead to network collapse under heavy packet loss.
- Selective-Repeat (SR) retransmits **only the packets that are lost or corrupted**.
- The receiver accepts and stores **out-of-order packets** until all preceding packets are received.
- Packets are delivered to the application layer **only in order**.

Figure 3.31 *Outline of Selective-Repeat*



Flow Diagram for Go-Back-N Protocol (Figure 3.30)

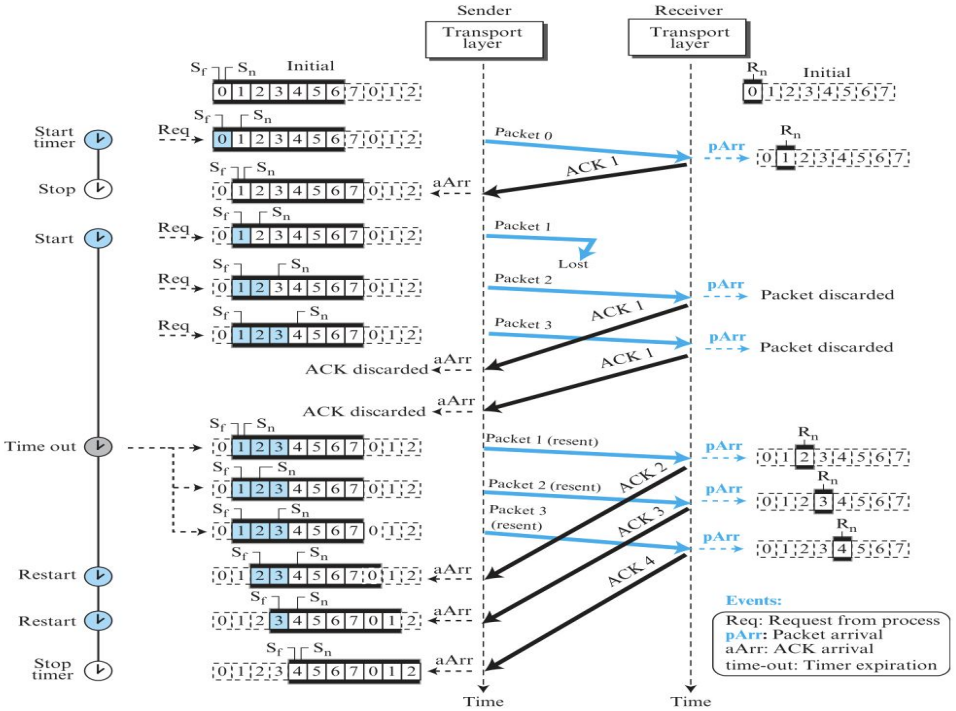
This diagram shows what happens when a packet is **lost** during transmission.

What happens step by step:

1. **Packet 0 sent** → Received correctly → Receiver sends **ACK 1** → Sender stops timer, slides window forward.
2. **Packet 1 sent** → **Lost** in the network → Receiver never gets it.
3. **Packets 2 & 3 sent** → Receiver gets them but **discards** both (because Packet 1 is missing — must be in order) → Sends **ACK 1** again (both times) → Sender discards both ACKs (already moved on).
4. **Timer expires (Time-out)** → Sender goes back and **resends Packets 1, 2, and 3** all over again.
5. **Resent packets received correctly** → Receiver accepts them in order → Sends ACK 2, ACK 3, ACK 4 → Sender restarts timer for each, window slides forward → **Stop timer**.

- In Go-Back-N, if one packet is lost, the sender must **retransmit that packet AND all packets sent after it**, even if those were received correctly.

Figure 3.30 *Flow diagram for Example 3.8*

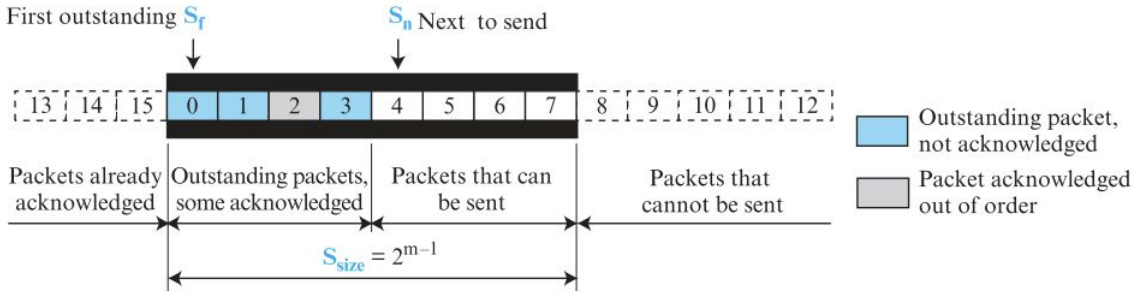


Windows

Send Window

- Maximum send window size = **2^(m−1)**, where **m** is the number of bits in the sequence number.
- Example: If **m = 4**, sequence numbers range from **0 to 15**, but the maximum window size is **8**.

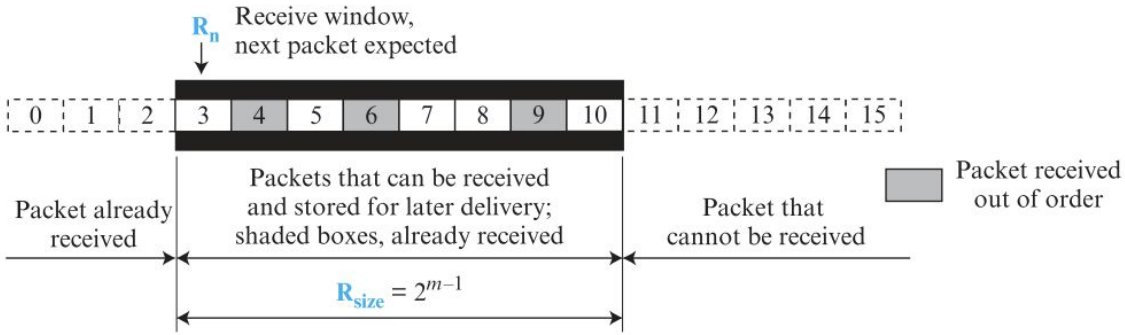
Figure 3.32 *Send window for Selective-Repeat protocol*



Receive Window

- The receive window size is **equal to the send window size** (maximum **2^(m−1)**).
- Out-of-order packets within the receive window are buffered until the missing packets arrive.
- The receiver **never delivers packets out of order** to the application layer.

Figure 3.33 *Receive window for Selective-Repeat protocol*



Timer

- Theoretically, **one timer is maintained for each outstanding packet**.
- When a timer expires, **only the corresponding packet is retransmitted**.
- In practice, many transport layer protocols implement SR using **a single timer**.

Acknowledgment

- In **Go-Back-N**, acknowledgments are **cumulative**. An ACK indicates that all previous packets have been received successfully.
- In **Selective-Repeat**, acknowledgments are **individual**. Each ACK corresponds to **one successfully received packet** and does not provide information about any other packet.

Bidirectional Protocols: Piggybacking

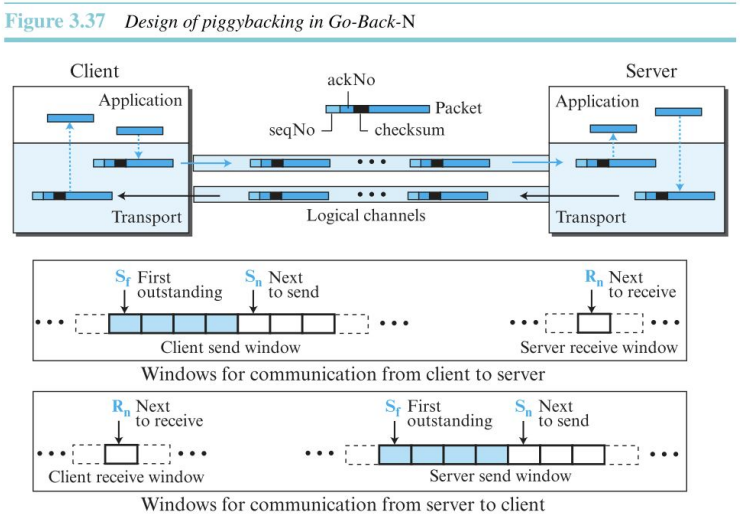
- The four protocols discussed earlier are **unidirectional**, where:
  - Data packets flow in one direction.
  - Acknowledgments (ACKs) flow in the opposite direction.
- In practical communication, data flows **in both directions** (client to server and server to client).

Piggybacking

- **Piggybacking** is a technique used to improve the efficiency of bidirectional protocols.
- A data packet sent from **A to B** carries:
  - Data from A to B.
  - Acknowledgment (ACK) for packets received from B.
- Similarly, a data packet sent from **B to A** carries:
  - Data from B to A.
  - Acknowledgment (ACK) for packets received from A.

Bidirectional GBN with Piggybacking

- In bidirectional **Go-Back-N (GBN)** protocol:
  - Both the **client** and **server** maintain two independent windows:
    - **Send window**
    - **Receive window**



Internet Transport-Layer Protocols

- The Internet uses several transport-layer protocols. The two main protocols are:
  - **User Datagram Protocol (UDP)**
  - **Transmission Control Protocol (TCP)**
- UDP and TCP are located **between the application layer and the network layer** in the TCP/IP protocol suite.
- They act as an **intermediary between application programs and network operations**.

User Datagram Protocol (UDP)

- **Connectionless** transport-layer protocol.
- **Unreliable** protocol.
- Used for its **simplicity and efficiency**.
- Suitable for applications where **error control is handled by the application layer**.

Transmission Control Protocol (TCP)

- **Connection-oriented** transport-layer protocol.
- **Reliable** protocol.
- Used for applications where **reliability is important**.

Responsibilities of the Transport Layer

- Provides **process-to-process communication**.
- Uses **port numbers** to achieve process-to-process communication.

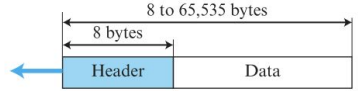
User Datagram Protocol (UDP)

- **UDP (User Datagram Protocol)** is a **connectionless** and **unreliable** transport-layer protocol.
- It provides **process-to-process communication** using **port numbers**.
- UDP is a **simple protocol** with **minimum overhead**.
- Suitable for sending **small messages** where reliability is not a major concern.

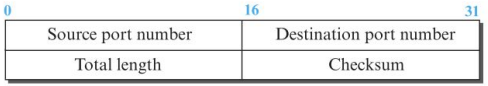
User Datagram

- UDP packets are called **User Datagrams**.
- The UDP header is **8 bytes** long and consists of **four 16-bit (2-byte) fields**:
  - Source Port Number
  - Destination Port Number
  - Length (Header + Data)
  - Checksum
- The maximum UDP datagram size is **65,535 bytes**.
- The maximum application data size is **65,507 bytes** (65,535 − 8-byte UDP header − 20-byte IP header).

Figure 3.39 *User datagram packet format*



a. UDP user datagram



b. Header format

UDP Services

Process-to-Process Communication

- Uses **socket addresses** (IP address + Port number) for process-to-process communication.

Connectionless Service

- Each user datagram is **independent**.
- No relationship exists between different user datagrams.
- No sequence numbers.
- No connection establishment or termination.
- Each datagram may travel through a **different path**.

Flow Control

- UDP **does not provide flow control**.
- No window mechanism.
- Flow control, if required, must be provided by the **application**.

Error Control

- UDP provides **no error control** except the **checksum**.
- Lost or duplicated packets are **not detected**.
- If the checksum detects an error, the datagram is **silently discarded**.
- Error control, if required, must be provided by the **application**.

Checksum

- The UDP checksum is calculated using:
  - Pseudoheader
  - UDP Header
  - Application Data
- The pseudoheader helps detect errors in the IP header.
- The **Protocol field value for UDP is 17**.

Congestion Control

- UDP **does not provide congestion control**.

Encapsulation and Decapsulation

- UDP encapsulates messages before transmission and decapsulates them at the receiver.
-

Queuing

- Queues are associated with **ports**.
- A process requests a port number from the operating system before communication.

Multiplexing and Demultiplexing

- UDP supports **multiplexing** (multiple processes use the same UDP protocol).
- UDP supports **demultiplexing** (delivers incoming datagrams to the correct process using port numbers).

Transmission Control Protocol (TCP)

- **Transmission Control Protocol (TCP)** is a **connection-oriented** and **reliable** transport-layer protocol.
- TCP explicitly defines three phases:
  - **Connection establishment**
  - **Data transfer**
  - **Connection termination**
- TCP provides reliability using:
  - Checksum (error detection)
  - Retransmission of lost or corrupted packets
  - Cumulative acknowledgments
  - Selective acknowledgments
  - Timers
- TCP is the **most commonly used transport-layer protocol** on the Internet.

TCP Services

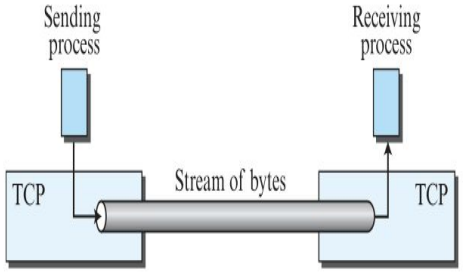
Process-to-Process Communication

- TCP provides **process-to-process communication** using **port numbers**.

Stream Delivery Service

- TCP is a **stream-oriented protocol**.
- The sending process delivers data as a **stream of bytes**.
- The receiving process obtains data as a **stream of bytes**.
- TCP creates an **imaginary "tube"** between the communicating processes, making them appear directly connected.
- Unlike UDP, TCP does **not preserve message boundaries**.

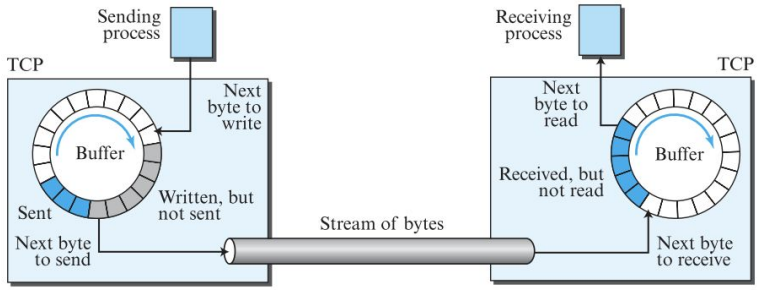
Figure 3.41 *Stream delivery*



Sending and Receiving Buffers

- TCP uses **sending** and **receiving buffers** in **both directions**.
- These buffers are required because the sending and receiving processes may operate at **different speeds**.
- Buffers are also necessary for **flow control** and **error control**.
- Buffers are commonly implemented as **circular arrays (circular buffers)**.
- **Sending buffer** stores:
    - Empty space for new data.
    - Bytes sent but not yet acknowledged.
    - Bytes waiting to be transmitted.
- Acknowledged bytes are removed and buffer space is **recycled**.
- **Receiving buffer** stores:
    - Empty space for incoming bytes.
    - Received bytes waiting to be read by the application.
- After the application reads the bytes, the buffer space is **recycled**.

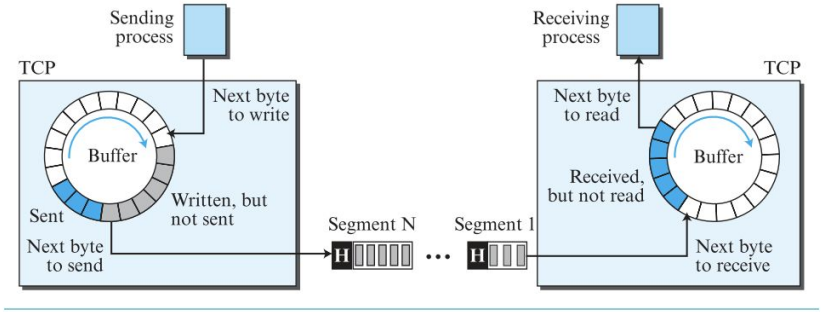
Figure 3.42 *Sending and receiving buffers*



Segments

- TCP groups bytes into packets called **segments**.
- Each segment contains a **TCP header** and data.
- Segments are passed to the network layer and **encapsulated into IP datagrams**.
- Segment creation and transmission are **transparent to the application process**.
- Segments:
  - May arrive **out of order**.
  - May be **lost**.
  - May be **corrupted**.
  - May be **retransmitted**.
- These operations are handled entirely by TCP.
- **Segment sizes are not fixed** and may contain **hundreds or thousands of bytes**.

Figure 3.43 *TCP segments*



Full-Duplex Communication

- TCP supports **full-duplex communication**.
- Data can flow **simultaneously in both directions**.
- Each TCP endpoint maintains its own:
    - Sending buffer
    - Receiving buffer

Multiplexing and Demultiplexing

- TCP performs **multiplexing** at the sender.
- TCP performs **demultiplexing** at the receiver.
- Since TCP is **connection-oriented**, a separate **connection** is established for each communicating pair of processes.

Reliable Service

- TCP is a **reliable transport protocol**.
- It uses an **acknowledgment mechanism** to ensure the successful delivery of data.

Connection-Oriented Service

- TCP establishes a **logical connection** before communication.
- Communication occurs in three phases:
    - Connection establishment
    - Data exchange
    - Connection termination
- The connection is **logical**, not physical.
- TCP segments are encapsulated in IP datagrams.
- Segments may:
    - Follow different routes.
    - Arrive out of order.
    - Be lost or corrupted.
- TCP ensures that the byte stream is delivered **in order** despite these conditions.

TCP Features

Numbering System

- TCP uses **Sequence Number** and **Acknowledgment Number** fields.
- Both fields refer to **byte numbers**, not segment numbers.

Byte Number

- Every transmitted byte is assigned a **byte number**.
- Numbering is **independent in each direction**.
- The first byte is assigned an **arbitrary number (0 to $2^{32} - 1$)**.
- Used for **flow control** and **error control**.

Sequence Number

- Indicates the **first byte number** in a segment.
- The first segment uses a **random Initial Sequence Number (ISN)**.
- Each subsequent segment's sequence number = **previous sequence number + bytes in the previous segment**.
- Control segments (connection establishment, termination, abortion) consume **one sequence number** as an **imaginary byte**.

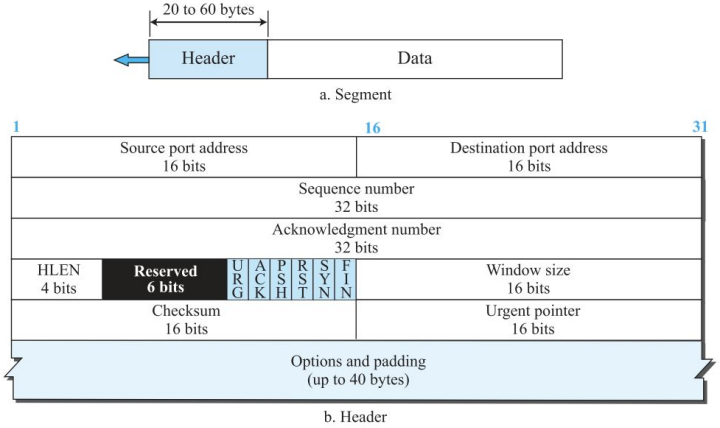
Acknowledgment Number

- Indicates the **next byte expected**.
- TCP acknowledgments are **cumulative**.
- Acknowledgment Number = **Last correctly received byte + 1**.
- Confirms that **all previous bytes have been received successfully**.

TCP Segment

- A **TCP packet** is called a **segment**.
- A TCP segment consists of:
  - **Header:** 20–60 bytes
  - **Data:** Application data

Figure 3.44 *TCP segment format*



TCP Header Fields

- **Source Port (16 bits):** Port number of the sending application.
- **Destination Port (16 bits):** Port number of the receiving application.
- **Sequence Number (32 bits):** Number of the **first byte** in the segment.
- **Acknowledgment Number (32 bits):** Number of the **next byte expected** from the sender.
- **Header Length (4 bits):** Specifies the header size (**20–60 bytes**).
- **Control Flags:** Used for **connection establishment, termination, flow control, connection abortion, and data transfer**.
- **Window Size (16 bits):** Specifies the receiver's window size (maximum **65,535 bytes**).
- **Checksum (16 bits):** Used for **error detection**. **Mandatory** in TCP.
- **Urgent Pointer (16 bits):** Indicates the **last urgent byte** (valid only when the urgent flag is set).
- **Options:** Optional field of up to **40 bytes**.

TCP Connection

- TCP is a **connection-oriented** protocol that establishes a **logical connection** between the sender and receiver.
- TCP controls the connection, while **IP only delivers individual segments**.
- If a segment is **lost or corrupted**, TCP **retransmits** it.
- If segments arrive **out of order**, TCP stores them until the missing segments arrive.

Phases of a TCP Connection

1. **Connection Establishment**
2. **Data Transfer**
3. **Connection Termination**

Connection Establishment (Three-Way Handshaking)

Step 1: SYN

- Client sends a **SYN** segment to the server.
- Synchronizes sequence numbers.
- Contains the **Initial Sequence Number (ISN)**.
- Carries **no data**.
- **Consumes one sequence number**.

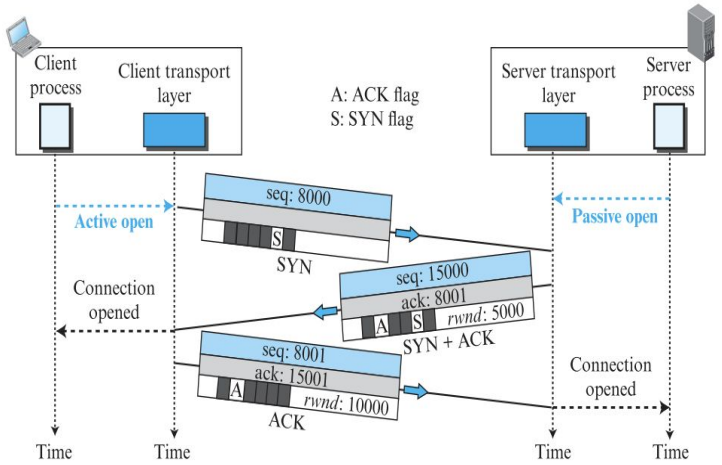
Step 2: SYN + ACK

- Server replies with a **SYN + ACK** segment.
- Acknowledges the client's SYN.
- Sends its own **ISN**.
- Specifies the **receive window size (rwnd)**.
- Carries **no data**.
- **Consumes one sequence number**.

Step 2: SYN + ACK

- Server replies with a **SYN + ACK** segment.
- Acknowledges the client's SYN.
- Sends its own **ISN**.
- Specifies the **receive window size (rwnd)**.
- Carries **no data**.
- **Consumes one sequence number**.

Figure 3.47 *Connection establishment using three-way handshaking*



TCP Three-Way Handshake (Figure 3.47)

- This diagram shows how a TCP connection is **established** between a Client and Server before data is sent.

3 Steps:

Step 1 — SYN (Client → Server)
Client sends a SYN packet with seq: 8000.
"Hey, I want to connect!"

Step 2 — SYN + ACK (Server → Client)
Server replies with SYN + ACK, seq: 15000, ack: 8001, window: 5000.
"OK, I'm ready. I got your message!"

Step 3 — ACK (Client → Server)
Client sends ACK, seq: 8001, ack: 15001, window: 10000.
"Great, connection confirmed!"

After these 3 steps, **both sides show "Connection Opened"** and data transfer can begin.

- The 3-way handshake is like a mutual agreement — Client says *hello*, Server says *hello back*, Client says *got it*— then they start talking.

Connection Termination (Three-Way Handshaking)

Step 1: FIN

- Client sends a **FIN** segment to close the connection.
- May carry the last data.
- If it carries no data, it **consumes one sequence number**.

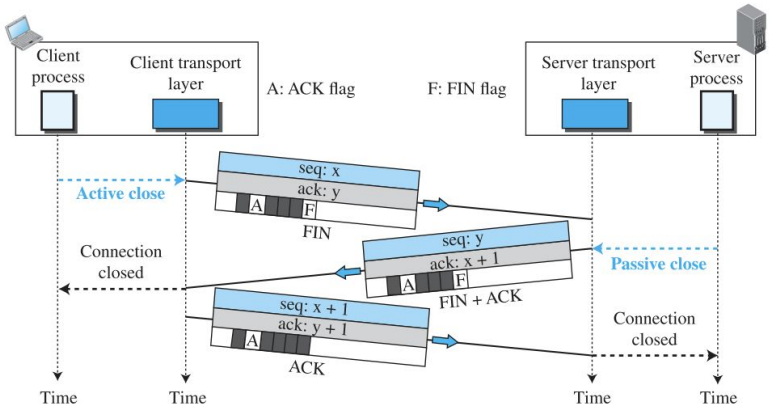
Step 2: FIN + ACK

- Server sends a **FIN + ACK** segment.
- Acknowledges the client's FIN.
- Announces closing of the connection.
- May carry the last data.
- If it carries no data, it **consumes one sequence number**.

Step 3: ACK

- Client sends an **ACK** segment.
- Acknowledges the server's FIN.
- Carries **no data**.
- **Does not consume a sequence number**.

Figure 3.49 *Connection termination using three-way handshaking*



TCP Connection Termination — Three-Way Handshake (Figure 3.49)

This diagram shows how a TCP connection is **closed** between a Client and Server.

3 Steps:

Step 1 — FIN (Client → Server)
Client sends a FIN + ACK packet (seq: x, ack: y).
"I'm done sending. I want to close!"

Step 2 — FIN + ACK (Server → Client)
Server replies with FIN + ACK (seq: y, ack: x+1).
"OK, I got it. I'm closing too!"
→ Client marks **Connection Closed**.

Step 3 — ACK (Client → Server)
Client sends a final ACK (seq: x+1, ack: y+1).
"Confirmed. Goodbye!"
→ Server marks **Connection Closed**.

- Just like the 3-way handshake *opens* a connection, this **closes** it — Client says *I'm done*, Server says *me too*, Client says *confirmed*. Both sides shut down cleanly.

Network Layer

The **Network Layer** is responsible for **host-to-host communication**. It ensures that data packets travel from the source to the destination across different networks.

Functions

- **Logical Addressing:** Assigns unique IP addresses to devices.
- **Packetizing:** Encapsulates data into IP packets.
- **Routing:** Finds the best path for packets.
- **Forwarding:** Sends packets to the next router or destination.
- **Congestion Control:** Reduces network traffic and improves performance.

*Internet = Collection of interconnected **LANs and WANs**.*

***Routers** connect different networks.*

*The Network Layer performs **encapsulation, routing, forwarding, and decapsulation** of packets.*

Introduction to the Network Layer

- The **Network Layer** is responsible for **host-to-host communication** across different networks.
- The Internet is an **internetwork**, made up of many **LANs (Local Area Networks)** and **WANs (Wide Area Networks)** connected through **routers**.
- The Network Layer is present in the **source host, destination host, and all routers** along the communication path.

At the Source Host

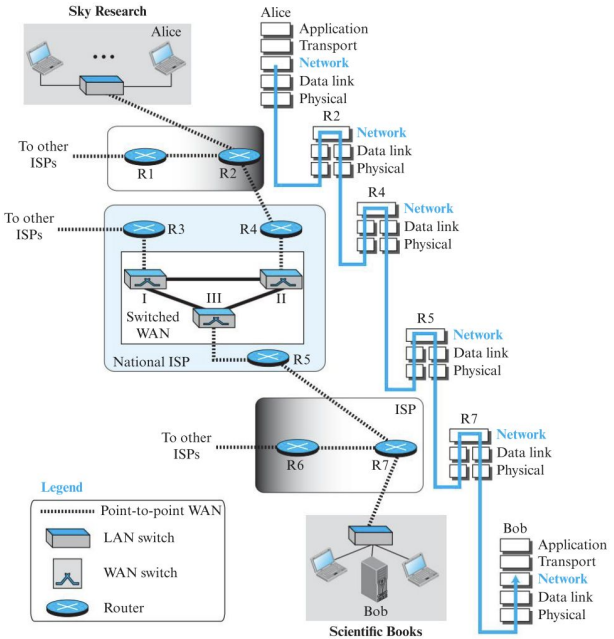
- Receives a packet from the **Transport Layer**.
- **Encapsulates** the packet into an **IP datagram**.
- Sends the datagram to the **Data Link Layer**.

At the Routers

- Receive the datagram.
- **Forward (route)** it to the next network using the best path.
- Routers mainly use the **Network, Data Link, and Physical Layers**.

At the Destination Host

- **Decapsulates** the datagram.
- Extracts the original packet.
- Delivers it to the **Transport Layer**.



It's active at **3 points**: (DIAGRAM)

1. **Sender (Alice)** — Takes data from the transport layer, wraps it in a **datagram**, and sends it down
2. **Routers (middle)** — Pass the datagram from one network to the next *(they only need 3 layers to do this)*
3. **Receiver (Bob)** — Unwraps the datagram and delivers the data to the transport layer

This diagram shows how **data travels from Alice to Bob** across the Internet.

The Internet is made up of multiple networks — LANs (local networks) and WANs (wide-area networks) — connected by **routers (R1–R7)**.

The path of data:

1. **Alice (sender)** — uses **all 5 TCP/IP layers** (Application → Transport → Network → Data Link → Physical)
2. **Routers R2, R4, R5, R7** — only use the **bottom 3 layers** (Network, Data Link, Physical) to forward the data
3. **Bob (receiver)** — receives the data through **all 5 layers** again

Key networks shown:

- **Sky Research** (Alice's LAN)
- **National ISP** (Switched WAN with routers R3, R4, R5)
- **ISP** (with routers R6, R7)
- **Scientific Books** (Bob's LAN)

The Network layer is the key layer that connects everything — it's active at every router and both endpoints along the path.

4.1 Network-Layer Services

The network layer provides several services to ensure the successful delivery of packets from the source to the destination.

1. Packetizing

- The primary function of the network layer is **packetizing**.
- At the **source**, it receives data (payload) from the transport layer, **adds a header** containing the source and destination IP addresses and other control information, and creates a **datagram**.
- The datagram is then passed to the data-link layer.
- At the **destination**, the datagram is **decapsulated**, and the original payload is delivered to the transport layer.
- If a packet is too large, it is **fragmented** during transmission and **reassembled** at the destination.
- Routers do not modify the payload or IP addresses; they only inspect the header for forwarding.

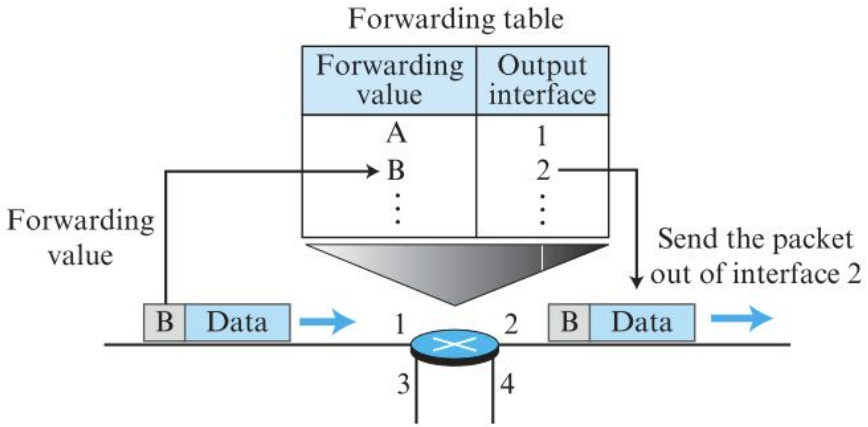
2. Routing

- Routing is the process of **finding the best path** from the source to the destination.
- Since the Internet consists of many interconnected LANs, WANs, and routers, multiple paths may exist.
- The network layer uses **routing protocols** to exchange routing information and build **routing tables**.
- These routing tables are used to determine the best route before data transmission begins.

3. Forwarding

- **Forwarding** is the action performed by a router when it receives a packet.
- The router examines the **destination IP address** (or label) in the packet header.
- Using the **forwarding (routing) table**, it selects the appropriate output interface and forwards the packet to the next network.
- In **unicast**, the packet is forwarded to one interface, whereas in **multicast**, it may be forwarded to multiple interfaces.

Figure 4.2 *Forwarding process*



4. Error Control

- The Internet's network layer **does not provide complete error control** for data.
- It checks only the **header** using a **header checksum** to detect header corruption.
- If an error occurs or a packet cannot be delivered, **ICMP (Internet Control Message Protocol)** is used to report the error.
- End-to-end error control is mainly handled by the **transport layer**.

5. Flow Control

- The network layer **does not provide flow control**.
- Packets are transmitted without considering whether the receiver is ready.
- Flow control is handled by the **transport layer**, which prevents the receiver from being overwhelmed.

6. Congestion Control

- **Congestion** occurs when the number of packets in the network exceeds its capacity.
- This may cause routers to **drop packets**, leading to retransmissions and reduced network performance.
- Severe congestion can even result in **network collapse**.
- Although congestion control is an important issue, it is **not directly implemented** in the Internet's network layer.

7. Quality of Service (QoS)

- QoS ensures better network performance for **real-time applications** such as voice and video.
- It aims to provide reliable bandwidth, low delay, and reduced packet loss.
- Most QoS functions are implemented in the **upper layers** rather than the network layer.

8. Security

- The original network layer was designed **without security features**.
- Modern networks use **IPSec (Internet Protocol Security)** to provide:
  - Authentication
  - Data integrity
  - Encryption
- IPSec improves secure communication over IP networks.

Packetizing: Encapsulation, Decapsulation, Fragmentation, Reassembly.

Routing: Finds the best path using routing protocols.

Forwarding: Uses the forwarding/routing table to send packets.

Error Control: Header checksum; ICMP reports errors.

Flow Control: Not provided by the network layer.

Congestion Control: Prevents excessive network traffic.

QoS: Improves multimedia communication.

Security: Provided using IPSec.

4.1.2 Packet Switching

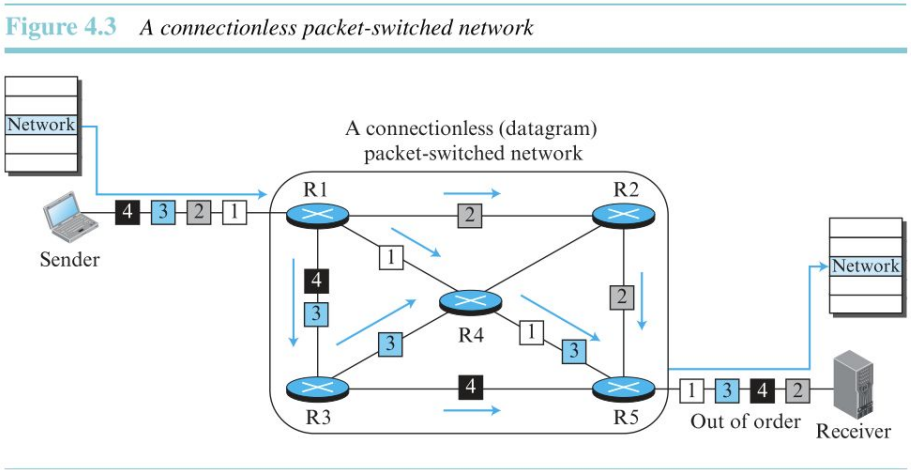
- **Packet switching** is the switching technique used at the **Network Layer**.
- A **router** acts as a switch, forwarding packets from an **input port** to an **output port**.
- A message is divided into **packets**, which are transmitted individually through the network.
- The destination receives all packets and **reassembles** them before delivering the message to the upper layer.
- Packet-switched networks use two approaches:
    1. **Datagram (Connectionless) Approach**
    2. **Virtual Circuit (Connection-Oriented) Approach**

1. Datagram Approach (Connectionless Service)

- Each packet is treated as an **independent entity**.
- **No connection** is established before transmission.
- Packets of the same message may travel through **different paths**.
- Each packet contains **source and destination addresses**.
- Routers forward packets using the **destination address**.
- The **source address** is mainly used for reporting errors (e.g., ICMP).
- This approach is used in the **Internet (IP)**.

Characteristics

- No setup phase.
- Different packets may take different routes.
- Packets may arrive **out of order**.
- Simple and efficient.



2. Virtual Circuit Approach (Connection-Oriented Service)

- A **virtual connection** is established before data transmission.
- All packets of the same message follow the **same path**.
- Each packet carries a **Virtual Circuit Identifier (VCI) / Label**.
- Routers forward packets based on the **label**, not the destination address.
- Packets arrive **in order**.

Three Phases

1. Setup Phase

- Establishes the virtual circuit.
- Source sends a **Request Packet**.
- Routers create **forwarding table entries**.
- Destination sends an **Acknowledgment Packet**.
- Labels are assigned for the virtual circuit.

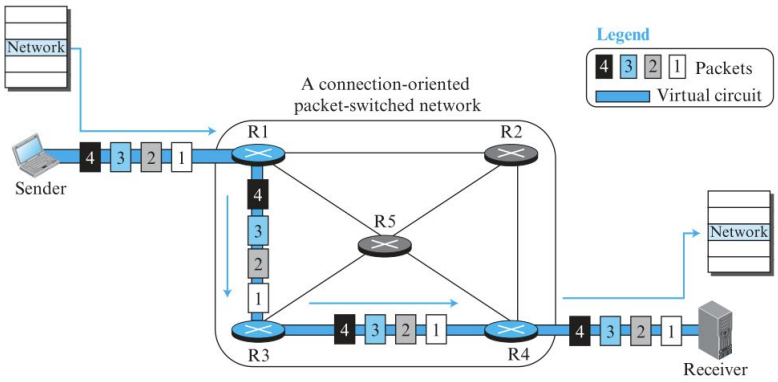
2. Data Transfer Phase

- Packets are transmitted using the assigned **VCI/Label**.
- Routers update the label at each hop.
- All packets follow the **same route**.

3. Teardown Phase

- After transmission, the source sends a **Teardown Packet**.
- Routers remove the virtual circuit entries from their forwarding tables.

Figure 4.5 *A virtual-circuit packet-switched network*



Datagram Approach

- Connectionless
- No setup required
- Packets may take different paths
- Forwarding based on destination address
- Packets may arrive out of order

Virtual Circuit Approach

- Connection-oriented
- Setup required
- All packets follow the same path
- Forwarding based on VCI/Label
- Packets arrive in order

4.1.3 Network-Layer Performance

The performance of a network is measured using three parameters:

- **Delay**
- **Throughput**
- **Packet Loss**

1. Delay

Delay is the time taken for a packet to travel from the source to the destination.

Types of Delay

a) Transmission Delay

- Time required to place all bits of a packet onto the transmission medium.
- Depends on **packet length** and **transmission rate**.

Formula:

Delaytr = (Packet length) / (Transmission rate).

b) Propagation Delay

- Time taken by a bit to travel from the sender to the receiver.
- Depends on **distance** and **propagation speed**.

Formula:Delaypg = (Distance) / (Propagation speed).

c) Processing Delay

- Time required by a router or destination host to:
    - Process the packet header.
    - Check for errors.
    - Forward or deliver the packet.

Delaypr = Time required to process a packet in a router or a destination host

d) Queuing Delay

- Time a packet waits in the **input** and **output queues** of a router before processing or transmission.
- Depends on **network traffic**.

  Delayqu = The time a packet waits in input and output queues in a router

d) Queuing Delay

- Time a packet waits in the **input** and **output queues** of a router before processing or transmission.
- Depends on **network traffic**.

Total Delay

For a path with **n routers**:Total delay = (n + 1) (Delaytr + Delaypg + Delaypr) + (n) (Delayqu)

where:

- **Dtr** = Transmission Delay
- **Dpg** = Propagation Delay
- **Dpr** = Processing Delay
- **Dqu** = Queuing Delay

2. Throughput

- **Throughput** is the **number of bits transmitted per second** through a network.
- In a path with multiple links, the throughput is determined by the **slowest link (bottleneck)**.

Formula:

Throughput = minimum {TR1, TR2, . . . TRn}.

where **TR** = Transmission Rate.

3. Packet Loss

- **Packet loss** occurs when a router's **buffer becomes full**.
- Excess packets are **dropped**.
- Lost packets must be **retransmitted**, which increases network congestion and reduces performance.

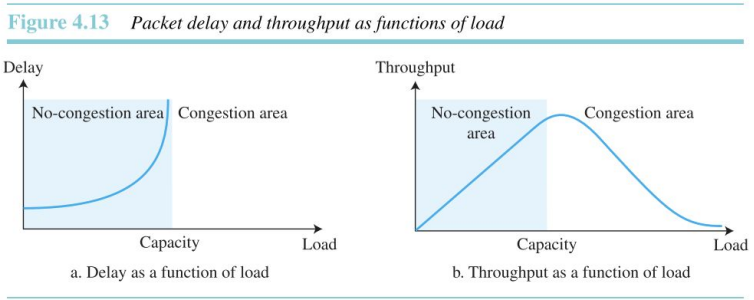
4.1.4 Network-Layer Congestion

Network-Layer Congestion

Network-layer congestion occurs when the **network load exceeds its capacity**. As the number of packets in the network increases, routers become overloaded, resulting in **increased delay, packet loss, and reduced throughput**.

Effects of Congestion

- When the network load is **below its capacity**, delay is very low and throughput increases with the load.
- When the load **reaches network capacity**, **queuing delay** increases rapidly.
- When the load **exceeds network capacity**:
  - Router queues become full.
  - Packets are discarded.
  - Sources retransmit lost packets.
  - Delay increases significantly.
  - Throughput decreases due to retransmissions.



Congestion Control

Congestion control refers to the techniques used to **prevent congestion before it occurs** or **remove congestion after it occurs**.

It is classified into two types:

- **Open-Loop Congestion Control (Prevention)**
- **Closed-Loop Congestion Control (Removal)**

1. Open-Loop Congestion Control (Prevention)

Open-loop congestion control applies preventive policies before congestion occurs.

a) Retransmission Policy

- Controls packet retransmissions.
- Efficient timeout and retransmission mechanisms reduce unnecessary retransmissions and network congestion.

b) Window Policy

- The sender's window protocol affects congestion.
- **Selective Repeat** is preferred over **Go-Back-N** because it retransmits only lost packets instead of all packets after the lost one.

c) Acknowledgment Policy

- Reduces the number of acknowledgments sent by the receiver.
- Fewer acknowledgments reduce network traffic and help prevent congestion.

d) Discarding Policy

- Routers discard selected packets during congestion.
- Discarding less important packets helps reduce network load while maintaining communication quality.

e) Admission Policy

- Used mainly in **virtual-circuit networks**.
- Before establishing a connection, the router checks whether sufficient resources are available.
- If congestion is likely, the connection request is rejected.

2. Closed-Loop Congestion Control (Removal)

Closed-loop congestion control detects congestion and takes corrective action after congestion occurs.

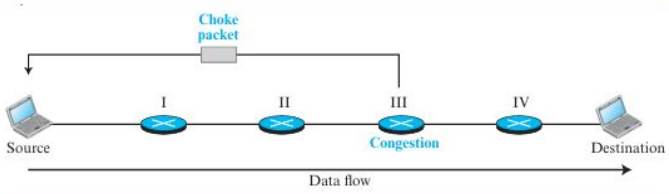
a) Backpressure

- A congested router informs its **upstream router** to slow down transmission.
- The message propagates back toward the source until congestion is relieved.
- Applicable only in **virtual-circuit networks**.

b) Choke Packet

- A congested router sends a **special control packet (cho** directly to the source.
- The source reduces its transmission rate.
- Intermediate routers do not take any action.

Figure 4.15 *Choke packet*



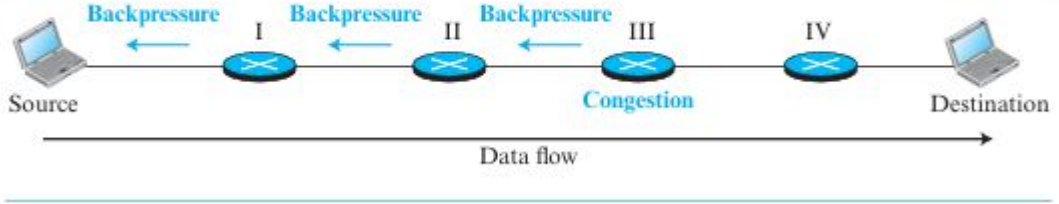
c) Implicit Signaling

- No explicit congestion message is sent.
- The sender detects congestion from **packet loss**, **timeouts**, or **delayed acknowledgments** and reduces its transmission rate.

d) Explicit Signaling

- Routers explicitly notify congestion by including congestion information in data packets.
- The notification may be sent to the source or destination, allowing the sender to reduce its transmission rate.

Figure 4.14 *Backpressure method for alleviating congestion*



Structure of a Router

A **router** forwards packets from an input port to the correct output port using a forwarding table. It has **four main components**:

1. Input Port

- Receives incoming packets.
- Performs **Physical Layer** and **Data Link Layer** functions.
- Checks for errors and removes the frame (decapsulation).
- Stores packets temporarily in **buffers (queues)** before forwarding.

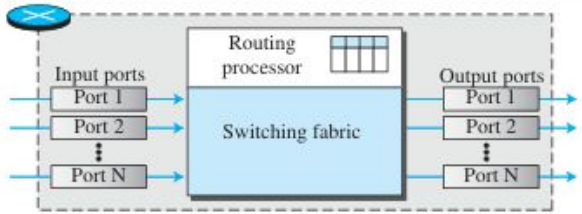
2. Output Port

- Sends packets to the destination network.
- Stores outgoing packets in queues.
- Encapsulates packets into frames.
- Performs Physical Layer functions to transmit signals.

3. Routing Processor

- Performs **Network Layer** functions.
- Looks up the forwarding table to determine:
  - Next-hop address
  - Correct output port
- Modern routers often perform this lookup at the input port for faster processing.

Figure 4.16 *Router components*



4. Switching Fabric

- Transfers packets from input ports to output ports.
- Its speed affects router performance and packet delay.

Types of Switching Fabric

a) Crossbar Switch

- Connects **n inputs** to **n outputs** using a grid of electronic switches.
- Simple and allows direct connections.

b) Banyan Switch

- A **multistage switching network**.
- Routes packets based on the binary representation of the output port.
- Faster and more efficient than a crossbar but may suffer from **internal collisions**.

c) Batcher-Banyan Switch

- Combines a **Batcher sorting switch** with a **Banyan switch**.
- Sorts packets before routing to reduce collisions.
- A **trap module** prevents multiple packets with the same destination from entering the Banyan switch simultaneously.

- ***Input Port:*** *Receives and processes packets.*
- ***Output Port:*** *Sends packets.*
- ***Routing Processor:*** *Determines the forwarding path.*
- ***Switching Fabric:*** *Moves packets between ports efficiently.*

Network-Layer Protocols

The **Network Layer** in the TCP/IP protocol suite is responsible for **packetizing, forwarding, and delivering data packets** across networks.

Protocols in the Network Layer

1. **IPv4 (Internet Protocol Version 4)**
    - Main network-layer protocol.
    - Performs **packetizing, routing, forwarding, and delivery** of packets.
    - **Connectionless** and **unreliable** (best-effort delivery).
2. **ICMPv4 (Internet Control Message Protocol)**
    - Supports IPv4 by reporting **errors** and providing diagnostic messages.
3. **IGMP (Internet Group Management Protocol)**
    - Supports **multicasting** by managing group memberships.
4. **ARP (Address Resolution Protocol)**
    - Maps **IP addresses** to **MAC addresses**, connecting the network layer with the data-link layer.

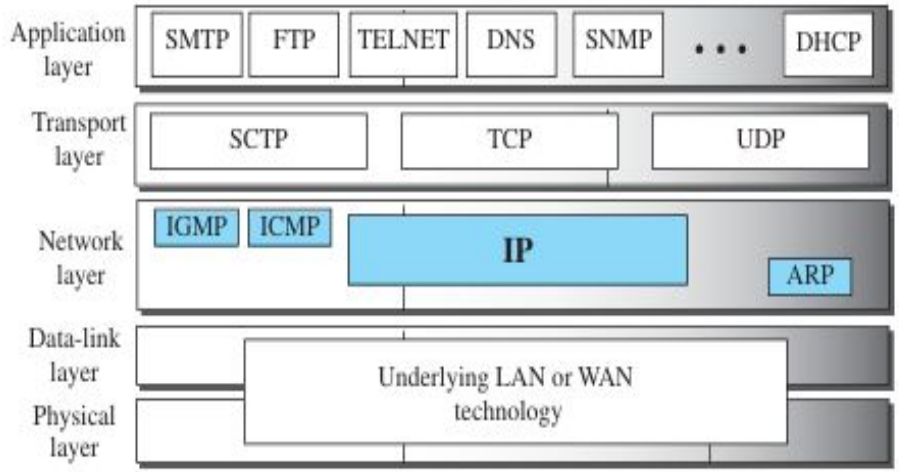
Features of IPv4

- **Connectionless Protocol:** Each packet (datagram) is sent independently.
- **Best-Effort Delivery:** No guarantee that packets will arrive correctly, in order, or at all.
- Packets may be **lost, delayed, corrupted, or arrive out of order**.
- Reliability is provided by higher-layer protocols like **TCP**.

Functions of IPv4

- Encapsulates data into packets.
- Assigns and uses IP addresses.
- Routes and forwards packets to their destination.

Figure 4.23 *Position of IP and other network-layer protocols in TCP/IP protocol suite*



IPv4: Packet delivery and routing.

ICMPv4: Error reporting and diagnostics.

IGMP: Multicasting support.

ARP: IP-to-MAC address mapping.

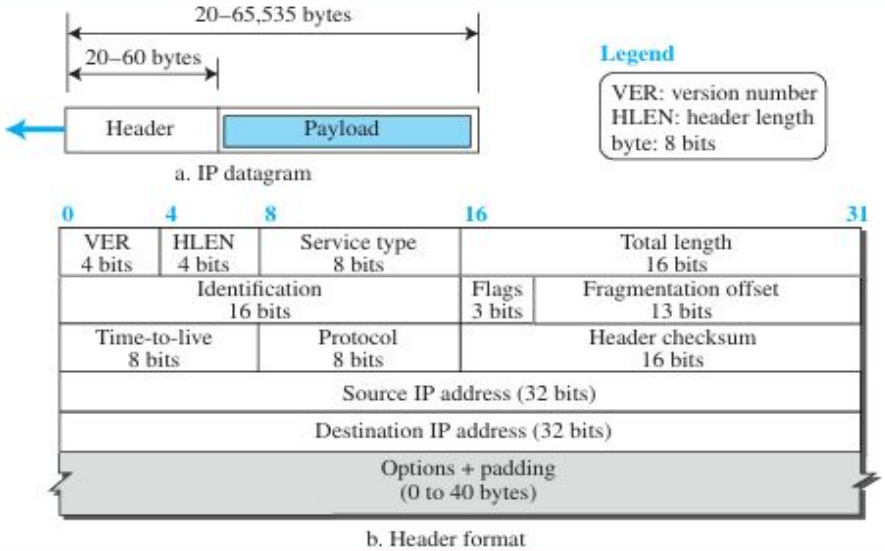
IPv4 is connectionless, unreliable, and provides best-effort delivery.

IPv4 Datagram Format

An **IPv4 datagram** is a **variable-length packet** consisting of:

- **Header:** 20–60 bytes
- **Payload (Data):** Actual data from upper-layer protocols

Figure 4.24 *IP datagram*



Field	Description
Version (4 bits)	Specifies the IP version (IPv4 = 4).
Header Length (HLEN)	Indicates the header size in 4-byte words (20–60 bytes).
Service Type (ToS/DiffServ)	Specifies the quality of service (QoS) or priority of the packet.
Total Length	Total size of the datagram (header + data), maximum 65,535 bytes.
Identification	Identifies fragments belonging to the same original datagram.
Flags	Controls fragmentation (e.g., whether fragmentation is allowed).
Fragment Offset	Indicates the position of a fragment in the original datagram.
Time To Live (TTL)	Limits the number of routers (hops) a packet can pass through to prevent looping.
Protocol	Identifies the upper-layer protocol (e.g., TCP = 6, UDP = 17).
Header Checksum	Detects errors in the IP header only.
Source Address	32-bit IP address of the sender.
Destination Address	32-bit IP address of the receiver.
Options	Optional field used for testing, debugging, and special functions (0–40 bytes).
Payload (Data)	Actual data received from upper-layer protocols (TCP, UDP, etc.).

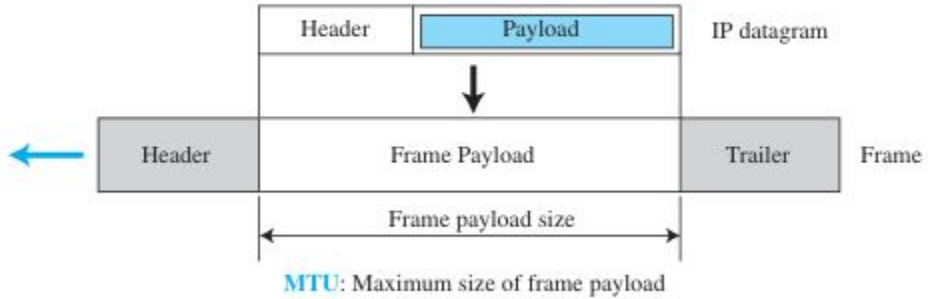
Fragmentation

An IP datagram may travel through different networks. Each network uses its own **link-layer frame format** and has a limit on the maximum size of data that can be carried in a frame. If an IP datagram is larger than the network can handle, it must be **fragmented** into smaller pieces.

Maximum Transfer Unit (MTU)

- **MTU (Maximum Transfer Unit)** is the maximum amount of data that a network can carry in a single frame.
- Different networks have different MTU values.
    - Example: **Ethernet (LAN) MTU = 1500 bytes**
- IPv4 allows a maximum datagram size of **65,535 bytes**.
- If a datagram is larger than the MTU of a network, it is divided into smaller fragments. This process is called **fragmentation**.

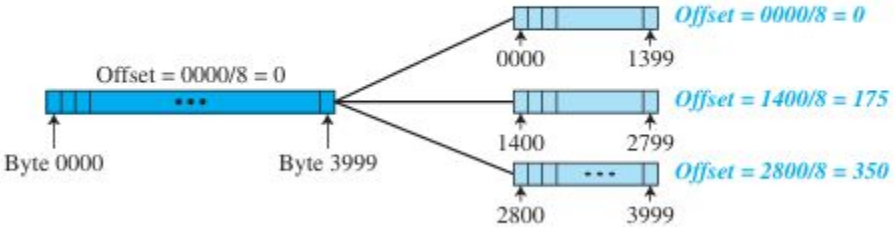
Figure 4.26 *Maximum transfer unit (MTU)*



Fragmentation Process

- Fragmentation can be performed by the **source host** or by any **router** along the transmission path.
- Each fragment becomes an **independent IP datagram** with its own header.
- Most header fields are copied into every fragment, while some fields are modified.
- A fragment may itself be fragmented again if it enters another network with a smaller MTU.
- **Reassembly is performed only at the destination host**, because fragments may travel through different routes and arrive out of order.
- After fragmentation, the **Header Checksum** is recalculated for every fragment.

Figure 4.27 *Fragmentation example*



3. Fields Related to Fragmentation

a) Identification Field (16 bits)

- Identifies the original datagram.
- All fragments of the same datagram carry the **same Identification value**.
- Helps the destination host reassemble the fragments correctly.

b) Flags Field (3 bits)

The flags field contains three bits:

- **Reserved bit:** Not used.
- **DF (Don't Fragment) bit**
  - **0** → Datagram can be fragmented.
  - **1** → Datagram must not be fragmented. If fragmentation is required, the router discards the datagram and sends an **ICMP error message**.
- **MF (More Fragments) bit**
  - **1** → More fragments follow.
  - **0** → This is the last fragment (or the only fragment).

c) Fragment Offset Field (13 bits)

- Indicates the position of a fragment in the original datagram.
- Offset is measured in **units of 8 bytes**.
- Used by the destination host to place fragments in the correct order during reassembly.

4. Fragment Offset Calculation

The fragment offset is calculated as follows:

1. **First fragment:** Offset = **0**.
2. **Second fragment:** Offset = **(Length of first fragment's data) ÷ 8**.
3. **Third fragment:** Offset = **(Length of first + second fragments' data) ÷ 8**.
4. Continue the same process for all remaining fragments.
5. The **last fragment** is identified by setting the **MF (More Fragments) bit = 0**.

Example

A datagram containing **4000 bytes** of data is fragmented into three parts:

Fragment	Data Bytes	Offset	MF Bit
Fragment 1	0–1399	0	1
Fragment 2	1400–2799	175 (1400 ÷ 8)	1
Fragment 3	2800–3999	350 (2800 ÷ 8)	0

5. Reassembly

- Fragments may arrive **out of order**.
- The destination host uses the **Identification**, **Fragment Offset**, and **MF bit** to reconstruct the original datagram.
- The **first fragment** always has an **offset of 0**.
- The **last fragment** is identified by **MF = 0**.
- Reassembly is successful only if **all fragments arrive**; if any fragment is lost, the original datagram cannot be reconstructed.

MTU *determines the maximum packet size a network can carry.*

Fragmentation *occurs when a datagram exceeds the MTU.*

Source hosts and routers *can fragment datagrams.*

Only the destination host *reassembles the fragments.*

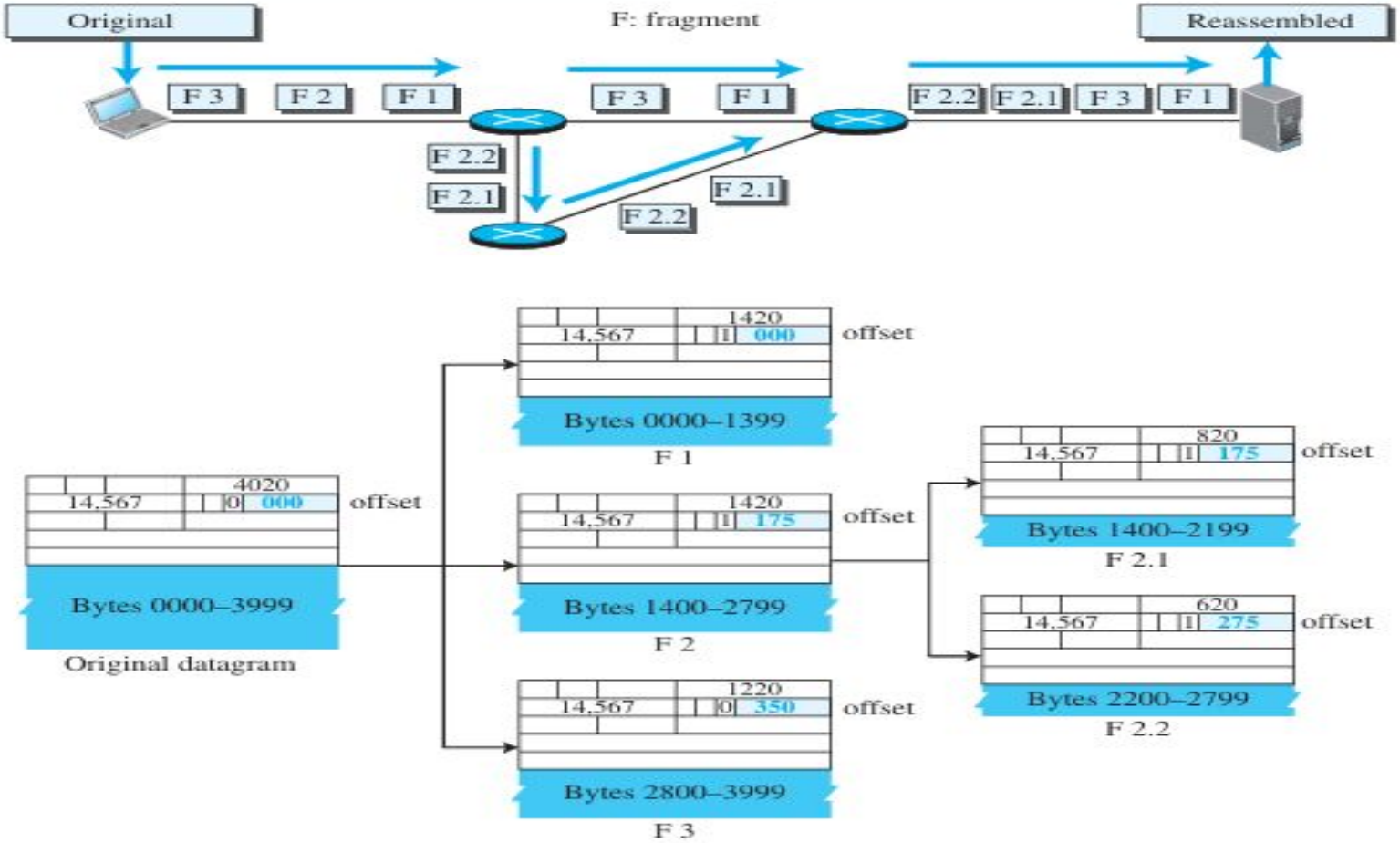
Fragmentation uses three important fields:

Identification

Flags (DF and MF)

Fragment Offset

Figure 4.28 *Detailed fragmentation example*



Security of IPv4 Datagrams

Originally, **IPv4** was designed without security because Internet users trusted each other. Today, the Internet is vulnerable to various attacks. The three major security threats to IPv4 are **packet sniffing, packet modification, and IP spoofing**.

1. Packet Sniffing

- An attacker intercepts and copies an IP packet without changing its contents.
- It is a **passive attack**, making it difficult to detect.
- **Prevention:** Encrypting the packet ensures that even if it is intercepted, the attacker cannot read its contents.

2. Packet Modification

- An attacker intercepts a packet, changes its contents, and forwards it to the receiver.
- The receiver may believe the packet came from the original sender.
- **Prevention:** A **data integrity mechanism** detects any changes made to the packet during transmission.

3. IP Spoofing

- An attacker creates a fake IP packet using the **source IP address of another computer**.
- This makes the attacker appear as a legitimate sender.
- **Prevention: Origin authentication** verifies that the packet actually came from the claimed sender.

IPSec (IP Security)

IPSec is a security protocol used with IPv4 to protect IP packets from security attacks. It provides a secure communication channel between two devices.

Services Provided by IPSec

1. **Algorithm and Key Management**
   - The communicating devices agree on the encryption algorithms and security keys.
2. **Packet Encryption**
   - Encrypts packets to maintain confidentiality.
   - Protects against **packet sniffing**.
3. **Data Integrity**
   - Ensures that packets are not modified during transmission.
   - Detects and rejects altered packets.
   - Protects against **packet modification**.
4. **Origin Authentication**
   - Verifies the identity of the sender.
   - Protects against **IP spoofing**.

Security Threat	Description	Protection
Packet Sniffing	Packet is copied without modification	Encryption
Packet Modification	Packet contents are altered	Data Integrity
IP Spoofing	Fake source IP address is used	Origin Authentication

IPSec protects IPv4 communication by providing:

Encryption

Data Integrity

Origin Authentication

Secure algorithm and key management

IPv4 Addresses

An **IPv4 address** is a **32-bit address** used to uniquely identify the **connection** of a host or router to the Internet.

- It identifies the **network connection**, not the device itself.
- If a device is connected to different networks, it may have different IP addresses.
- A device with multiple network connections has **multiple IPv4 addresses**.
- IPv4 addresses are **unique** (each address identifies only one connection) and **universal** (accepted throughout the Internet).

1. Address Space

- The **address space** is the total number of possible IPv4 addresses.
- Since IPv4 uses **32 bits**, the address space is:

$$2^{32} = 4,294,967,296$$

- Thus, IPv4 supports **more than 4 billion unique addresses**.

2. IPv4 Address Notations

IPv4 addresses can be represented in three formats:

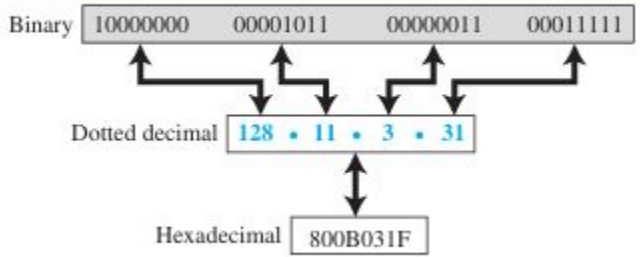
a) Binary Notation

- Written as **32 binary bits**.
- Usually divided into **four 8-bit groups (octets)**.

Example:

11000000 10101000 00000001 00000001

Figure 4.29 *Three different notations in IPv4 addressing*



b) Dotted-Decimal Notation

- Most commonly used format.
- Four decimal numbers separated by dots.
- Each octet ranges from **0 to 255**.

Example:

192.168.1.1

c) Hexadecimal Notation

- Written using **8 hexadecimal digits**.
- Commonly used in network programming.

Example:

C0A80101

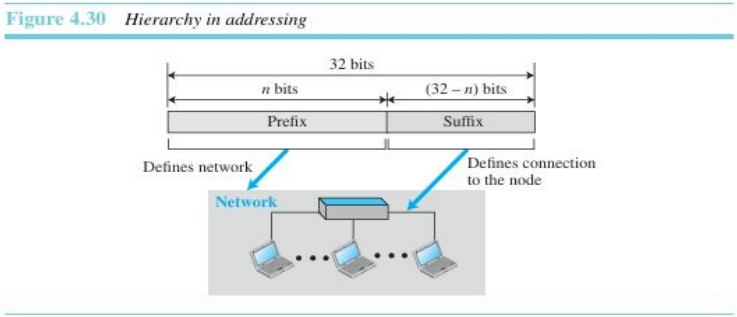
3. Hierarchical Addressing

An IPv4 address is divided into **two parts**:

- **Prefix (Network ID):** Identifies the network.
- **Suffix (Host ID):** Identifies the host (device) within the network.
- If the prefix length is **n bits**, the suffix length is **(32 − n) bits**.

Addressing Schemes

- **Classful Addressing:** Uses a **fixed-length prefix** (obsolete).
- **Classless Addressing (CIDR):** Uses a **variable-length prefix** and is the current addressing scheme.



- IPv4 address = **32 bits**.
- Identifies a **network connection**, not the device.
- Address space = **2^{32} = 4.29 billion addresses**.
- Three notations:
  - **Binary**
  - **Dotted-decimal**
  - **Hexadecimal**
- IPv4 addresses consist of:
  - **Prefix (Network ID)**
  - **Suffix (Host ID)**
- Modern networks use **Classless Addressing (CIDR)**.

Classful Addressing

When the Internet was first developed, IPv4 addresses used **fixed-length network prefixes**. To support networks of different sizes, the address space was divided into **five classes: A, B, C, D, and E**. This scheme is called **Classful Addressing**.

Class	Starting Bits	Network Prefix	Host Portion	Purpose	Example Address
A	0	8 bits (7 bits for Network ID)	24 bits	Large networks	10.20.30.40
B	10	16 bits (14 bits for Network ID)	16 bits	Medium-sized networks	172.16.25.100
C	110	24 bits (21 bits for Network ID)	8 bits	Small networks	192.168.1.10
D	1110	Not divided	Not divided	Multicasting	224.0.0.5
E	1111	Not divided	Not divided	Reserved / Experimental	240.0.0.1

Figure 4.30 *Hierarchy in addressing*

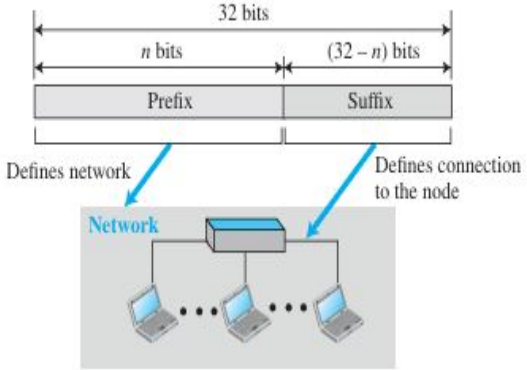
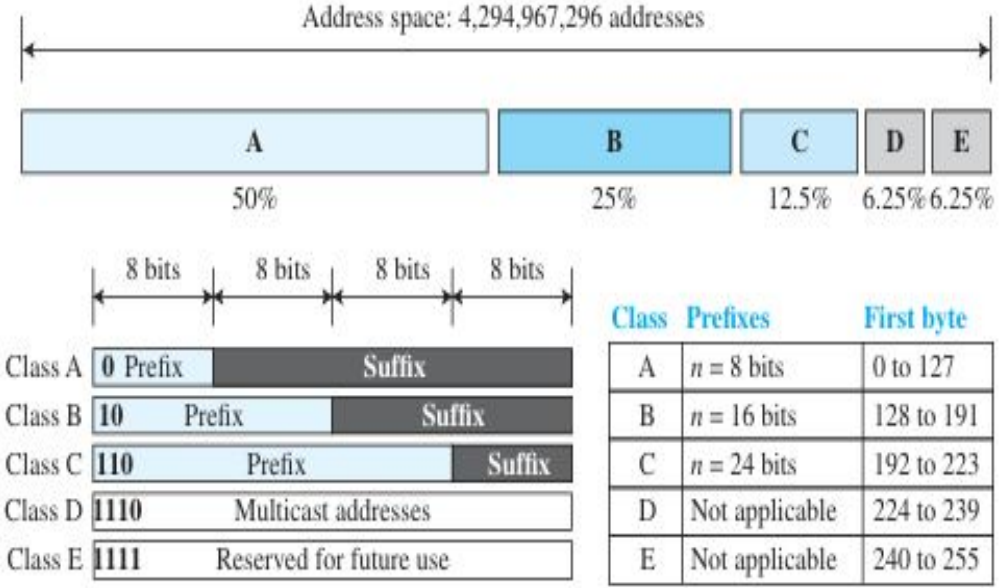


Figure 4.31 *Occupation of the address space in classful addressing*



Class	Prefixes	First byte
A	$n = 8$ bits	0 to 127
B	$n = 16$ bits	128 to 191
C	$n = 24$ bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

Number of Networks

- **Class A:** 2^7 = **128 networks**
- **Class B:** 2^{14} = **16,384 networks**
- **Class C:** 2^{21} = **2,097,152 networks**

Class	First Octet Range	Address Range
A	1 – 126	1.0.0.0 – 126.255.255.255
B	128 – 191	128.0.0.0 – 191.255.255.255
C	192 – 223	192.0.0.0 – 223.255.255.255
D	224 – 239	224.0.0.0 – 239.255.255.255
E	240 – 255	240.0.0.0 – 255.255.255.255

- **Class A:** Starts with **0** → **1–126** → Example: **10.20.30.40**
- **Class B:** Starts with **10** → **128–191** → Example: **172.16.25.100**
- **Class C:** Starts with **110** → **192–223** → Example: **192.168.1.10**
- **Class D:** Starts with **1110** → **224–239** → Example: **224.0.0.5** *(Multicast)*
- **Class E:** Starts with **1111** → **240–255** → Example: **240.0.0.1** *(Reserved)*

Address Depletion

The major drawback of classful addressing was **address depletion** (wastage of IP addresses).

- **Class A** provided about **16 million host addresses per network**, which was far more than most organizations required, resulting in many unused addresses.
- **Class B** also wasted many addresses because many organizations did not need such large networks.
- **Class C** allowed only **256 addresses per network**, which was too small for many organizations.
- **Class E** addresses were reserved and were generally not used.

As a result, IPv4 addresses were **not efficiently utilized**, leading to rapid exhaustion of available addresses.

Subnetting

Subnetting divides a **Class A or Class B network** into **smaller subnetworks (subnets)**.

- Increases the **network prefix length**.
- Makes better use of available addresses.
- Example: A Class A network can be divided into several smaller subnets.
-

Supernetting

Supernetting combines **multiple Class C networks** into **one larger network**.

- Provides more addresses for organizations that need more than **256 host addresses**.
- However, it makes **routing more complex**.

Advantage of Classful Addressing

The main advantage of classful addressing was its **simplicity**.

- The **class of an address** could be identified from its starting bits.
- Since each class had a **fixed prefix length**, the **Network ID** and **Host ID** could be determined without any additional information.

Classful addressing *uses* ***fixed-length network prefixes****.*

IPv4 address space is divided into ***Classes A, B, C, D, and E****.*

Class A: *Large networks.*

Class B: *Medium networks.*

Class C: *Small networks.*

Class D: *Multicasting.*

Class E: *Reserved.*

Major drawback: ***Address depletion*** *due to inefficient allocation.*

Subnetting *divides large networks into smaller ones.*

Supernetting *combines multiple Class C networks into a larger network.*

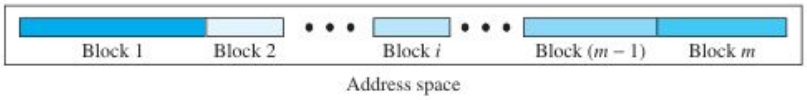
Advantage: *The address class and prefix length can be identified directly from the address.*

Classless Addressing (CIDR)

Classless Addressing uses **variable-length network prefixes** instead of fixed classes (A, B, and C). It provides efficient allocation of IPv4 addresses and overcomes the address depletion problem of classful addressing.

It is also known as **CIDR (Classless Inter-Domain Routing)**.

Figure 4.32 *Variable-length blocks in classless addressing*



Features of Classless Addressing

- Uses **variable-length prefixes**.
- Address space is divided into **variable-sized blocks**.
- A block can contain **1, 2, 4, 8, 16, 32, ... addresses**.
- The number of addresses in a block must be a **power of 2**.
- The **prefix** identifies the network (block).
- The **suffix** identifies the host.
- Prefix length can vary from **0 to 32 bits**.
- **Smaller prefix → Larger network**
- **Larger prefix → Smaller network**

Note: Classful addressing is a special case of classless addressing.

- Class A → **/8**
- Class B → **/16**
- Class C → **/24**

Prefix Length (Slash Notation)

In classless addressing, the prefix length is written after the IP address using a slash (/).

Format:

IP Address / Prefix Length

Examples:

- **192.168.1.0/24**
- **172.16.10.0/16**
- **10.0.0.0/8**

This is called **CIDR (slash) notation**.

Extracting Information from a CIDR Address

For an address **A.B.C.D/n**:

1. Number of Addresses

$$\text{Number of Addresses} = 2^{(32-n)}$$

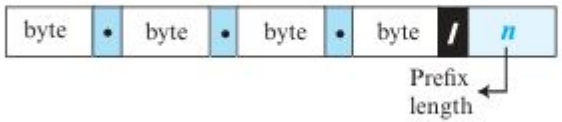
2. First Address (Network Address)

- Keep the first **n bits** unchanged.
- Change the remaining **(32 − n) bits** to **0**.

3. Last Address (Broadcast Address)

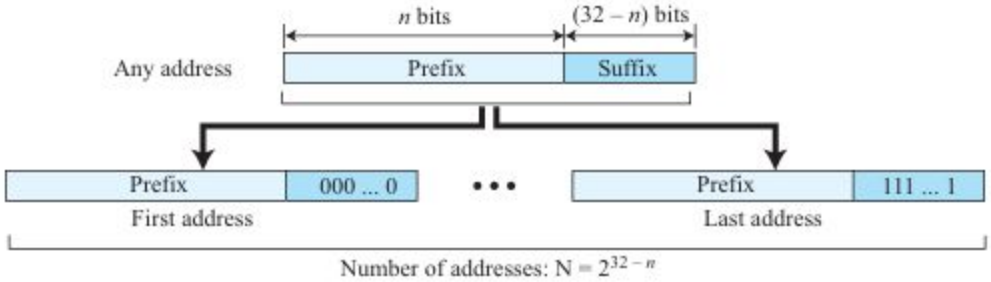
- Keep the first **n bits** unchanged.
- Change the remaining **(32 − n) bits** to **1**.

Figure 4.33 *Slash notation (CIDR)*



Examples:
12.24.76.8/8
23.14.67.92/12
220.8.24.255/25

Figure 4.34 *Information extraction in classless addressing*



Example

Given:

167.199.170.82/27

Number of Addresses

$$2^{(32-27)} = 2^5 = 32$$

First Address: 167.199.170.64

Last Address: 167.199.170.95

Using the Address Mask

1. Number of Addresses

$$\boxed{\text{NOT (Mask)} + 1}$$

2. First Address

$$\boxed{\text{Address AND Mask}}$$

3. Last Address

$$\boxed{\text{Address OR (NOT Mask)}}$$

Example

Given:

- **Address:** 167.199.170.82/27
- **Subnet Mask:** 255.255.255.224

Results:

- **Number of Addresses:** 32
- **First Address:** 167.199.170.64
- **Last Address:** 167.199.170.95

Address Mask (Subnet Mask)

The **Address Mask (Subnet Mask)** is a **32-bit number** in which:

- First **n bits = 1**
- Remaining **(32 − n) bits = 0**

Common Subnet Masks

Prefix	Subnet Mask
/8	255.0.0.0
/16	255.255.0.0
/24	255.255.255.0
/27	255.255.255.224

Example 4.1: Finding Information from a Classless Address

Given Address: 167.199.170.82/27

The prefix length is **27**, which means:

- The **first 27 bits** represent the **network (prefix)**.
- The **remaining 5 bits (32 − 27 = 5)** represent the **host (suffix)**.

Step 1: Find the Number of Addresses

The number of addresses in a CIDR block is calculated using:

$$\boxed{\text{Number of Addresses} = 2^{(32-n)}}$$

where **n = 27**

$$= 2^{(32-27)} = 2^5 = 32$$

Therefore, the block contains 32 IP addresses.

Step 2: Find the First Address (Network Address)

Rule: Keep the **first 27 bits unchanged** and change the **remaining 5 host bits to 0.**

Given Address

```
167.199.170.82/27

10100111 11000111 10101010 01010010
```

After changing the last 5 bits to 0

```
10100111 11000111 10101010 01000000
```

First Address = 167.199.170.64/27

- The first (network) 27 bits remain the same.
- The last 5 host bits are replaced with **00000**.
- This gives the **smallest (network) address** in the block.

Step 3: Find the Last Address (Broadcast Address)

Rule: Keep the **first 27 bits unchanged** and change the **remaining 5 host bits to 1.**

Given Address

```
167.199.170.82/27

10100111 11000111 10101010 01010010
```

After changing the last 5 bits to 1

```
10100111 11000111 10101010 01011111
```

Last Address = 167.199.170.95/27

- The first 27 network bits remain unchanged.
- The last 5 host bits are replaced with **11111**.
- This gives the **largest (broadcast) address** in the block.

Example 4.2: Finding Network Information Using Address Mask

Given Address: 167.199.170.82/27

Step 1: Find the Address Mask

For **/27**, the subnet mask is:

255.255.255.224

Binary form:

```
11111111.11111111.11111111.11100000
```

Step 2: Find the First Address (Network Address)

Rule:

First Address = IP Address AND Subnet Mask

```
IP Address      : 167.199.170.82
Subnet Mask     : 255.255.255.224
---------------------------------
Network Address : 167.199.170.64
```

- Perform the **AND** operation between the IP address and subnet mask.
- The result is the **first (network) address** of the block.

Step 3: Find the Last Address (Broadcast Address)

Rule:

Last Address = IP Address OR (NOT Subnet Mask)

```
IP Address         : 167.199.170.82
NOT Subnet Mask    : 0.0.0.31
--------------------------------
Broadcast Address : 167.199.170.95
```

- First, find the complement (**NOT**) of the subnet mask.
- Perform the **OR** operation with the IP address.
- The result is the **last (broadcast) address** of the block.

Example 4.3: One Address Can Belong to Different Blocks

Given Address: 230.8.24.56

In **Classless Addressing**, an IP address **alone cannot identify the network.** The **prefix length (/n)** must also be specified.

For example, the address **230.8.24.56** can belong to different networks depending on the prefix length.

Address	Belongs to Network
230.8.24.56/8	230.0.0.0/8
230.8.24.56/16	230.8.0.0/16
230.8.24.56/24	230.8.24.0/24
230.8.24.56/27	230.8.24.32/27
230.8.24.56/32	230.8.24.56/32 (Single Host)

The **same IP address** can belong to **different networks** depending on the **prefix length**.

Therefore, an IP address by itself **does not define the network**.

The **prefix length (CIDR notation)** is required to identify the correct network block.

Network Address

The **Network Address** is the **first address** in an address block. It uniquely identifies a network and is used by **routers** to forward packets to the correct destination network.

Working of Network Address

- When a packet arrives at a router, the router first finds the **network address** of the destination.
- The router then checks its **forwarding table**.
- Based on the network address, the router selects the correct **output interface**.
- After the packet reaches the destination network, it is delivered to the destination host.

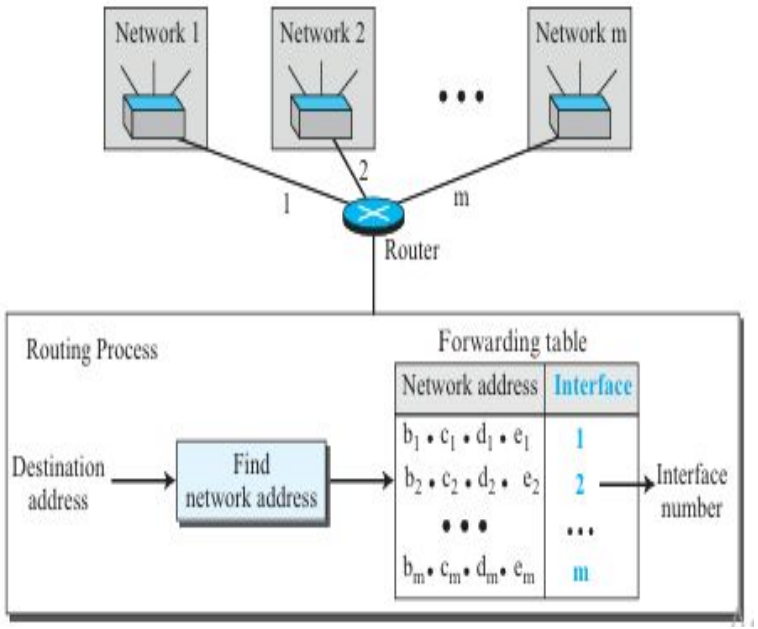
> **Note:** The **Network Address** is the **identifier of the network**.

Block Allocation

In **Classless Addressing (CIDR)**, IP address blocks are allocated by the **Internet Corporation for Assigned Names and Numbers (ICANN)**.

- **ICANN** allocates large blocks of addresses to **Internet Service Providers (ISPs)**.
- ISPs then divide these large blocks into smaller blocks and assign them to organizations or individual users.

Figure 4.35 *Network address*



Rules for Block Allocation

1. Number of Addresses Must Be a Power of 2

- The size of an address block must be:
  - $2^1 = 2$
  - $2^2 = 4$
  - $2^3 = 8$
  - $2^4 = 16$
  - $2^5 = 32$
  - $2^6 = 64$, etc.
- The prefix length is calculated as:

$$n = 32 - \log_2 N$$

where:

- **N** = Number of addresses
- **n** = Prefix length

2. Addresses Must Be Contiguous

- All addresses in the allocated block must be **continuous (contiguous)**.
- The **first address** of the block must be **divisible by the number of addresses** in the block.

Example:

For a block of **32 addresses**, valid starting addresses are:

- 0
- 32
- 64
- 96
- 128
- ...

Since these numbers are divisible by **32**, they can be the first address of a **/27** network.

Example 4.4 An ISP has requested a block of 1000 addresses. Since 1000 is not a power of 2, 1024 addresses are granted. The prefix length is calculated as n = 32 − log 2 18.14.12.0/ 22 , 1024 = 22. An available block, is granted to the ISP. It can be seen that the first address in decimal is 302,910,464, which is divisible by 1024.

Example 4.4: Block Allocation

An **ISP** requests a block of **1000 IP addresses.**

Step 1: Check if the Requested Addresses are a Power of 2

- Requested addresses = **1000**
- Since **1000 is not a power of 2**, the next higher power of 2 is chosen.

$$2^{10} = 1024$$

Therefore, the ISP is allocated 1024 addresses.

Step 2: Calculate the Prefix Length

Formula:

$$\boxed{n = 32 - \log_2 N}$$

where:

- $N = 1024$

$$n = 32 - \log_2(1024)$$

$$= 32 - 10$$

$$= 22$$

Prefix Length = /22

Step 3: Allocate the Address Block

An available block is assigned:

18.14.12.0/22

This block contains **1024 contiguous addresses.**

Step 4: Verify the First Address

The **first address** must be divisible by the **block size (1024).**

- First Address = **18.14.12.0**
- Decimal equivalent = **302,910,464**
- **302,910,464 ÷ 1024 = 295,811** (No remainder)

Hence, the first address satisfies the allocation rule.

Subnetting

Subnetting is the process of dividing a network into **smaller subnetworks (subnets)**. It creates multiple levels of hierarchy and improves network management and routing.

- An organization (or ISP) that receives a block of IP addresses can divide it into **several subnets**.
- Each subnet can be further divided into **sub-subnets**, and so on.

Designing Subnets

When designing subnetworks, the following rules must be followed:

1. Number of Addresses

- The number of addresses in each subnet must be a **power of 2**.

Examples: 2, 4, 8, 16, 32, 64, 128, ...

2. Prefix Length

The prefix length of each subnet is calculated using:

$$n_{sub} = 32 - \log_2(N_{sub})$$

where:

- N_{sub} = Number of addresses in each subnet
- n_{sub} = Prefix length of the subnet

3. Starting Address

- The **starting address** of each subnet must be **divisible by the number of addresses** in that subnet.
- This ensures proper subnet allocation.

Finding Information About Each Subnet

After subnetting, the information for each subnet can be found using the same method as in **CIDR blocks**.

For each subnet, determine:

- **Number of addresses**
- **First (Network) address**
- **Last (Broadcast) address**

Example 4.5

An organization is granted a block of addresses with the beginning address 14.24.74.0/24. The organization needs to have 3 subblocks of addresses to use in its three subnets: one subblock of 10 addresses, one subblock of 60 addresses, and one subblock of 120 addresses. Design the subblocks.

Solution

There are $2^{32-24} = 256$ addresses in this block. The first address is 14.24.74.0/24; the last address is 14.24.74.255/24. To satisfy the third requirement, we assign addresses to subblocks, starting with the largest and ending with the smallest one.

a. The number of addresses in the largest subblock, which requires 120 addresses, is not a power of 2. We allocate 128 addresses. The subnet mask for this subnet can be found as $n_1 = 32 - \log_2 128 = 25$. The first address in this block is 14.24.74.0/25; the last address is 14.24.74.127/25.

b. The number of addresses in the second largest subblock, which requires 60 addresses, is not a power of 2 either. We allocate 64 addresses. The subnet mask for this subnet can be found as $n_2 = 32 - \log_2 64 = 26$. The first address in this block is 14.24.74.128/26; the last address is 14.24.74.191/26.

c. The number of addresses in the smallest subblock, which requires 10 addresses, is not a power of 2 either. We allocate 16 addresses. The subnet mask for this subnet can be found as $n_3 = 32 - \log_2 16 = 28$. The first address in this block is 14.24.74.192/28; the last address is 14.24.74.207/28.

Address Aggregation

Address Aggregation is a technique in **CIDR** where **multiple smaller address blocks are combined into one larger block**. It is also called **Route Summarization** or **Address Summarization**.

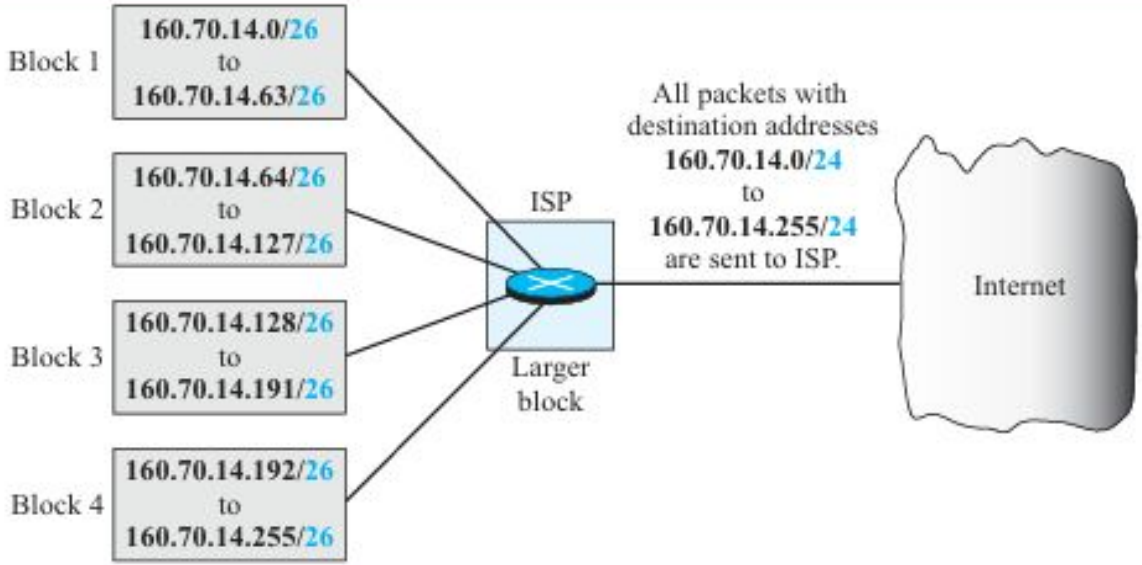
Working of Address Aggregation

- **ICANN** assigns a **large block of IP addresses** to an **Internet Service Provider (ISP)**.
- The **ISP** divides the large block into **smaller sub-blocks**.
- These smaller blocks are allocated to different customers.
- During routing, routers use the **prefix of the larger block** instead of maintaining separate entries for every smaller block.

Advantages of Address Aggregation

- Reduces the size of **routing tables**.
- Simplifies routing.
- Improves routing efficiency.
- Reduces routing overhead.

Figure 4.37 *Example of address aggregation*



1. This-Host Address

- **Address: 0.0.0.0/32**
- Used when a host **does not know its own IP address.**
- Used as the **source address** during IP address configuration.

Example: `0.0.0.0`

2. Limited Broadcast Address

- **Address: 255.255.255.255/32**
- Used to send a packet to **all devices in the local network.**
- **Routers do not forward** these packets outside the network.

Example: `255.255.255.255`

3. Loopback Address

- **Block: 127.0.0.0/8**
- Used to **test the TCP/IP protocol stack and network applications.**
- Packets sent to a loopback address **never leave the host.**

Common Example: `127.0.0.1` (localhost)

4. Private Addresses

Private addresses are used **inside private networks** and are **not routable on the Internet.**

Private Address Block	Purpose
10.0.0.0/8	Large private networks
172.16.0.0/12	Medium private networks
192.168.0.0/16	Home and small office networks
169.254.0.0/16	Automatic Private IP Addressing (APIPA) when no DHCP server is available

4. Private Addresses

Private addresses are used **inside private networks** and are **not routable on the Internet.**

Private Address Block	Purpose
10.0.0.0/8	Large private networks
172.16.0.0/12	Medium private networks
192.168.0.0/16	Home and small office networks
169.254.0.0/16	Automatic Private IP Addressing (APIPA) when no DHCP server is available

5. Multicast Addresses

- **Block: 224.0.0.0/4**
- Used for **multicasting**, where one sender transmits data to a group of receivers.

Example: `224.0.0.1`

Special Address	Address/Block	Purpose
This-Host	0.0.0.0/32	Used when the host does not know its own IP address
Limited Broadcast	255.255.255.255/32	Sends data to all devices in the local network
Loopback	127.0.0.0/8	Tests the TCP/IP stack; packets never leave the host
Private Addresses	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16	Used in private/local networks
Multicast	224.0.0.0/4	Used for multicast communication

Dynamic Host Configuration Protocol (DHCP)

Dynamic Host Configuration Protocol (DHCP) is an **application-layer protocol** that automatically assigns IP configuration to hosts in a network. It follows the **client-server model** and is often called a **Plug-and-Play protocol** because devices can join a network without manual configuration.

Features of DHCP

- Automatically assigns **IP addresses** to hosts.
- Uses the **client-server architecture**.
- Can assign:
    - **Permanent (Static) IP addresses**
    - **Temporary (Dynamic) IP addresses**
- Widely used in organizations, homes, and ISPs.

Functions of DHCP

DHCP provides the following information to a host:

1. **IP Address**
2. **Network Prefix (Subnet Mask)**
3. **Default Gateway (Router Address)**
4. **DNS Server Address (Name Server)**

These parameters allow a host to communicate within the local network and access other networks.

Advantages of DHCP

- Eliminates manual IP address configuration.
- Reduces configuration errors.
- Efficiently manages IP address allocation.
- Supports temporary IP assignment for mobile users and ISP customers.

DHCP Message Format

DHCP communication takes place using **request** and **response** messages exchanged between the **client** and the **server**.

The **Options field** is the most important part of the DHCP message.

Options Field

- Size: **64 bytes**
- Used to carry:
    - Additional configuration information.
    - Vendor-specific information.

Magic Cookie

- DHCP uses a special number called the **Magic Cookie**.
- Value:

99.130.83.99

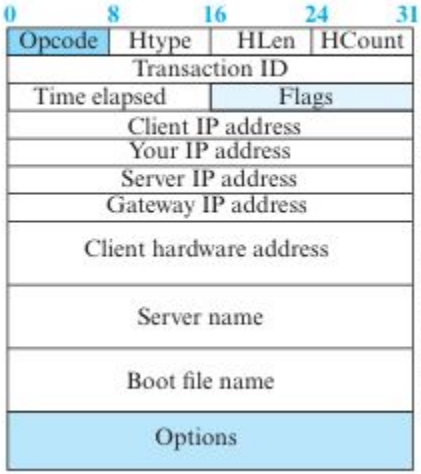
- It identifies the message as a DHCP message.
- If the Magic Cookie is present, the remaining bytes contain DHCP options.

DHCP Option Format

Each DHCP option consists of three fields:

Field	Description
Tag (1 byte)	Identifies the type of option
Length (1 byte)	Specifies the length of the value field
Value (Variable Length)	Contains the option data

Figure 4.38 *DHCP message format*



Fields:

Opcode: Operation code, request (1) or reply (2)
Htype: Hardware type (Ethernet, ...)
HLen: Length of hardware address
HCount: Maximum number of hops the packet can travel
Transaction ID: An integer set by client and repeated by the server
Time elapsed: The number of seconds since the client started to boot
Flags: First bit defines unicast (0) or multicast (1); other 15 bits not used
Client IP address: Set to 0 if the client does not know it
Your IP address: The client IP address sent by the server
Server IP address: A broadcast IP address if client does not know it
Gateway IP address: The address of default router
Server name: A 64-byte domain name of the server
Boot file name: A 128-byte file name holding extra information
Options: A 64-byte field with dual purpose described in text

DHCP Message Type

- If the **Tag field = 53**, the **Value field** specifies the **DHCP message type** (one of the standard DHCP message types used during communication).

DHCP *automatically assigns network configuration to hosts.*

It is an ***application-layer, client-server protocol****.*

Also called the ***Plug-and-Play protocol****.*

Provides:

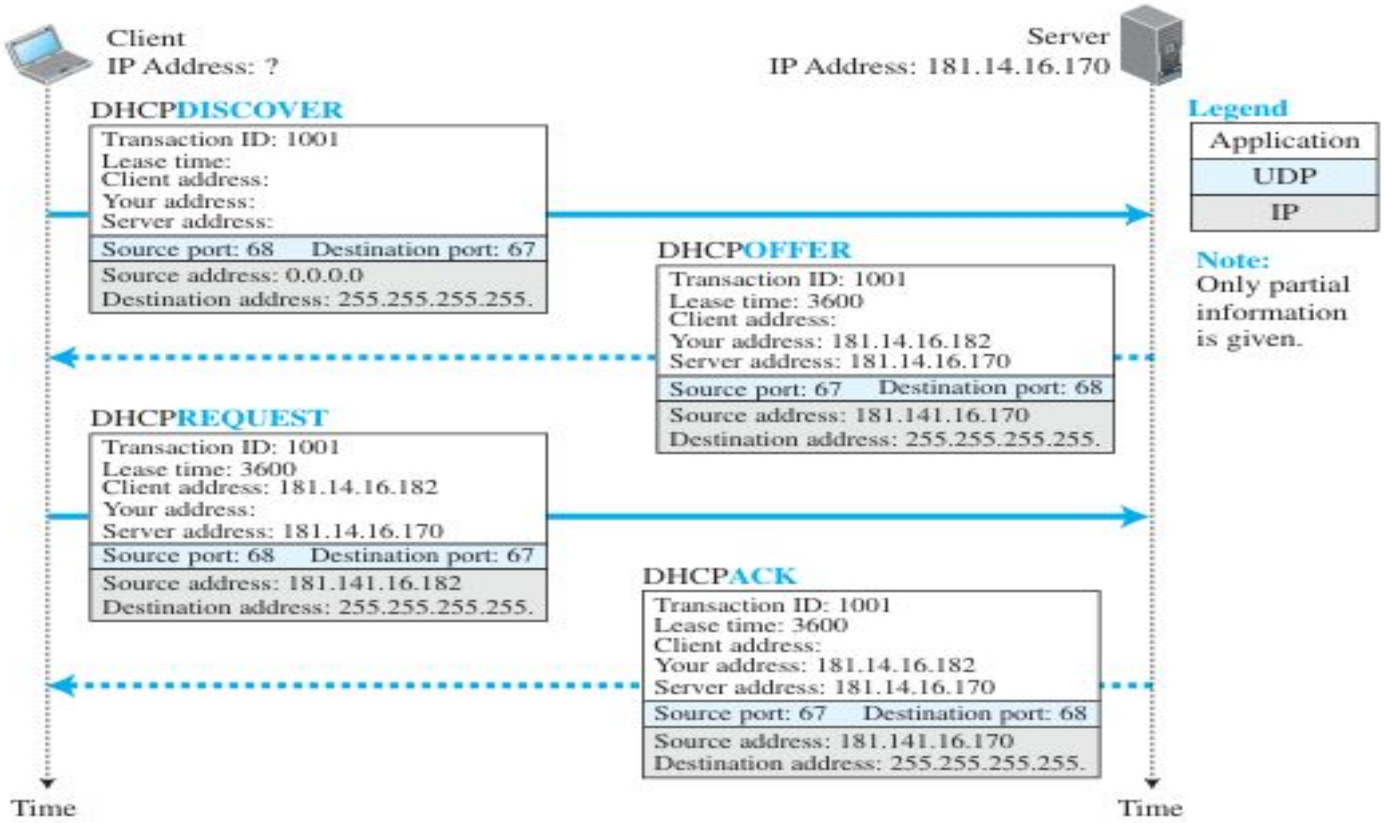
IP Address

Subnet Mask

Default Gateway

DNS Server Address

Figure 4.40 *Operation of DHCP*



DHCP Operation (DORA Process)

DHCP assigns an IP address to a host through a **four-step process**, commonly known as the **DORA process**:

1. **DHCPDISCOVER**
2. **DHCPOFFER**
3. **DHCPREQUEST**
4. **DHCPACK**

Step 1: DHCPDISCOVER (Discover)

- When a host joins the network, it sends a **DHCPDISCOVER** message to locate available DHCP servers.
- Only the **Transaction ID** field is set (to a random number).
- Since the host does not know its own IP address or the DHCP server's IP address:
  - **Source IP Address:** 0.0.0.0
  - **Destination IP Address:** 255.255.255.255 (Broadcast)
- **UDP Ports:**
  - Source Port = **68** (Client)
  - Destination Port = **67** (Server)

Step 2: DHCPOFFER (Offer)

- One or more DHCP servers respond with a **DHCPOFFER** message.
- The offer includes:
  - Offered **IP address**
  - **Server IP address**
  - **Lease time** (duration for which the IP address can be used)
- **UDP Ports:**
  - Source Port = **67** (Server)
  - Destination Port = **68** (Client)
- The message is sent as a **broadcast**, allowing multiple DHCP servers to respond.
- The client selects the **best offer**.

Step 3: DHCPREQUEST (Request)

- The client sends a **DHCPREQUEST** message to the selected DHCP server requesting the offered IP address.
- The destination is still a **broadcast address** so that other DHCP servers know their offers were not accepted.
- **UDP Ports:**
    - Source Port = **68** (Client)
    - Destination Port = **67** (Server)

Step 4: DHCPACK (Acknowledgement)

- The selected DHCP server replies with a **DHCPACK** message if the requested IP address is still available.
- The client receives:
    - IP Address
    - Subnet Mask
    - Default Gateway
    - DNS Server
    - Lease Time
- If the address is no longer available, the server sends a **DHCPNACK** message.
- The client must then restart the DHCP process.

Step	Message	Purpose
1	**DHCPDISCOVER**	**Client searches for available DHCP servers.**
2	**DHCPOFFER**	**Server offers an IP address and lease information.**
3	**DHCPREQUEST**	**Client requests the offered IP address.**
4	**DHCPACK**	**Server confirms the IP address allocation.** **If unavailable, it sends DHCPNACK.**

Additional Features of DHCP

1. Two Well-Known Ports

DHCP uses:

- **UDP Port 67** – Server
- **UDP Port 68** – Client

Reason: Fixed ports ensure DHCP broadcast messages are delivered only to the DHCP client. Multiple clients are identified using the **Transaction ID**.

2. Using FTP

- The **DHCPACK** message may provide the path of a configuration file.
- The client uses **FTP** to obtain additional network information (e.g., DNS server details).
-

3. Error Control

Since DHCP uses **UDP**, it provides reliability by:

- Using **UDP checksum**.
- Using **timers and retransmission** if no reply is received.
- Using **random timers** to avoid network congestion.

4. Transition States

- DHCP client works as a **state machine**.
- It changes states based on the messages it sends and receives during IP address allocation.

Network Address Translation (NAT)

- **Network Address Translation (NAT)** is a technology that maps **private IP addresses** to **public (global) IP addresses**. It allows a private network to communicate with the Internet using one or more public IP addresses.

Need for NAT

- The number of available public IPv4 addresses is limited. An organization or home network may contain many computers, but only a few require Internet access at the same time.

For example, a small office may have **20 computers**, but only **4 computers** may access the Internet simultaneously. Instead of assigning 20 public IP addresses, the organization can use:

- **Private IP addresses** for internal communication.
- **A few public IP addresses** assigned by the ISP for Internet communication.

This helps conserve public IPv4 addresses.

Working of NAT

- All devices inside the local network use **private IP addresses**.
- The network is connected to the Internet through a **NAT-enabled router**.
- Before a packet is sent to the Internet, the NAT router **translates the private IP address into a public IP address**.
- When a reply is received, the NAT router **translates the public IP address back to the corresponding private IP address** and forwards the packet to the correct device.

Thus, external networks communicate only with the public IP address of the NAT router.

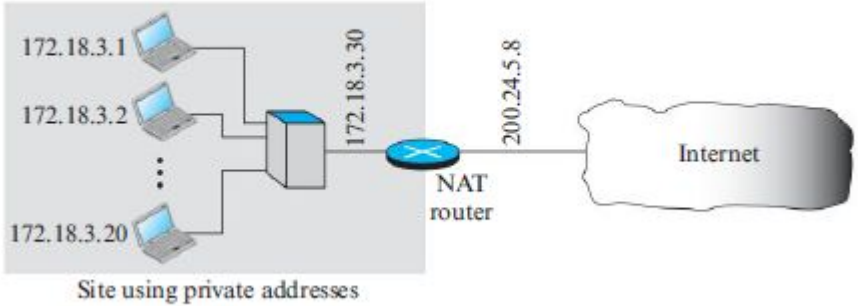
NAT Router

A NAT router has:

- One interface connected to the **private network**.
- One interface connected to the **Internet**.
- One or more **public IP addresses** assigned by the ISP.
- NAT software that performs address translation.

The **private network remains invisible** to the Internet because only the NAT router's public IP address is visible.

Figure 4.42 *NAT*



Advantages of NAT

- Conserves public IPv4 addresses.
- Allows multiple devices to share one or more public IP addresses.
- Supports communication between private networks and the Internet.
- Hides internal IP addresses, providing an additional level of security.
- Supports Virtual Private Networks (VPNs).

Example

A company has **20 computers**.

- **Private IP addresses:** 192.168.1.1 – 192.168.1.20
- **Public IP address assigned by ISP:** 200.24.5.8

When a computer accesses the Internet, the NAT router translates its private IP address into **200.24.5.8**. To the outside world, all communication appears to come from **200.24.5.8**, while the private network remains hidden.

Address Translation in NAT

- **Address Translation** is the process performed by the **NAT router** to convert **private IP addresses** into **public IP addresses** and vice versa. When a packet leaves the private network, the NAT router replaces the **source private IP address** with its **public (global) IP address**. Similarly, when a packet returns from the Internet, the NAT router replaces the **destination public IP address** with the corresponding **private IP address** and forwards it to the correct host.

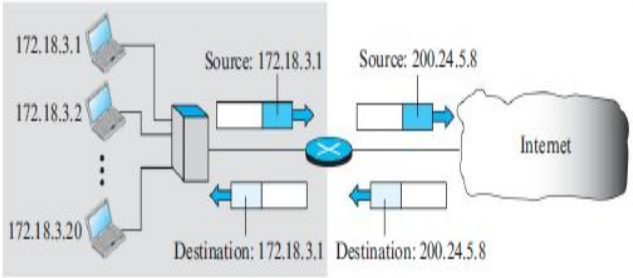
Translation Table

- To perform address translation correctly, the NAT router maintains a **translation table**. This table stores the mapping between private addresses and public addresses, enabling the router to identify the correct destination when response packets arrive from the Internet.

Using One IP Address

- In the simplest form of NAT, the translation table contains **two columns**: the **private IP address** and the **external (destination) IP address**. When a host sends a packet to the Internet, the NAT router records the destination address in the translation table. When the reply arrives, the router uses the source address of the incoming packet to identify the corresponding private host.
- In this method, **communication must always be initiated from the private network**. It is commonly used in home and small office networks where a single public IP address is shared among multiple devices.

Figure 4.43 *Address translation*



Site using private addresses

Figure 4.44 *Translation*

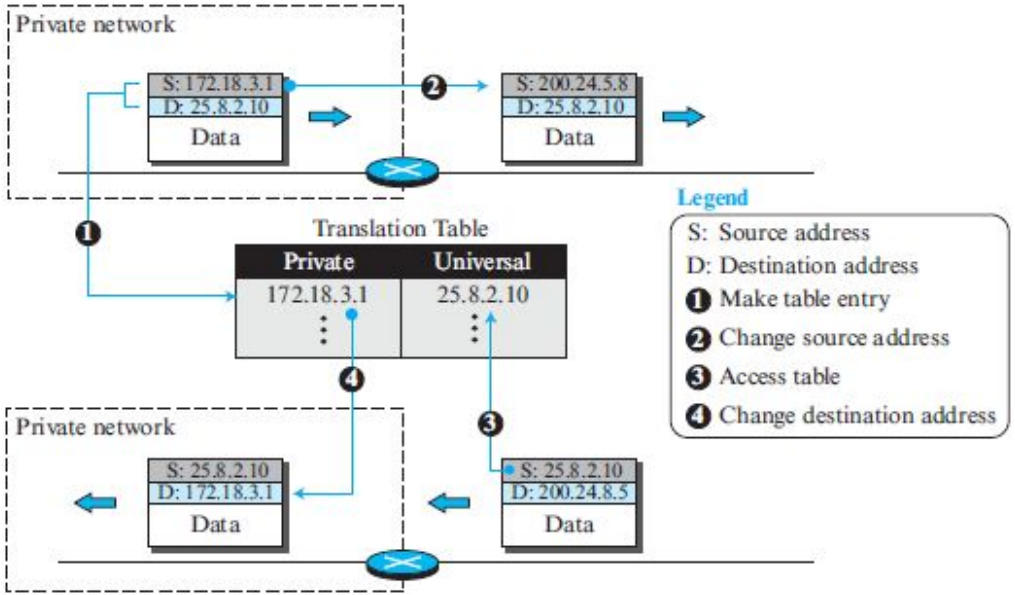


Table 4.1 *Five-column translation table*

Private address	*Private port*	*External address*	*External port*	*Transport protocol*
172.18.3.1	1400	25.8.3.2	80	TCP
172.18.3.2	1401	25.8.3.2	80	TCP
⋮	⋮	⋮	⋮	⋮

Using a Pool of IP Addresses

Instead of using a single public IP address, the NAT router can use a **pool of public IP addresses.** This allows multiple hosts in the private network to communicate with the same external host simultaneously, with each connection using a different public IP address. However, the number of simultaneous connections is limited by the number of available public IP addresses.

Using Both IP Addresses and Port Addresses (PAT/NAPT)

To support multiple simultaneous connections, NAT uses both **IP addresses and port numbers.** The translation table stores the **private IP address, public IP address, source port number, destination port number, and transport protocol (TCP/UDP).**

Each outgoing connection is assigned a **unique port number**, allowing multiple hosts to share a single public IP address while communicating with different servers or even the same server simultaneously. When the reply arrives, the NAT router uses the combination of the **public IP address and port number** to identify the correct private host and forward the packet.

Forwarding of IP Packets

Forwarding is the process of sending an IP packet to the **next hop** (router or destination) so that it reaches its final destination.

There are two methods of IP forwarding:

1. Forwarding Based on Destination Address

- Used in **connectionless IP networks** (traditional IPv4).
- Forwarding is based on the **destination IP address**.
- The host or router searches its **forwarding table** to determine the next hop.

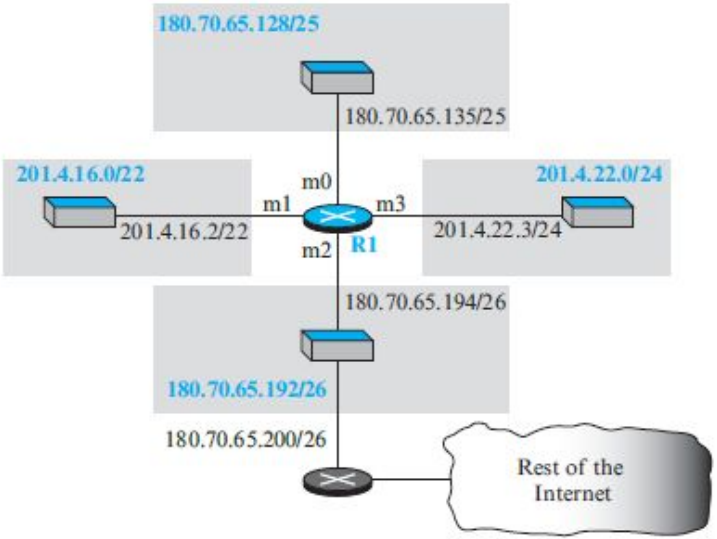
Forwarding Table

In **classless addressing (CIDR)**, each forwarding table entry contains:

- **Network Address (Prefix)**
- **Prefix Length (/n or Mask)**
- **Interface Number**
- **Next-Hop IP Address**

The router compares the destination IP address with each network prefix in the table. If a match is found, the packet is forwarded through the corresponding interface to the next hop. If no match is found, the **default route** is used.

Figure 4.46 *Configuration for Example 4.7*



Solution

Table 4.2 shows the corresponding table.

Table 4.2 *Forwarding table for router R1 in Figure 4.46*

Network address/mask	*Next hop*	*Interface*
180.70.65.192/26	—	m2
180.70.65.128/25	—	m0
201.4.22.0/24	—	m3
201.4.16.0/22	—	m1
Default	180.70.65.200	m2

Address Aggregation

- **Address Aggregation (Route Summarization)** combines multiple smaller address blocks into one larger block.
- It reduces the size of the **forwarding table** and speeds up packet forwarding.
- Routers outside the network use a **single routing entry** for the aggregated block instead of separate entries for each subnet.

Longest Mask Matching

- In **Classless Addressing (CIDR)**, a destination address may match multiple entries.
- The router selects the entry with the **longest prefix (largest mask)**.
- The forwarding table is arranged from the **longest mask to the shortest mask**.
- This ensures packets are forwarded to the **most specific destination**.

Hierarchical Routing

- **Hierarchical Routing** organizes routing into multiple levels to reduce forwarding table size.
- The Internet is divided into **Backbone ISPs, National ISPs, Regional ISPs, and Local ISPs.**
- Large address blocks are assigned to ISPs, which further divide them into smaller blocks for customers.
- Routers outside the ISP store only **one routing entry** for the entire address block.

Geographical Routing

- The address space is divided into **large geographical regions** such as America, Europe, Asia, and Africa.
- Routers use **one routing entry per geographical region**, reducing the forwarding table size.

Forwarding Table Search Algorithms

- Routers search the forwarding table using the **Longest Prefix Match** algorithm.
- The router checks the **longest matching prefix first.**
- Efficient data structures such as **tries (prefix trees)** can be used to speed up the search.

ICMPv4 (Internet Control Message Protocol Version 4)

ICMPv4 is a **network-layer protocol** that works along with IPv4. Since IPv4 does not provide **error reporting** or **network diagnostic** functions, ICMPv4 is used to report errors and exchange control information. ICMP messages are **encapsulated inside IP datagrams**, and the **Protocol field in the IP header is set to 1**.

Functions of ICMPv4

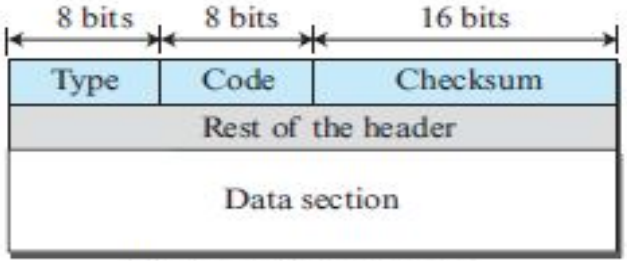
- Reports errors during IP packet transmission.
- Provides diagnostic and query services.
- Helps hosts and routers exchange control information.

ICMP Message Format

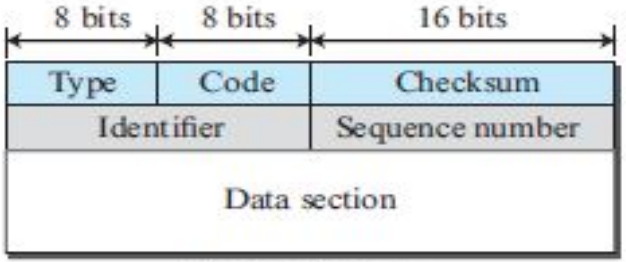
An ICMP message consists of:

- **Type (8 bits):** Specifies the message type.
- **Code (8 bits):** Gives additional information about the message.
- **Checksum (16 bits):** Detects errors in the ICMP message.
- **Rest of Header:** Depends on the message type.
- **Data Section:** Contains error details or query information.

Figure 4.54 *General format of ICMP messages*



Error-reporting messages



Query messages

Type and code values

Error-reporting messages
03: Destination unreachable (codes 0 to 15)
04: Source quench (only code 0)
05: Redirection (codes 0 to 3)
11: Time exceeded (codes 0 and 1)
12: Parameter problem (codes 0 and 1)

Query messages
08 and 00: Echo request and reply (only code 0)
13 and 14: Timestamp request and reply (only code 0)

Note: See the book website for more explanation about the code values.

Types of ICMP Messages

ICMP messages are divided into two categories:

1. Error Reporting Messages

Used to report problems encountered while processing IP packets.

Rules for Error Messages

- Sent only to the **original source** of the datagram.
- **Not generated** for:
  - Multicast or special addresses.
  - Another ICMP error message.
  - Fragments other than the first fragment.

The data section contains:

- Original IP header.
- First 8 bytes of the original data (helps identify TCP/UDP ports).

Common Error Messages

Type	Message	Purpose
3	Destination Unreachable	Destination network/host cannot be reached.
4	Source Quench	Indicates network congestion; sender should slow down. *(Obsolete today)*
5	Redirect	Suggests a better router for sending packets.
11	Time Exceeded	TTL becomes 0 or fragments do not arrive in time.
12	Parameter Problem	Invalid IP header or incorrect options.

2. Query Messages

Used for testing, diagnostics, and obtaining network information.

Type	Message	Purpose
8	Echo Request	Checks whether a host/router is reachable.
0	Echo Reply	Response to Echo Request.
13	Timestamp Request	Measures transmission time or clock synchronization.
14	Timestamp Reply	Returns timing information to the sender.

Common ICMP Message Types

Type	Message
0	Echo Reply
3	Destination Unreachable
4	Source Quench *(obsolete)*
5	Redirect
8	Echo Request
11	Time Exceeded
12	Parameter Problem
13	Timestamp Request
14	Timestamp Reply

Applications of ICMP

- **Ping** → Uses **Echo Request** and **Echo Reply** to check whether a host is reachable.
- **Traceroute** → Uses **Time Exceeded** messages to identify the path taken by packets.

Unicast Routing

Unicast Routing is the process of delivering an IP packet from **one source host to one destination host (one-to-one communication)**. In the Internet, packets are forwarded **hop by hop** through routers until they reach the destination. Since the Internet consists of a large number of interconnected networks, **hierarchical routing** is used to efficiently deliver packets.

Types of Routing

- **"Unicast: One sender → One receiver"**
- **"Multicast: One sender → Multiple selected receivers"**
- **"Broadcast: One sender → All hosts"**

General Idea of Unicast Routing

In unicast routing, packets are forwarded from the **source router** to the **destination router** using **forwarding tables.**

- The **source host** sends the packet to its **default router.**
- The **destination host** receives the packet from its **default router.**
- Only **routers** maintain forwarding tables.
- Each router checks its forwarding table and forwards the packet to the **next hop** until it reaches the destination.

Thus, routing mainly involves determining the **best path between the source router and the destination router.**

Internet as a Graph

To determine the best route, an Internet is represented as a **weighted graph**.

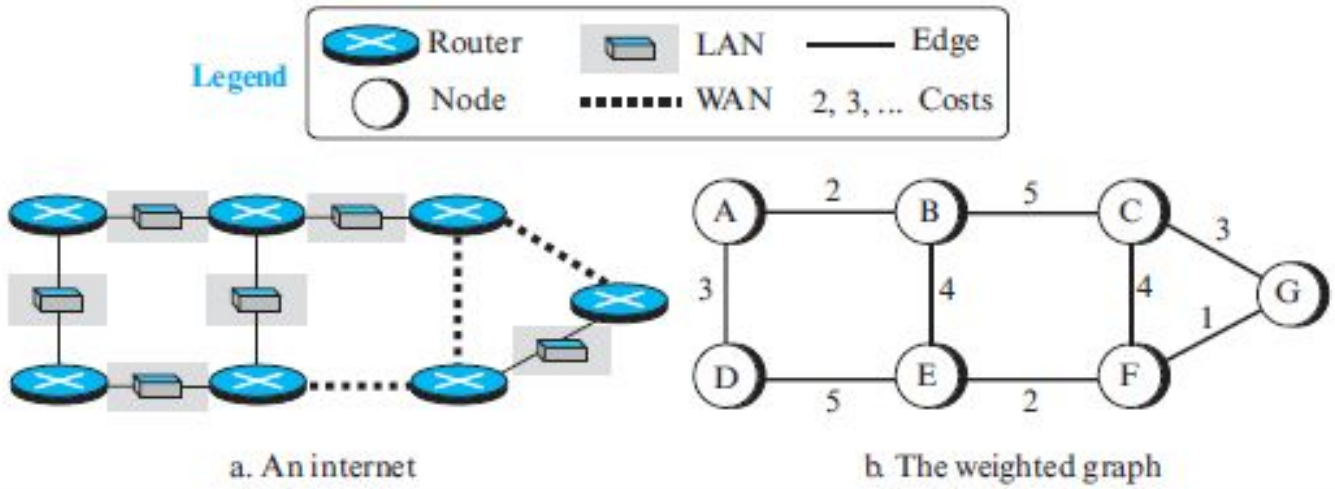
- **Nodes (Vertices):** Routers
- **Edges (Links):** Networks connecting routers
- **Weight (Cost):** Cost of transmitting data through a link

The cost may represent:

- Distance
- Delay
- Bandwidth
- Network load
- Administrative cost

If there is **no direct connection** between two routers, the cost is considered **infinity** (∞).

Figure 4.56 *An internet and its graphical representation*



a. An internet

b. The weighted graph

Least-Cost Routing

In a weighted graph, the **best route** is the route having the **minimum total cost.**

The router compares all possible paths and selects the one with the lowest cumulative cost.

Example

Suppose there are two possible routes from **A to E:**

- A → B → E = Cost **6**
- A → C → D → E = Cost **9**

The router selects:

A → B → E

because **6 < 9.**

Thus, **Least-Cost Routing** always chooses the path with the **minimum total cost.**

Least-Cost Trees (Shortest Path Trees)

Instead of finding a separate path for every destination, each router creates a **Least-Cost Tree**.

A **Least-Cost Tree** is a tree:

- Rooted at one router.
- Includes all routers in the network.
- Gives the **shortest (least-cost) path** from the root to every other router.

For an Internet with **N routers**:

- Each router has **one least-cost tree**.
- Therefore, there are **N least-cost trees** in total.

This method is more efficient than storing separate paths for every destination.

Properties of Least-Cost Trees

1. Symmetric Paths

The least-cost path from router **X to Y** is the reverse of the least-cost path from **Y to X**, and both have the same cost.

Example

- A → B → E → F
- F → E → B → A

Both routes have the same total cost.

2. Path Consistency

A least-cost path can be divided into smaller least-cost paths.

Example

Instead of traveling directly:

A → B → E → F → G

we can divide the journey into:

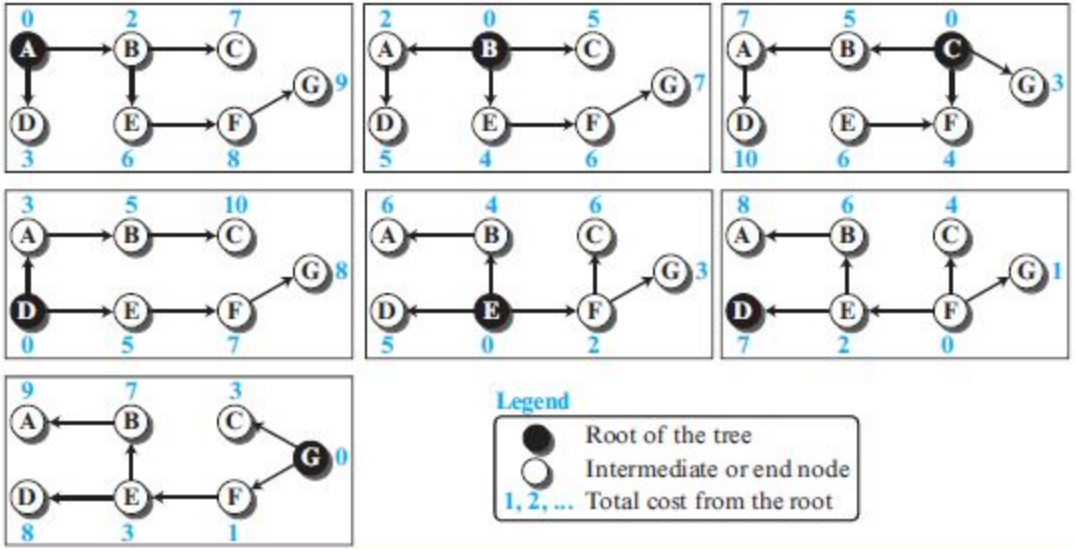
- A → E (using A's least-cost tree)
- E → G (using E's least-cost tree)

The **total cost remains the same**.

Advantages of Unicast Routing

- Efficient one-to-one communication.
- Selects the **best available path.**
- Reduces transmission cost.
- Supports scalable Internet communication.
- Uses forwarding tables for fast packet delivery.

Figure 4.57 *Least-cost trees for nodes in the internet of Figure 4.56*



Routing Algorithms

After understanding the concept of **least-cost trees** and **forwarding tables**, the next step is to learn how routers actually find these least-cost paths. This is done using **routing algorithms**.

Different routing algorithms have been developed over time. They mainly differ in:

- **How they calculate the least (shortest) cost** between networks.
- **How they construct the least-cost tree** for each router.

One of the most widely used routing algorithms is **Distance-Vector Routing**.

Distance-Vector (DV) Routing

Distance-Vector Routing is a dynamic routing algorithm in which each router determines the shortest path to every destination based on information received from its neighboring routers.

Working Principle

1. **Initial Information**
   - Every router knows only:
     - Itself (distance = 0)
     - Its directly connected neighbors
     - The cost to reach each neighbor
2. **Creating the Distance Vector**
   - Each router prepares a **distance vector (routing table)** containing the minimum known distance to every destination.
3. **Sharing Information**
   - Routers periodically exchange their distance vectors **only with their immediate neighbors**.
4. **Updating Routes**
   - After receiving a neighbor's distance vector, a router compares the new information with its current routing table.
   - If a shorter path is found, the routing table is updated.
5. **Convergence**
   - Through continuous exchange of routing information, all routers eventually learn the shortest paths to every destination in the network.

Each router continuously informs its neighboring routers about the best routes it knows to every destination. Over time, this information spreads throughout the network, allowing every router to build a complete view of the shortest paths.

Bellman-Ford Equation

The **Bellman-Ford Equation** is the mathematical foundation of Distance-Vector Routing. It is used to calculate the **least-cost (shortest) path** from a source router to a destination router.

Suppose a router **x** wants to find the shortest path to destination **y** through one of its neighboring routers.

The Bellman-Ford equation is:

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\}$$

Where:

- $D_x(y)$ = Minimum cost from router **x** to destination **y**
- $c(x, v)$ = Cost of the direct link from router **x** to its neighboring router **v**
- $D_v(y)$ = Cost from neighbor **v** to destination **y**
- **min** = Choose the route with the lowest total cost

To reach destination **Y**, router **X**:

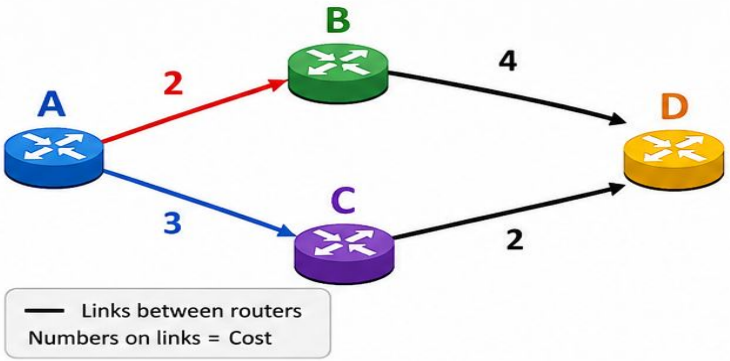
1. Looks at each of its neighboring routers.
2. Calculates:

   Cost to Neighbor + Neighbor's Cost to Destination
3. Selects the neighbor that gives the **minimum total cost**.

- *Distance-Vector Routing is a **dynamic routing algorithm**.*
- *Each router knows only the costs to its **direct neighbors** initially.*
- *Routers **periodically exchange routing information** with their immediate neighbors.*
- *The **Bellman-Ford Equation** is used to compute the shortest path.*
- *After several updates, all routers learn the **least-cost paths** to every destination in the network.*

Bellman-Ford Example: Router A wants to reach Router D

1 Step 1: Network Topology



2 Step 2: Router A checks each neighbor

Router A will choose the neighbor that gives the lowest total cost to reach Router D.

Neighbor	Cost to Neighbor	Neighbor's Cost to D	Total Cost (= Cost to Neighbor + Neighbor's Cost to D)
B	2	4	6
C	3	2	5

3 Step 3: Calculation using Bellman-Ford Equation

Bellman-Ford Equation

$$D_A(D) = \min_v \{ c(A,v) + D_v(D) \}$$

Via B:
$c(A,B) + D_B(D) = 2 + 4 = 6$

Via C:
$c(A,C) + D_C(D) = 3 + 2 = 5$

4 Step 4: Compare Total Costs

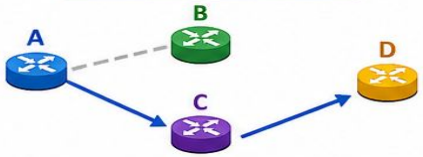
Via B
Total Cost = 6

Via C
Total Cost = 5

5 < 6

5 Step 5: Decision

Since 5 < 6, Router A chooses C as the next hop to reach D.



Next hop from A to D is C

Animation Flow

1 Network Topology → 2 Check Each Neighbor → 3 Calculate Total Cost → 4 Compare Costs → 5 Choose Best Next Hop

Bellman-Ford Equation and Distance Vector

Bellman-Ford Equation

The **Bellman-Ford Equation** is the core of the **Distance-Vector Routing algorithm**. It helps a router calculate the **minimum (least) cost** from a **source node** to a **destination node** by considering all possible neighboring routers.

Instead of checking every possible route in the network, the router compares the cost through each neighboring router and selects the route with the **lowest total cost**.

General Bellman-Ford Equation

$$D_{xy} = \min\{(c_{xa} + D_{ay}),\ (c_{xb} + D_{by}),\ (c_{xc} + D_{cy}),\ \ldots\}$$

Where

- D_{xy} = Least cost from source node **x** to destination **y**
- c_{xa} = Cost from **x** to neighbor **a**
- D_{ay} = Least cost from neighbor **a** to destination **y**
- **min** = Select the smallest total cost

Meaning

To reach destination **y**, router **x**:

1. Checks each neighboring router.
2. Calculates

   Cost to Neighbor + Neighbor's Cost to Destination

3. Chooses the route with the **minimum total cost.**

Simplified Bellman-Ford Equation

When the router already knows a route to the destination, it simply compares the **current route** with a **new route through a neighboring router z.**

$$D_{xy} = \min\{D_{xy},\ (c_{xz} + D_{zy})\}$$

Where

- **Current Cost** = D_{xy}
- **New Cost through z** = $c_{xz} + D_{zy}$

If the new cost is smaller, the router updates its routing table.

Example

Suppose Router **A** wants to reach Router **D**.

Path	Calculation	Total Cost
Existing route	8	8
Via Router C	3 + 2	5

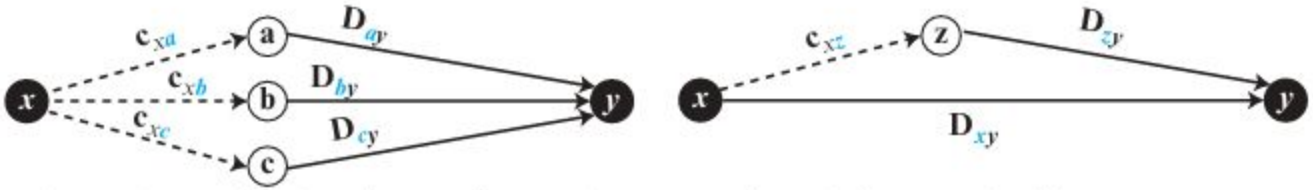
Using the Bellman-Ford equation,

$$D_{AD} = \min(8,\ 3 + 2)$$

$$D_{AD} = \min(8,\ 5) = 5$$

Since **5 < 8**, Router **A** updates its routing table and chooses **Router C** as the next hop.

Figure 4.58 *Graphical idea behind Bellman-Ford equation*



a. General case with three intermediate nodes

b. Updating a path with a new route

The Bellman-Ford equation allows routers to:

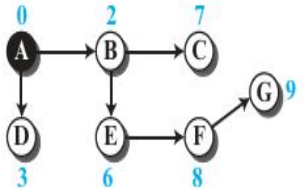
- Find the shortest path.
- Update routes automatically when a better path is discovered.
- Build the network's least-cost paths step by step.
- Adapt to changes in the network.

Bellman-Ford builds a new shortest path using already known shortest paths from neighboring routers.

Distance Vectors

- The **Distance Vector** is the concept behind the name **Distance-Vector Routing**.
- A **least-cost tree** consists of all the least-cost paths from a **source (root) node** to every other destination in the network. These paths are connected together graphically to form a tree.
- Distance-vector routing **breaks (unglues)** this tree into a **one-dimensional array**, called the **distance vector**, which stores only the minimum cost to reach every destination.
- Thus, a distance vector is a compact representation of the least-cost tree.

Figure 4.59 *The distance vector corresponding to a tree*



a. Tree for node A

	A
A	0
B	2
C	7
D	3
E	6
F	8
G	9

b. Distance vector for node A

Least-Cost Tree vs Distance Vector

Least-Cost Tree	Distance Vector
Graphical representation of shortest paths	One-dimensional array (table)
Shows complete path from source to every destination	Shows only the least cost to each destination
Easier for humans to visualize	Easier for routers to store and exchange

- Distance vector routing is a dynamic routing method used in computer networks in which every router keeps a table, called a distance vector, listing the lowest known cost from itself to each other node in the network.
- A router does not need to know the network's complete topology — only the cost to reach each destination and which neighbour to forward through. Routers build and correct these tables by periodically exchanging their vectors with the neighbours they are directly connected to.
- This document walks through the idea using three worked diagrams: how a distance vector is derived from a node's shortest-path tree, how all routers set up their initial vectors, and how a router updates its vector when a neighbour's vector arrives.

Deriving a Distance Vector from a Tree

- Every node can be pictured as the root of its own shortest-path tree spanning the network.
- The distance vector of that node is just a one-column table listing the cumulative cost from the root to every other node along that tree.

- In the tree on the left, node A reaches B with cost 2, C with cost 7 (2 + 5 through B), D directly with cost 3, E with cost 6 (through D), F with cost 8 (through E), and G with cost 9 (through F).
- Reading these accumulated costs off the tree and placing them in a column, indexed by destination, produces exactly the distance vector shown on the right.
- This is the core idea: a distance vector is nothing more than a flattened, cost-only summary of a routing tree — it hides the path itself and keeps only the total distance.

Initializing the Distance Vectors

- When a network first comes up, no router yet knows the paths beyond its immediate neighbours. Each router therefore builds a first, very simple vector using only local information:
- The cost to itself is set to 0.
- The cost to each node it is directly connected to is set to the cost of that link.
- The cost to every other (not directly connected) node is set to infinity (∞), meaning "unknown for now."
-

Figure 4.60 *The first distance vector for an internet*

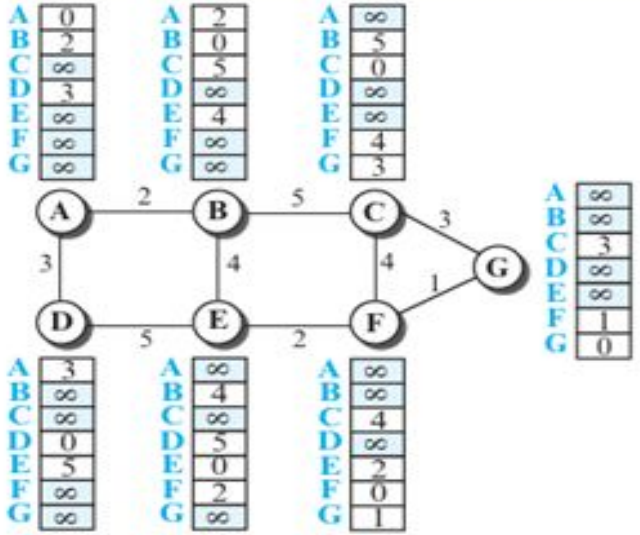
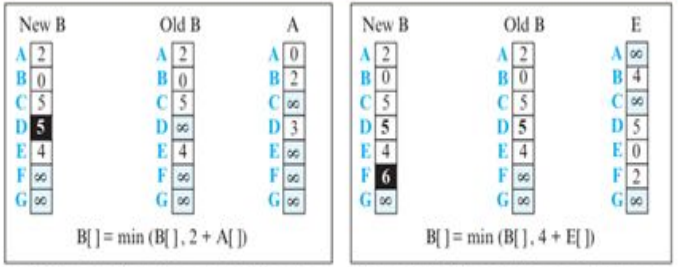


Figure 4.61 *Updating distance vectors*



a. First event: B receives a copy of A's vector.

b. Second event: B receives a copy of E's vector.

Note:
X[ ]: the whole vector

- In the seven-node network shown, node A is directly linked to B (cost 2) and D (cost 3), so A's first vector shows A = 0, B = 2, D = 3, and everything else (C, E, F, G) as ∞.
- Each of the other routers — B, C, D, E, F, G — builds its own first vector the same way, using only the costs of the links touching it.
- These initial vectors are incomplete, but they are enough to start the exchange process that will eventually let every router learn the shortest cost to every destination.

Exchanging and Updating Distance Vectors

After initialization, routers exchange their distance vectors with their immediate neighbours, and each router updates its own vector using a simple comparison rule — the Bellman-Ford idea:

1. A router receives a neighbour's complete distance vector.
2. For every destination X, it adds the cost of the link to that neighbour to the neighbour's reported cost for X.
3. If this new total is smaller than the router's current cost to X, the router replaces its entry with the smaller value and remembers the neighbour as the next hop for X.
4. If the new total is not smaller, the router keeps its existing entry.

This is written compactly as: New[X] = min(Old[X], cost(to neighbour) + Neighbour[X])

- In the left panel, B receives A's vector. Using B[X] = min(B[X], 2 + A[X]), B updates its entry for D from ∞ to 5 (2 + A's cost of 3 to D), because 5 is smaller than the old ∞. All of B's other entries either stay the same or are already smaller than what A offers, so they are left unchanged.
- In the right panel, B later receives E's vector. Using B[X] = min(B[X], 4 + E[X]), B updates its entry for F from ∞ to 6 (4 + E's cost of 2 to F), since 6 improves on ∞. Each time a router receives a neighbour's vector, it repeats this same comparison for every destination, one column at a time.

Overall Procedure

1. Build the initial vector: cost 0 to self, link cost to direct neighbours, ∞ to everyone else (Figure 2).
2. Send this vector to every directly connected neighbour.
3. Whenever a vector arrives from a neighbour, apply the update rule New[X] = min(Old[X], cost-to-neighbour + Neighbour[X]) to every destination X (Figure 3).
4. If any entry changed, send the updated vector out to all neighbours again, since they may now be able to improve their own vectors.
5. Repeat steps 3–4 until no router's vector changes any further — at this point every router has converged on the shortest-cost path to every destination, matching the kind of tree-derived vector shown in Figure 1.

- **The name of the distance vector** identifies the **root (source) node.**
- **The index (destination)** identifies the destination router.
- **Each value** represents the **least cost** from the root to that destination.

For example,

- Cost(A → B) = 2
- Cost(A → D) = 3
- Cost(A → G) = 9

Unlike a least-cost tree, the distance vector **does not show the actual path**. It stores **only the minimum cost.**

Later, this distance vector can be converted into a **forwarding table**, which contains the next-hop information.

Creation of Initial Distance Vectors

A distance vector must first be created before routers can exchange routing information.

When a router **boots (starts)**, it has only limited knowledge about the network.

It knows only

- Itself
- Its immediate neighbors
- The cost to each immediate neighbor

It has no information about routers beyond its neighbors.

Step 1: Discover Immediate Neighbors

Each router sends **greeting (Hello) messages** through all its interfaces.

Using these messages, it discovers

- The identity of each neighboring router.
- The distance (link cost) to each neighbor.

Step 2: Create the Initial Distance Vector

After discovering its neighbors, the router creates a simple distance vector.

The entries are filled as follows:

- Distance to itself = **0**
- Distance to directly connected neighbors = **Actual link cost**
- Distance to all other routers = ∞ **(Infinity)**

Infinity means that the router **does not yet know any route** to those destinations.

Figure 4.60 *The first distance vector for an internet*

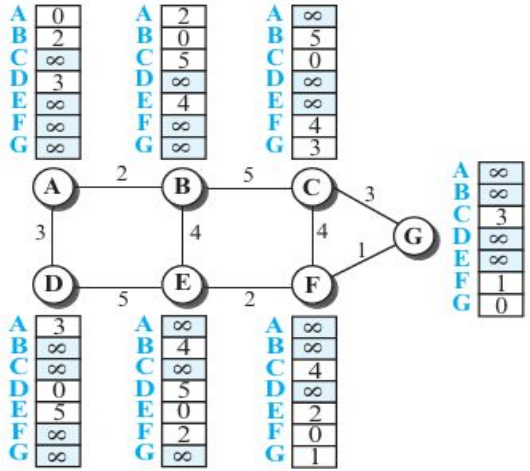


Table 4.4 *Distance-Vector Routing Algorithm for A Node*

```
1  Distance_Vector_Routing ( )
2  {
3      // Initialize (create initial vectors for the node)
4      D[myself] = 0
5      for (y = 1 to N)
6      {
7          if (y is a neighbor)
8              D[y] = c[myself][y]
9          else
10             D[y] = ∞
11     }
12     send vector {D[1], D[2], ..., D[N]} to all neighbors
13     // Update (improve the vector with the vector received from a neighbor)
14     repeat (forever)
15     {
16         wait (for a vector D_w from a neighbor w or any change in the link)
17         for (y = 1 to N)
18         {
19             D[y] = min [D[y], (c[myself][w] + D_w[y])]          // Bellman-Ford equation
20         }
21         if (any change in the vector)
22             send vector {D[1], D[2], ..., D[N]} to all neighbors
23     }
24 } // End of Distance Vector
```

Distance Vector Routing Algorithm for a Node

1 Initialize (Create Initial Vectors)

Myself (Source Node)



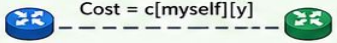
D[myself] = 0

For each destination y (1 to N)

2 Set Initial Costs

For each destination y (1 to N)

If y is a neighbor

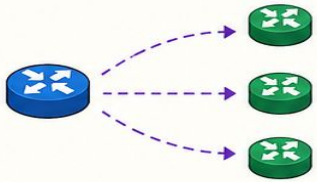


D[y] = c[myself][y]

Else (y is not a neighbor)

D[y] = ∞

3 Send Vectors to Neighbors



Send vector (D[1], D[2], …, D[N]) to all neighbors

4 Wait for Updates



Wait (forever) for a vector D_w from a neighbor w or any change in the link

5 Update Using Bellman-Ford Equation

For each destination y (1 to N)

$$D[y] = \min\{ D[y], c[myself][w] + D_w[y] \}$$

$D[y]$ = current cost to y

$c[myself][w]$ = cost from myself to neighbor w

$D_w[y]$ = cost from neighbor w to y

6 Check for Any Change

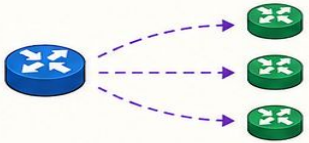
After updating all destinations

Any change in the vector?

Yes (Updated)

No (No Change)

7 If Changed, Send Updated Vector



If any change in the vector, send updated vector (D[1], D[2], …, D[N]) to all neighbors

8 Repeat Until Convergence



Repeat steps 4 to 7 until no more updates occur (Convergence)

Legend

- My Router (Source Node)
- Neighbor Router
- Vector Exchange



Count to Infinity Problem

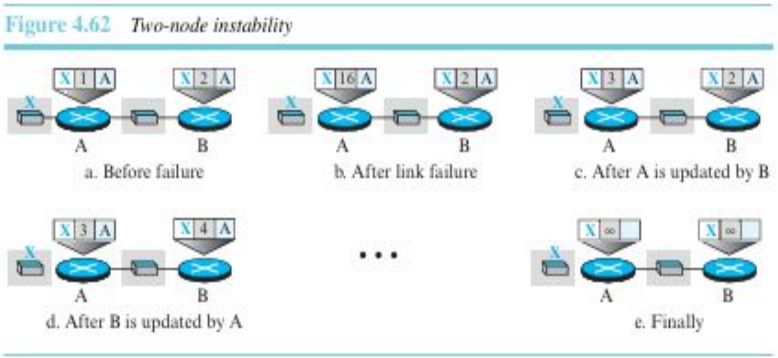
- One drawback of **Distance-Vector Routing** is the **Count-to-Infinity Problem**.

Definition

- When the cost of a route **decreases (good news)**, the information spreads quickly through the network.
- However, when the cost **increases or a link fails (bad news)**, the information spreads slowly.
- As a result, routers may continue to believe that a failed route is still available, causing incorrect routing information.
- Eventually, the route cost keeps increasing step by step until it reaches ∞ **(Infinity)**.
- This slow propagation of bad news is called the **Count-to-Infinity Problem**.

Suppose a link between two routers fails.

- The router directly connected to the failed link immediately updates its routing table.
- Other routers are unaware of the failure.
- They continue advertising the old route.
- Neighboring routers mistakenly believe an alternate path exists.
- The route cost increases gradually after every update until it finally becomes **Infinity**.During this time, routing becomes unstable.



Two-Node Loop

The **Two-Node Loop** is a common example of the **Count-to-Infinity Problem.**

Initial State (Figure 4.62a)

- Router **A** and Router **B** both know how to reach destination **X.**
- Router **A** is directly connected to **X.**

Step 1: Link Failure (Figure 4.62b)

The link between **A** and **X** fails.

- Router **A** updates its routing table.
- The cost to reach **X** becomes ∞.

Step 2: Incorrect Update (Figure 4.62c)

Before receiving A's updated table,

Router **B** sends its old routing table to Router **A.**

Since B still believes that **X** is reachable,

Router **A** mistakenly assumes that B has another route to X.

A updates its routing table to reach **X via B.**

Step 3: Cost Increases (Figure 4.62d)

Router **A** sends its updated table to **B.**

Router **B** now assumes that **A** has found a better route.

B updates its routing table.

The route cost increases gradually.

Step 3: Cost Increases (Figure 4.62d)

Router **A** sends its updated table to **B**.

Router **B** now assumes that **A** has found a better route.

B updates its routing table.

The route cost increases gradually.

Step 4: Count to Infinity (Figure 4.62e)

This exchange continues repeatedly.

The routing cost becomes

```
2 → 3 → 4 → 5 → ...
```

until it finally reaches

```
∞
```

Only then do both routers realize that **X is unreachable**.

Problem During This Time

Before reaching infinity,

- Router **A** believes the path to X is through **B**.
- Router **B** believes the path to X is through **A**.

If a packet for X arrives,

```
A → B → A → B → A → B ...
```

The packet continuously moves between the two routers without reaching its destination.

This creates a **Two-Node Routing Loop**.

Split Horizon

A solution to the two-node loop problem is the **Split Horizon** technique.

Principle

A router **does not advertise a route back to the neighbor from which it learned that rout**

In other words,

> **Never send routing information back in the direction from which it was received.**

Example

Suppose Router **B** learned that the route to **X** is through **A**.

When B sends its routing table to A,

it **does not include** the route to X.

Since Router A receives no incorrect information,

it keeps the cost to X as ∞.

Later,

A informs B that X is unreachable.

B also updates its table.

The network becomes stable after the first update.

Advantage

- Prevents two-node routing loops.
- Speeds up convergence.
- Reduces unnecessary routing updates.

Limitation

Split Horizon has one drawback.

Most routing protocols use **timers**.

If a router does not receive updates for a route within a certain time

it removes that route from its routing table.

When Router **B** does not advertise the route to X,

Router **A** cannot determine whether

- B intentionally omitted the route because of **Split Horizon**, or
- B no longer knows the route because it has expired.

This ambiguity can cause confusion.

Example

Suppose Router **B** learned the route to **X** from **A**.

Instead of deleting the entry,

B sends

```
Destination X = ∞
```

to Router A.

Router A immediately understands that

B is **not offering an alternate route**.

Therefore,

A keeps X as unreachable.

Advantages of Poisoned Reverse

- Removes ambiguity present in Split Horizon.
- Prevents two-node routing loops.
- Faster detection of failed routes.
- Improves routing stability.

Three-Node Instability

Although **Split Horizon** and **Poisoned Reverse** successfully prevent **two-node loops**, they **cannot completely eliminate routing loops involving three or more routers**.

For example,

```
A → B → C → A
```

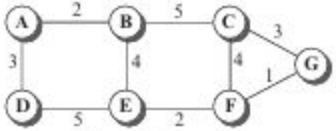
In such cases,

incorrect routing information may continue circulating among the three routers.

Therefore,

three-node instability cannot always be prevented using Split Horizon and Poisoned Reverse alone.

Figure 4.63 *Example of a link-state database*

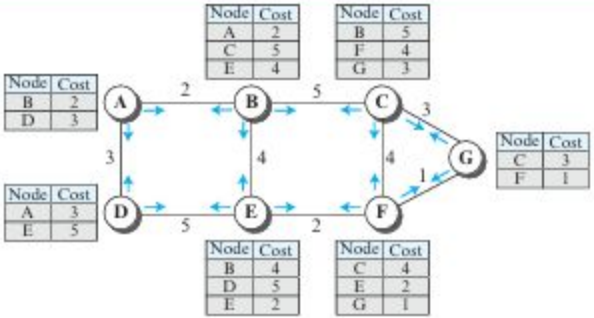


a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

Figure 4.64 *LSPs created and sent out by each node to build LSDB*



Link-State (LS) Routing

Link-State Routing (LS) is a routing algorithm that creates the **least-cost tree** and the corresponding **forwarding table** for each router.

Unlike **Distance-Vector Routing**, where routers exchange their entire routing tables with their neighbors, **Link-State Routing** allows each router to build a **complete map of the network** and independently calculate the shortest path to every destination.

What is Link State?

A **link** is the connection (edge) between two routers.

The **state of a link** is determined by its **cost (weight)**.

- **Low cost** → Preferred link (better route)
- **High cost** → Less preferred link
- **Cost = ∞ (Infinity)** → Link is unavailable or broken

Thus, the routing algorithm uses the **cost of each link** to determine the best path.

Link-State Database (LSDB)

To create the shortest-path tree, every router must know the **complete network topology**.

The collection of information about **all the links and their costs** in the network is called the **Link-State Database (LSDB)**.

> **Definition:**
> A **Link-State Database (LSDB)** is a database that contains the state (cost) of every link in the network.

There is **only one logical LSDB for the entire network**, but **each router maintains an identical copy** of it.

Because every router has the same LSDB, each router has a complete map of the network and can independently calculate the shortest paths.

Representation of LSDB (Figure 4.63)

The LSDB can be represented as a **two-dimensional matrix.**

- Each **row and column** represent routers.
- Each **cell** contains the **cost of the direct link** between two routers.

For example,

Link	Cost
A – B	2
A – D	3
B – C	5
C – G	3

If two routers are **not directly connected**, the cost is ∞ **(Infinity).**

Thus, the LSDB represents the complete network topology.

How is the LSDB Created?

The LSDB is created using a process called **Flooding**.

Step 1: Discover Immediate Neighbors

Each router sends **Hello (Greeting) messages** through all its interfaces.

Using these messages, it discovers:

- The identity of each neighboring router.
- The cost of the link to each neighbor.

Step 2: Create a Link-State Packet (LSP)

The information collected by a router is stored in a **Link-State Packet (LSP).**

An **LSP** contains:

- Router identity
- List of neighboring routers
- Cost of each link
- Sequence number (to identify the latest information)

Step 3: Flooding the LSP

Each router sends its LSP to **all of its immediate neighbors.**

When a neighboring router receives the LSP, it:

- Stores the LSP if it is new.
- Forwards (floods) it to all its neighbors **except the one from which it was received.**

This process continues until every router has received the LSP.

This method is called **Flooding.**

Handling Duplicate LSPs

A router may receive the same LSP multiple times.

To avoid unnecessary processing, every LSP contains a **sequence number.**

When a router receives an LSP:

- If the received LSP is **older**, it is discarded.
- If the received LSP is **newer**, the old LSP is replaced.
- The new LSP is then forwarded to all neighboring routers except the incoming interface.

This prevents infinite flooding and ensures that every router has the **latest network information.**

Result of Flooding

After all routers exchange their LSPs,

- every router has received the latest information from every other router,
- every router builds the **same Link-State Database (LSDB)**,
- every router has a complete map of the network.

Difference Between Distance Vector and Link-State Routing

Distance-Vector Routing	Link-State Routing
Router shares its entire routing table with neighbors.	Router shares only information about its directly connected links (LSPs).
Routers know only the information received from neighbors.	Every router knows the complete network topology.
Uses the **Bellman-Ford Algorithm**.	Uses **Dijkstra's Algorithm**.
Slower convergence.	Faster convergence.
Count-to-Infinity problem may occur.	No Count-to-Infinity problem.

Key Difference

Distance-Vector Routing:
Each router tells **its neighbors** what it knows about the **whole network**.

Link-State Routing:
Each router tells the **entire network** what it knows about **its immediate neighbors**.

Formation of Least-Cost Tree

Once the LSDB has been created,

each router independently builds its own **Least-Cost Tree** using **Dijkstra's Algorithm**.

Although all routers have the same LSDB,

each router chooses **itself as the root**.

Therefore,

each router produces a different least-cost tree.

Dijkstra's Algorithm

Dijkstra's Algorithm is an **iterative shortest-path algorithm.**

It builds the least-cost tree one node at a time.

Step 1: Select the Root

The router selects **itself** as the root node.

Initially,

- only the root is included in the tree.
- the cost to every other router is obtained from the LSDB.

Step 2: Select the Nearest Node

Among all routers **not yet included** in the tree,

choose the router having the **minimum total cost** from the root.

Add that router to the tree.

After adding it,

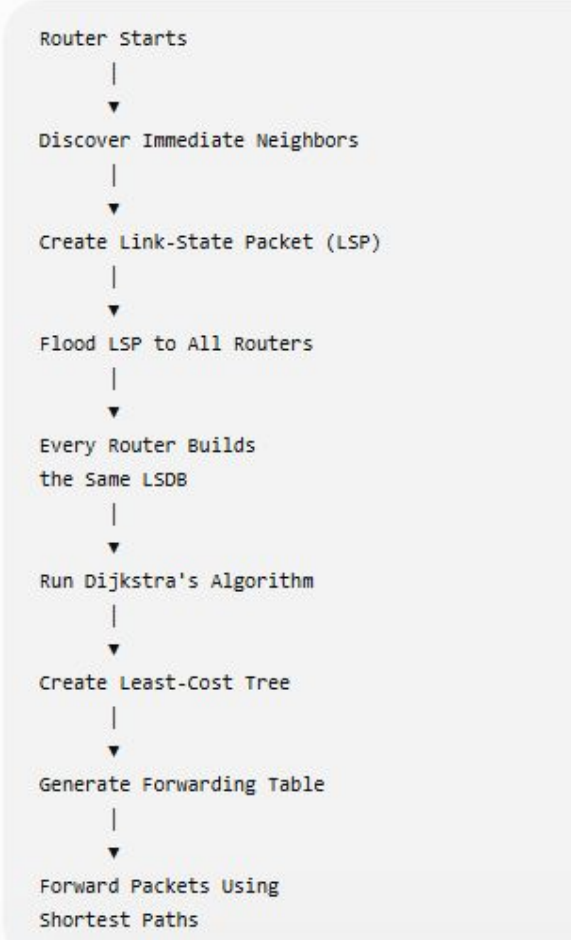
update the costs of all remaining routers because a shorter path may now exist.

Step 3: Repeat

Repeat Step 2 until **all routers** have been added to the tree.

The completed tree represents the **shortest paths** from the source router to every destination.

Flow of Link-State Routing



- **Link-State Routing (LS)** builds the shortest paths using complete network information.
- The **cost of a link** represents its **state**.
- A **Link-State Database (LSDB)** stores the cost of every link in the network.
- Every router maintains an **identical copy** of the LSDB.
- Routers exchange **Link-State Packets (LSPs)** using **Flooding**.
- **Sequence numbers** ensure that only the latest LSPs are accepted and prevent endless flooding.
- After flooding, every router has the same view of the network.
- Each router independently runs **Dijkstra's Algorithm** to build its own **least-cost tree**.
- Link-State Routing provides **faster convergence**, **more accurate routing**, and **avoids the Count-to-Infinity problem**, making it more efficient than Distance-Vector Routing.

Table 4.5 *Dijkstra's Algorithm*

```
1  Dijkstra's Algorithm ( )
2  {
3      // Initialization
4      Tree = {root}                    // Tree is made only of the root
5      for (y = 1 to N)                 // N is the number of nodes
6      {
7          if (y is the root)
8              D[y] = 0                 // D[y] is shortest distance from root to node y
9          else if (y is a neighbor)
10             D[y] = c[root][y]        // c[x][y] is cost between nodes x and y in LSDB
11         else
12             D[y] = ∞
13     }
14     // Calculation
15     repeat
16     {
17         find a node w, with D[w] minimum among all nodes not in the Tree
18         Tree = Tree ∪ {w}            // Add w to tree
19         // Update distances for all neighbor of w
20         for (every node x, which is neighbor of w and not in the Tree)
21         {
22             D[x] = min{D[x], (D[w] + c[w][x])}
23         }
24     } until (all nodes included in the Tree)
25 } // End of Dijkstra
```

Path-Vector Routing (PV Routing)

Both **Link-State (LS)** and **Distance-Vector (DV)** routing use the **least-cost path** to forward packets. However, the least-cost path is not always the best choice.

For example, a sender may want to **avoid certain routers or networks** because:

- They are not secure.
- They belong to a rival organization.
- They inspect or monitor packets.

Least-cost routing cannot enforce these policies because it always selects the lowest-cost route.

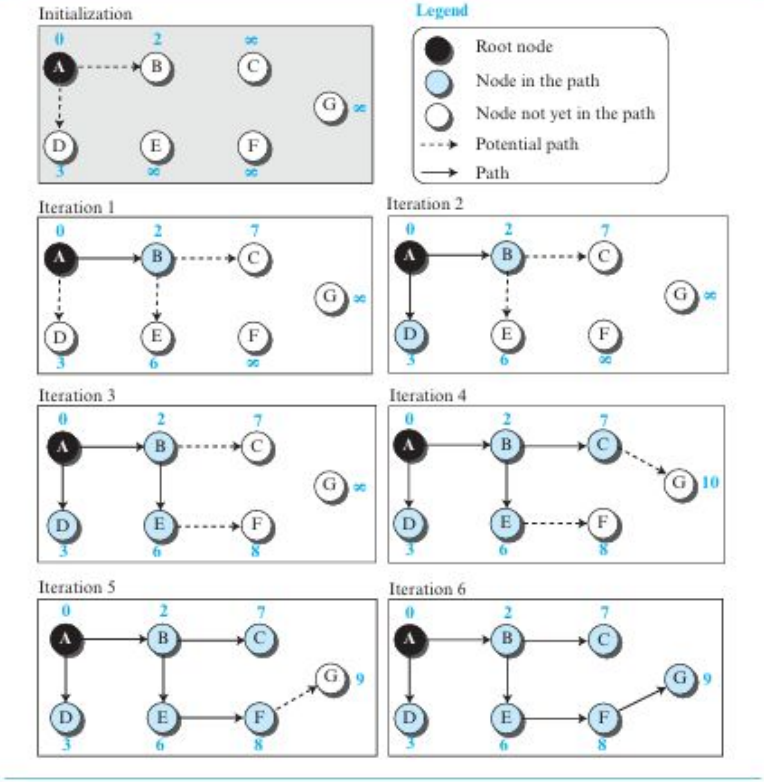
To overcome this limitation, **Path-Vector (PV) Routing** was developed.

Definition

Path-Vector Routing is a routing algorithm in which the **source selects the best route based on its own policies instead of the least-cost path**.

In other words, **the source controls the path** taken by the packet.

Figure 4.65 *Least-cost tree*



Features of Path-Vector Routing

- Does **not depend on least-cost routing**.
- Allows the sender to apply **routing policies**.
- The sender can **avoid specific routers or networks**.
- The complete path to the destination is known.
- Mainly used for **routing between Internet Service Providers (ISPs)**.

Spanning Tree in Path-Vector Routing

Like LS and DV routing, Path-Vector Routing also creates a **spanning tree**.

However,

- It is **not a least-cost spanning tree**.
- It is the **best spanning tree according to the sender's policy**.

If multiple routes exist, the source chooses the one that best satisfies its policy.

Common Routing Policies

Some common policies are:

1. **Minimum number of intermediate nodes (hops).**
2. **Avoid specific routers or networks** while forwarding packets.
3. Any other policy defined by the source.

Creation of Spanning Trees

Path-Vector Routing is an **asynchronous** and **distributed** routing algorithm.

The spanning tree is created gradually as follows:

1. When a router starts (boots), it creates an **initial path vector** using information from its **immediate neighbors**.
2. The router sends **greeting messages** to its neighbors to collect routing information.
3. It creates its **path vector table**.
4. The router sends its path vector to all immediate neighbors.
5. When a router receives a path vector from a neighbor, it updates its own table according to its routing policy.
6. This process continues until every router has the best policy-based routes.

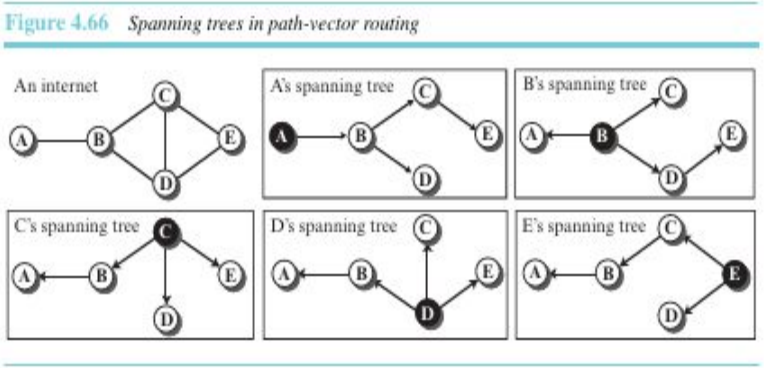


Figure 4.66 *Spanning trees in path-vector routing*

Path-Vector Update Equation

$$\text{Path}(x, y) = \text{Best}\,\{\text{Path}(x, y),\ [x + \text{Path}(v, y)]\}$$

where

- **x** = source node
- **y** = destination node
- **v** = neighboring router
- **+** = add node **x** to the beginning of the received path
- **Best** = path that satisfies the routing policy

Advantages

- Source can control packet routing.
- Supports security and administrative policies.
- Avoids unwanted routers or networks.
- Complete path information is available.
- Prevents routing through restricted areas.

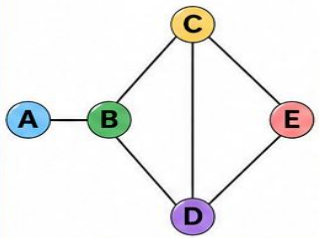
Disadvantages

- Routing tables are larger because complete paths are stored.
- Requires more memory than Distance-Vector routing.
- Path information increases routing overhead.

FIGURE 4.67 PATH VECTORS MADE AT BOOTING TIME

At boot time, each router knows only its immediate neighbors.

NETWORK TOPOLOGY



Links: A–B B–C B–D C–D C–E D–E

A Router A

Destination	Path
A	A
B	B,A
C	–
D	–
E	–

B Router B

Destination	Path
A	B,A
B	B
C	B,C
D	B,D
E	–

C Router C

Destination	Path
A	–
B	C,B
C	C
D	C,D
E	C,E

D Router D

Destination	Path
A	–
B	D,B
C	D,C
D	D
E	D,E

E Router E

Destination	Path
A	–
B	–
C	E,C
D	E,D
E	E

1. Each router creates its initial path vector using only its neighbors.
2. It sends the path vector to all immediate neighbors.
3. Arrows show exchange of path vectors between neighbors.

FIGURE 4.68 UPDATING PATH VECTORS

Router C updates its path vector when it receives path vectors from its neighbors.

1 Event 1: C receives a copy of B's vector

Old Path Vector of C (before update)		Vector received from B		New Path Vector of C (after update)	
Destination	Path	Destination	Path	Destination	Path
A	–	A	B,A	A	C,B,A
B	C,B	B	B	B	C,B
C	C	C	B,C	C	C
D	C,D	D	B,D	D	C,D
E	C,E	E	–	E	C,E

Update rule:
C[ ] = best { C[ ], C + B[ ] }
(Choose the best path according to the policy)

C does not know A.
From B's vector, C learns that B can reach A through B,A. So C adds itself at the beginning.

New path to A becomes C,B,A.

2 Event 2: C receives a copy of D's vector

Current Path Vector of C (before update)		Vector received from D		Path Vector of C after comparing	
Destination	Path	Destination	Path	Destination	Path
A	C,B,A	A	–	A	C,B,A
B	C,B	B	D,B	B	C,B
C	C	C	D,C	C	C
D	C,D	D	D	D	C,D
E	C,E	E	D,E	E	C,E

C compares new possible paths via D:

- To B: via D → C,D,B (longer) ✗ Keep C,B
- To C: via D → C,D,C (contains C) ✗ Reject (loop)
- To D: via D → C,D (same) ✓ Keep C,D
- To E: via D → C,D,E (longer) ✗ Keep C,E
- To A: D does not know A. No change.

No better path found via D.
Table remains unchanged.

KEY POINTS

- Each router builds a path vector and shares it with neighbors.
- When a router receives a vector, it adds its own name at the beginning of each path.
- It selects the **BEST path** according to its policy (e.g., minimum hops, avoid certain nodes, etc.).
- Any path that already contains the router's own name is rejected to avoid loops.
- This process continues until all routers have the best policy-based routes.

POLICY EXAMPLES

- Choose path with minimum number of nodes (hops).
- Avoid certain routers or networks.
- Prefer secure or trusted networks.
- Choose route with minimum cost, delay, etc.

LOOP AVOIDANCE

C → D → C → B → A

Path contains C again → Loop!
Such paths are rejected.

Rule: Never accept a path that already contains your own node.

FLOW OF PATH–VECTOR ROUTING

Router boots → Create initial path vector (only neighbors) → Send path vector to neighbors → Receive neighbors' path vectors → Add own name, compare and choose best path (policy) → Update table and send to neighbors → Repeat until stable (best routes found)

```
1  Path_Vector_Routing ( )
2  {
3      // Initialization
4      for (y = 1 to N)
5      {
6          if (y is myself)
7              Path[y] = myself
8          else if (y is a neighbor)
9              Path[y] = myself + neighbor node
10         else
11             Path[y] = empty
12     }
13     Send vector {Path[1], Path[2], …, Path[y]} to all neighbors
14     // Update
15     repeat (forever)
16     {
17         wait (for a vector Path_w from a neighbor w)
18         for (y = 1 to N)
19         {
20             if (Path_w includes myself)
21                 discard the path                    // Avoid any loop
22             else
23                 Path[y] = best {Path[y], (myself + Path_w[y])}
24         }
25         If (there is a change in the vector)
26             Send vector {Path[1], Path[2], …, Path[y]} to all neighbors
27     }
28 } // End of Path Vector
```

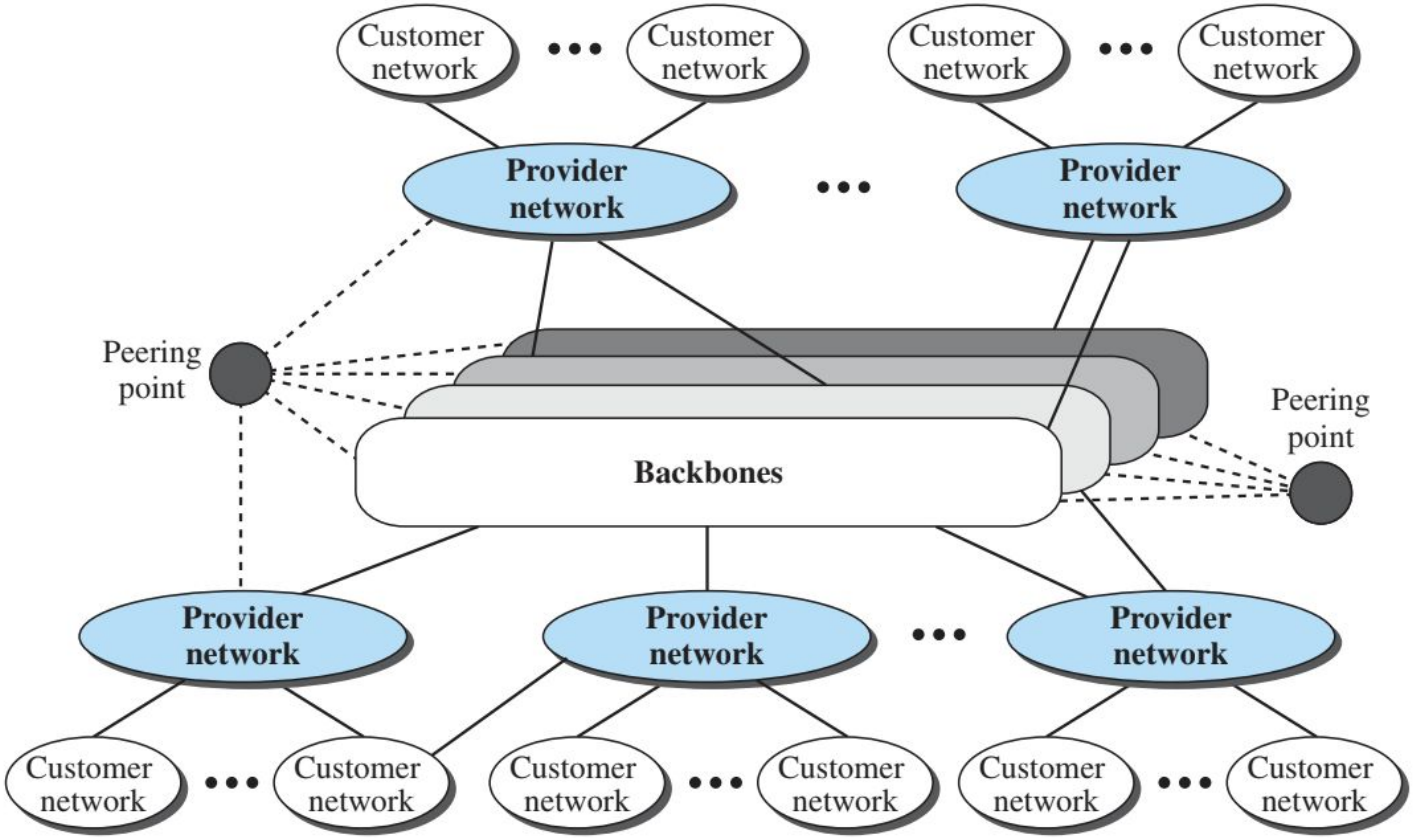
Unicast Routing Protocols

- A **unicast routing protocol** is a protocol that helps routers determine the best path for sending data from one source host to one destination host in an IP network.
- Unlike a routing algorithm, which only calculates the shortest or best path, a routing protocol also defines how routers exchange routing information, how routing tables are maintained and updated, and how routers communicate with one another.
- In the Internet, three important unicast routing protocols are used: **Routing Information Protocol (RIP)**, which is based on the **Distance Vector Algorithm**; **Open Shortest Path First (OSPF)**, which is based on the **Link State Algorithm**; and **Border Gateway Protocol (BGP)**, which is based on the **Path Vector Algorithm**.

Internet Structure

- The Internet is a collection of millions of interconnected networks.
- Earlier, the Internet had a simple tree-like structure with a single backbone, but today it consists of multiple backbone networks operated by different organizations.
- These backbone networks provide global Internet connectivity and are connected through **peering points**, which allow communication between different backbone providers.
- Below the backbone networks are **provider networks**, commonly called Internet Service Providers (ISPs), which obtain connectivity from the backbone and provide Internet services to organizations and users.
- At the lowest level are **customer networks**, such as homes, colleges, offices, and companies, which access the Internet through provider networks. Therefore, the Internet follows a hierarchical structure consisting of backbone networks, provider networks, and customer networks.

Figure 4.69 *Internet structure*



Hierarchical Routing

- Since the Internet contains an enormous number of routers and networks, using a single routing protocol throughout the Internet is not practical. The first reason is **scalability**. A single routing table containing information about every network in the world would become extremely large, making routing table searches slow and generating excessive routing updates.
- The second reason is **administrative control**. Different organizations have different routing requirements, network policies, hardware, and security rules. Therefore, each organization should be able to manage its own network independently.
- To solve these problems, the Internet uses **hierarchical routing**. In hierarchical routing, the Internet is divided into several **Autonomous Systems (ASs)**. Each Autonomous System manages its own internal routing using an **Interior Gateway Protocol (IGP)** such as **RIP** or **OSPF**, while communication between different Autonomous Systems is handled by an **Exterior Gateway Protocol (EGP)**, namely **BGP**.

Autonomous System (AS)

- An **Autonomous System (AS)** is a collection of networks and routers that are managed by a single administrative organization. Each AS is assigned a unique **Autonomous System Number (ASN)** by **ICANN**, which uniquely identifies the network on the Internet.
- Autonomous Systems are classified into three types based on how they connect with other ASs.
- A **Stub AS** has only one connection to another Autonomous System. Data traffic can either start or end in a Stub AS, but it cannot pass through it. Home networks and college networks are common examples.
- A **Multihomed AS** is connected to two or more Autonomous Systems but does not allow transit traffic to pass through it. Large organizations that connect to multiple Internet Service Providers for reliability are examples of Multihomed ASs.
- A **Transient AS** has multiple connections to other Autonomous Systems and also allows traffic from one AS to another. Backbone networks and Internet Service Providers are examples of Transient ASs.

Routing Information Protocol (RIP)

The **Routing Information Protocol (RIP)** is one of the oldest and simplest routing protocols used inside an Autonomous System. It is an **Interior Gateway Protocol (IGP)** based on the **Distance Vector Routing Algorithm**. Instead of considering factors such as bandwidth or delay, RIP uses **Hop Count** as its routing metric. Because of its simplicity, RIP is mainly used in small and medium-sized networks.

Hop Count in RIP

The routing metric used by RIP is called **Hop Count**. A hop represents the number of routers that a packet must pass through before reaching the destination network. Every time a packet passes through one router, the hop count increases by one.

For example, if a packet travels through three routers before reaching the destination network, the hop count is **3**.

In RIP, the maximum allowable hop count is **15**. Any destination requiring **16 hops** is considered **unreachable (infinity)**. Because of this limitation, RIP is suitable only for small Autonomous Systems.

- **Forwarding Table**

Every RIP router maintains a **forwarding table**, which contains the information required to forward packets toward their destination. Each entry in the forwarding table consists of three fields:

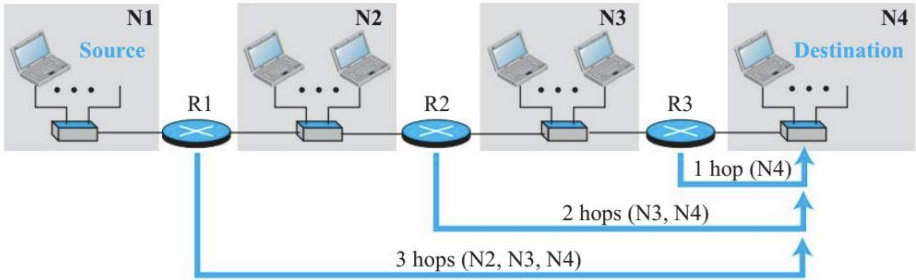
- **Destination Network**
- **Next Router (Next Hop)**
- **Hop Count (Cost)**

When a packet arrives at a router, the router searches the forwarding table, identifies the destination network, forwards the packet to the specified next router, and repeats the process until the destination is reached.

RIP Implementation

- RIP operates as an **Application Layer protocol**, although it is responsible for creating routing information used by the IP layer. RIP messages are carried using the **User Datagram Protocol (UDP)** and use the well-known **port number 520**.
- Two versions of RIP exist: **RIP Version 1 (RIP-1)** and **RIP Version 2 (RIP-2)**. RIP-2 is an enhanced version that supports additional routing information and is the version commonly used today.

Figure 4.70 *Hop counts in RIP*

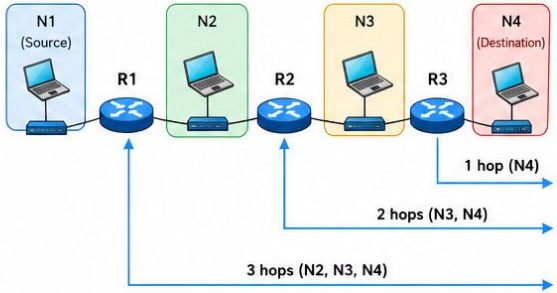


RIP (Routing Information Protocol)

Overview

RIP is a distance-vector routing protocol used within an Autonomous System (AS).
It uses hop count (number of routers) as the metric. The maximum hop count is 15; 16 means unreachable (infinity). RIP runs over UDP on port 520.

Hop Count in RIP



Forwarding Tables

Forwarding Table for R1

Destination Network	Next Router	Cost in Hops
N1	—	1
N2	—	1
N3	R2	2
N4	R2	3

Forwarding Table for R2

Destination Network	Next Router	Cost in Hops
N1	R1	2
N2	—	1
N3	—	1
N4	R3	2

Forwarding Table for R3

Destination Network	Next Router	Cost in Hops
N1	R2	3
N2	R2	2
N3	—	1
N4	—	1

RIP forwarding table tells the next router and the hop count to reach each destination network.

Key Points

- **Metric:** Hop count (1 hop = 1 router). Max 15 hops, 16 = infinity.
- **Updates:** Entire routing table is shared with neighbors every 25–35 seconds or when there is a change.
- **Timers:**
  - **Periodic Timer:** Sends updates every 25–35 seconds.
  - **Expiration Timer:** If no update for a route in 180 seconds, mark as unreachable (16).
  - **Garbage Collection Timer:** Route is removed after 120 seconds.
- **Maximum hops:** 15 (16 means unreachable).

RIP Message Format (RIP-2)

1 Byte	1 Byte	2 Bytes	2 Bytes	4 Bytes	4 Bytes	4 Bytes	4 Bytes
Command (1=Request, 2=Response)	Version (2)	Must be Zero	Address Family (2=IP)	Route Tag	IP Address (Network)	Subnet Mask	Next Hop (0.0.0.0 if unknown)	Metric (Hop Count)

This entry format is repeated for each route in the message.

How RIP Works (Simplified Algorithm)

1 Send Updates
Router sends its entire routing table to neighbors periodically or on change.

2 Receive Updates
Neighbor receives update and adds 1 hop to each cost. +1

3 Compare Routes
Compare received routes with current (old) routes.
Old Table ↔ Received Table

4 Update Rules
Update the route if:
- New route not in old table
- Lower cost route
- Higher cost but same next router (route became worse)

5 Update Table
Sort and update the forwarding table.

RIP is simple and widely used in small to medium networks. For large networks, link-state protocols like OSPF are preferred.

Performance

Update Messages: Simple format, sent only to neighbors, so low traffic.

Convergence: Generally fast since max hops = 15; extensions like poison reverse & split horizon reduce loops and speed up.

Robustness: If one router has wrong info, it can affect others since routers depend on neighbor information.

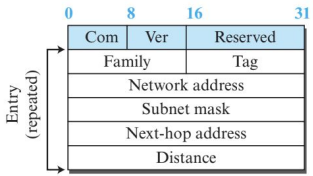
RIP Messages

Routers exchange routing information using two types of RIP messages: **Request** and **Response**.

A **Request message** is sent when a router starts for the first time or when some routing information has expired. The router requests routing information from its neighboring routers.

A **Response message**, also called an **Update message**, contains the router's routing table. A response may be **solicited**, meaning it is sent as a reply to a request, or **unsolicited**, meaning it is sent automatically every **30 seconds** or whenever there is a significant change in the routing table.

Figure 4.72 *RIP message format*



Fields

Com: Command, request (1), response (2)
Ver: Version, current version is 2
Family: Family of protocol, for TCP/IP value is 2
Tag: Information about autonomous system
Network address: Destination address
Subnet mask: Prefix length
Next-hop address: Address length
Distance: Number of hops to the destination

RIP Routing Algorithm

- In RIP, each router periodically sends its **entire forwarding table** to all neighboring routers.
- When a neighboring router receives this update, it first **adds one hop** to every received route because the packet has passed through one additional router.
- The router then compares the received routes with the routes already present in its forwarding table.
- If a destination does not exist in the table, a new entry is added. If the received route has a lower hop count than the existing route, the router replaces the old route with the new one.
- If the received route has a higher cost but arrives through the same next router, the existing entry is updated because it indicates that the network condition has changed. After updating, the forwarding table is sorted according to the destination network.

Timers Used in RIP

- RIP uses three timers to maintain accurate routing information.
- The **Periodic Timer** controls the transmission of routing updates. Each router sends its routing table every **25 to 35 seconds**, with the exact interval chosen randomly to prevent all routers from transmitting simultaneously.
- The **Expiration Timer** determines whether a route is still valid. If a router does not receive an update for a particular route within **180 seconds**, the route is marked as **unreachable**, and its hop count is changed to **16**.
- The **Garbage Collection Timer** removes invalid routes from the forwarding table. Instead of deleting an invalid route immediately, the router advertises it with a hop count of **16** for **120 seconds**.
- After this period, the invalid route is permanently removed from the routing table.
    -

Performance of RIP

- RIP is simple to configure and generates relatively little routing traffic because updates are exchanged only between neighboring routers.
- Since RIP is limited to a maximum of **15 hops**, routing convergence is generally fast in small networks.
- However, RIP may suffer from routing loops and the **count-to-infinity** problem, although techniques such as **Split Horizon** and **Poison Reverse** help reduce these issues.
- The main limitation of RIP is its poor scalability. Because it uses only hop count as its routing metric and supports a maximum of 15 hops, it is unsuitable for large enterprise networks or the global Internet.
- Modern large-scale networks therefore prefer protocols such as **OSPF** for intradomain routing and **BGP** for interdomain routing.

RIP is an Interior Gateway Protocol (IGP).
RIP is based on the Distance Vector Algorithm.
Routing metric used is Hop Count.
Maximum hop count = 15.
Hop count 16 = Unreachable (Infinity).
RIP runs at the Application Layer.
RIP uses UDP Port 520.
RIP sends routing updates every 25–35 seconds.
RIP uses three timers: Periodic (25–35 s), Expiration (180 s), and Garbage Collection (120 s).
RIP is suitable only for small networks due to its hop-count limitation.

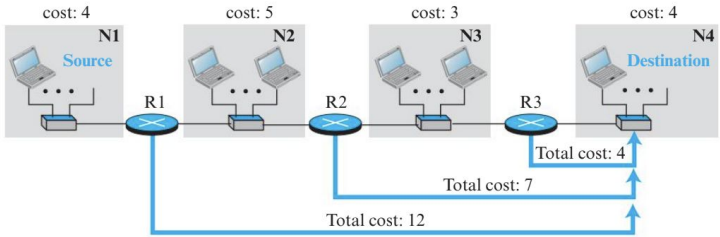
Open Shortest Path First (OSPF)

- **Open Shortest Path First (OSPF)** is an **Interior Gateway Protocol (IGP)** used for routing **within an Autonomous System (AS)**. Unlike RIP, which is based on the **Distance Vector Algorithm**, OSPF is based on the **Link-State Routing Algorithm**.
- OSPF is an **open standard protocol**, meaning that its specifications are publicly available and can be implemented by different networking vendors. It is widely used in medium and large enterprise networks because it provides faster convergence, better scalability, and more efficient routing than RIP.

Metric in OSPF

- OSPF determines the best route using a **cost metric**. The cost represents the expense of sending a packet from the source router to the destination network.
- Unlike RIP, which uses only **hop count**, OSPF allows the administrator to assign costs based on several factors such as **bandwidth (throughput), delay, round-trip time, reliability, or link quality**. The route with the **lowest total cost** is selected as the best path. If required, an administrator can also configure OSPF to use hop count as the routing metric.

Figure 4.74 *Metric in OSPF*



Forwarding Tables

- After collecting information about the entire network, every OSPF router uses **Dijkstra's Shortest Path Algorithm (SPF Algorithm)** to calculate the shortest path from itself to every destination network.
- Based on this shortest-path tree, the router creates its **forwarding table**.
- Each forwarding table contains the destination network, the next-hop router, and the total path cost. Unlike RIP, the forwarding table in OSPF uses the calculated **cost** instead of simple hop count, resulting in more efficient routing decisions.

Areas in OSPF

- OSPF is designed to support **both small and large Autonomous Systems**.
- In a large network, flooding Link-State Advertisements (LSAs) throughout the entire AS would generate excessive routing traffic. To overcome this problem, OSPF divides the Autonomous System into smaller logical groups called **Areas**.
- Each area maintains its own Link-State Database (LSDB), and routers exchange link-state information only within their area. This significantly reduces routing traffic and improves network performance.
- All areas are connected through a special area called the **Backbone Area (Area 0)**.
- The backbone area acts as the central connection between all other areas and is responsible for exchanging routing information between them. Routers that connect different areas are called **Area Border Routers (ABRs)**.

Figure 4.75 *Forwarding tables in OSPF*

Forwarding table for R1

Destination network	Next router	Cost
N1	——	4
N2	——	5
N3	R2	8
N4	R2	12

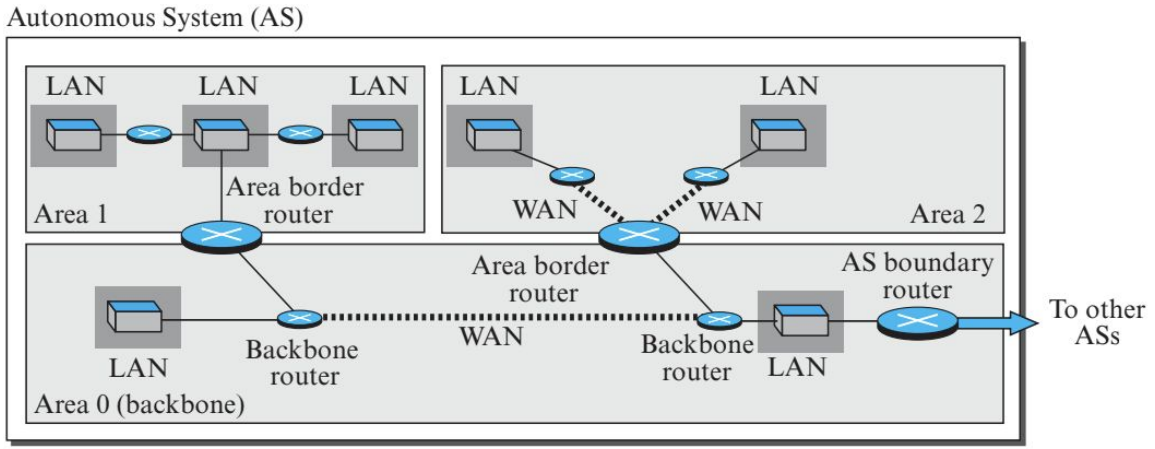
Forwarding table for R2

Destination network	Next router	Cost
N1	R1	9
N2	——	5
N3	——	3
N4	R3	7

Forwarding table for R3

Destination network	Next router	Cost
N1	R2	12
N2	R2	8
N3	——	3
N4	——	4

Figure 4.76 *Areas in an autonomous system*



Link-State Advertisements (LSAs)

OSPF uses **Link-State Advertisements (LSAs)** to exchange routing information. Each router advertises the state of its links to all routers within its area. These advertisements are stored in the **Link-State Database (LSDB)**, which is used by every router to calculate the shortest paths.

OSPF uses five main types of Link-State Advertisements.

1. Router Link Advertisement

A Router Link Advertisement describes the links directly connected to a router. It contains information about neighboring routers, connected networks, and the cost of each connection.

2. Network Link Advertisement

A Network Link Advertisement describes a shared network, such as an Ethernet LAN. Since a network cannot advertise itself, one router is selected as the **Designated Router (DR)** to send this advertisement on behalf of the network.

3. Summary Link to Network

This advertisement is generated by an **Area Border Router (ABR)**. It summarizes routing information from one area and advertises it to other areas through the backbone, enabling communication between different areas.

4. Summary Link to AS Border Router

This advertisement informs routers about the location of an **Autonomous System Border Router (ASBR)** that connects the current Autonomous System to external Autonomous Systems.

5. External Link Advertisement

This advertisement is generated by an **AS Border Router (ASBR)** to announce routes that exist **outside the current Autonomous System**. These routes are then distributed throughout the OSPF network.

OSPF Messages

OSPF uses five different message types to establish and maintain routing information.

Hello Message (Type 1)

The Hello message is used to discover neighboring routers, establish neighbor relationships, and periodically check whether neighboring routers are still active.

Database Description (DBD) Message (Type 2)

After neighboring routers are discovered, they exchange Database Description messages. These messages provide a summary of the Link-State Database so that routers can compare their databases.

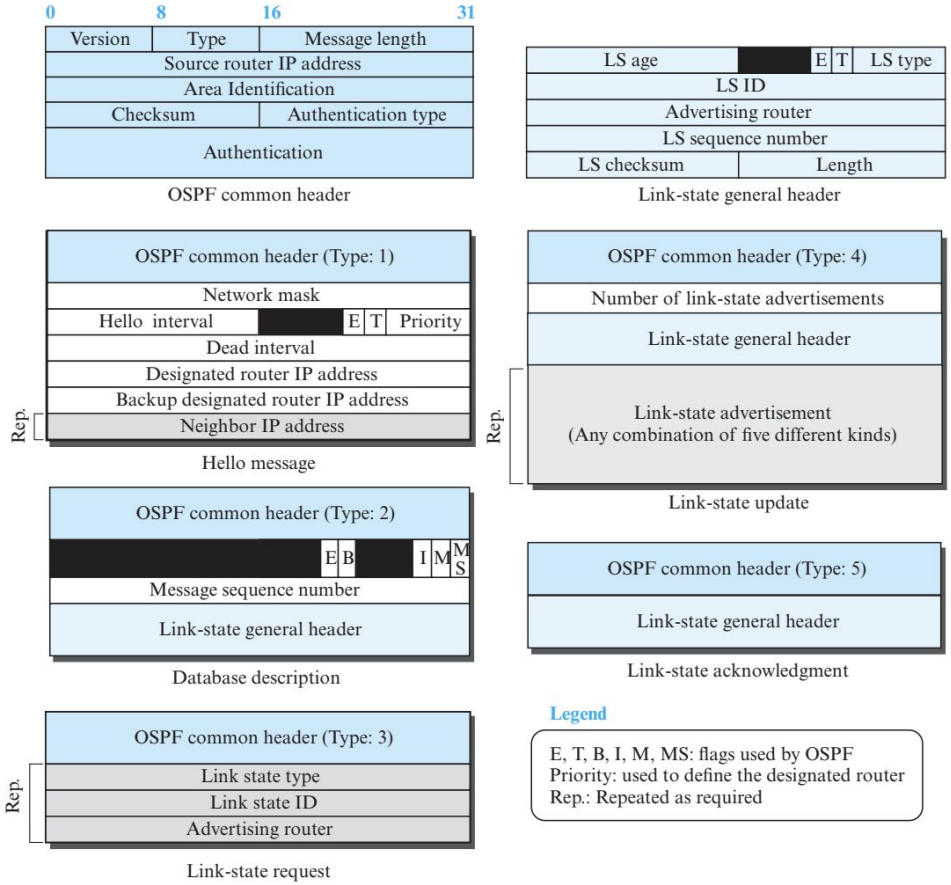
Link-State Request (LSR) Message (Type 3)

If a router finds that some link-state information is missing or outdated, it sends a Link-State Request message asking its neighbor for the required information.

Link-State Update (LSU) Message (Type 4)

The Link-State Update message contains one or more Link-State Advertisements (LSAs). It is the primary message used to distribute routing information and build the Link-State Database.

Figure 4.78 *OSPF message formats*



Link-State Acknowledgment (LSAck) Message (Type 5)

- Whenever a router receives a Link-State Update message, it sends a Link-State Acknowledgment message to confirm successful reception.
- This ensures reliable delivery of routing information.

Authentication

OSPF supports **authentication** to improve network security. Authentication ensures that only authorized routers are allowed to exchange OSPF routing information. This prevents unauthorized or malicious devices from becoming part of the routing system and sending false routing updates.

OSPF Routing Algorithm

OSPF follows the **Link-State Routing Algorithm**. Every router first discovers its neighboring routers and exchanges Link-State Advertisements. After receiving LSAs from all routers in the area, each router builds an identical **Link-State Database (LSDB)**. Using this database, every router independently executes **Dijkstra's Shortest Path First (SPF) Algorithm** to construct the shortest-path tree. Finally, the router creates its forwarding table based on the shortest paths calculated from the tree.

Performance of OSPF

OSPF provides excellent performance in medium and large networks. Although Link-State Advertisements are more complex than RIP update messages and require additional bandwidth during flooding, OSPF converges much faster because each router independently calculates the shortest path using the complete Link-State Database. OSPF is also more robust than RIP because routers do not depend solely on information received from neighboring routers. Even if one router fails, other routers continue to operate correctly using their own copy of the Link-State Database.

Advantages of OSPF

- Supports **large enterprise networks**.
- Uses **cost** instead of only hop count.
- Provides **fast convergence**.
- Supports **hierarchical routing using Areas**.
- Prevents routing loops using the Link-State Algorithm.
- More **robust and reliable** than RIP.
- Supports **authentication** for secure routing.

Limitations of OSPF

- More complex to configure and manage than RIP.
- Requires more memory and CPU resources.
- Flooding Link-State Advertisements may consume additional bandwidth.
- Every router must execute **Dijkstra's Algorithm**, increasing computational overhead.

Border Gateway Protocol Version 4 (BGP4)

- Border Gateway Protocol version 4 (BGP4) is the routing protocol used for communication between different Autonomous Systems (ASs) on the Internet. It is based on the **path-vector routing algorithm**, in which routers exchange reachability information along with the sequence of Autonomous Systems (AS path).
- This enables routers to determine the best path to destination networks while avoiding routing loops. Unlike intradomain routing protocols such as RIP and OSPF, which operate within a single Autonomous System, BGP is used for routing between different Autonomous Systems.

Autonomous Systems in BGP

- In a network consisting of multiple Autonomous Systems, each AS uses an intradomain routing protocol such as RIP or OSPF to route packets within its own network.
- However, routers inside an AS initially know only the networks belonging to that AS and have no information about networks in other Autonomous Systems. BGP is used to exchange routing information between Autonomous Systems so that routers can reach networks located anywhere on the Internet.

External BGP (eBGP)

- External BGP (eBGP) operates between border routers that belong to different Autonomous Systems. Border routers establish a logical TCP connection called a **BGP session** using TCP port **179** and exchange update messages containing information about the networks they can reach.
- In the example shown in Figure 4.80, three eBGP sessions are established between router pairs **R1–R5**, **R2–R6**, and **R4–R9**.
- Through these sessions, each border router advertises the networks available within its own Autonomous System. After receiving these updates, neighboring border routers add the new routing information to their path tables.

Figure 4.79 *A sample internet with four ASs*

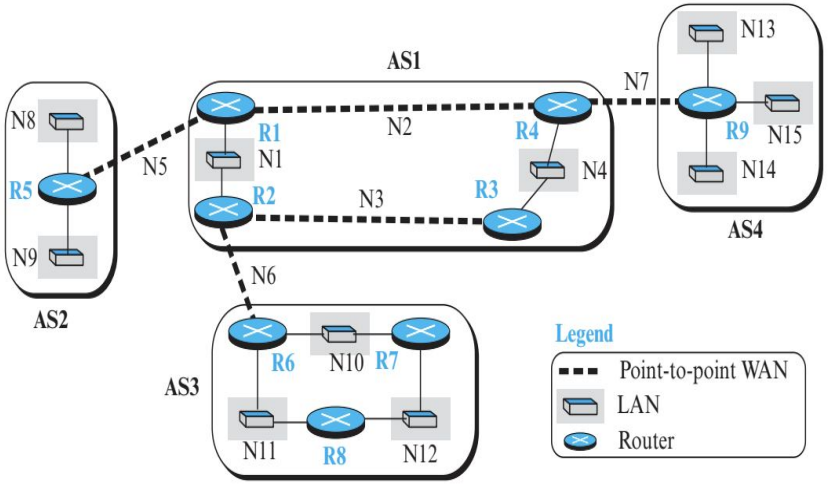
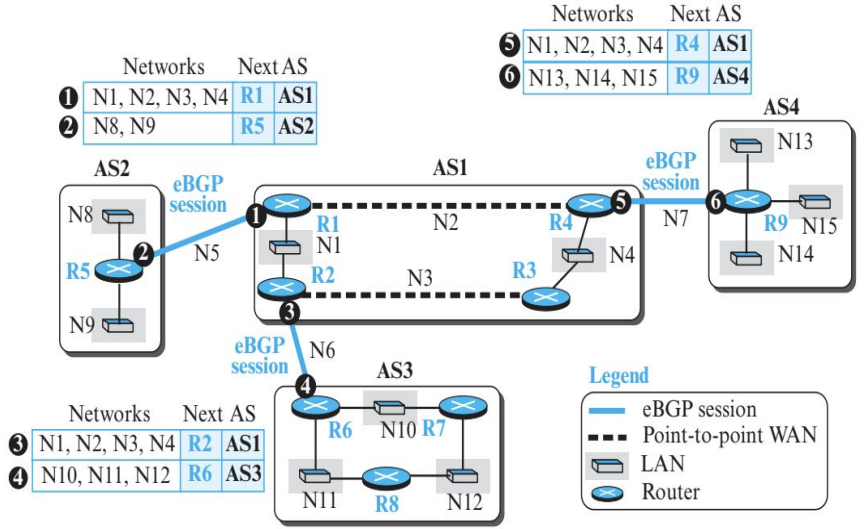


Figure 4.80 *eBGP operation*



Need for Internal BGP (iBGP)

The routing information exchanged through eBGP is available only to the border routers that participate in the sessions. Routers inside the same Autonomous System still do not know how to reach networks in other Autonomous Systems. To distribute this information throughout the AS, Internal BGP (iBGP) is used.

Internal BGP (iBGP)

Internal BGP (iBGP) operates between routers within the same Autonomous System. Every router exchanges routing information with the other routers in the AS through iBGP sessions, allowing all routers to learn the routes received from eBGP. This ensures that every router inside the Autonomous System can forward packets to networks located in other Autonomous Systems.

BGP Sessions

BGP communication takes place through logical TCP connections called **BGP sessions**, which use the well-known **TCP port 179**. Routers participating in a BGP session are known as **BGP peers** or **BGP speakers**. These peers continuously exchange update messages whenever routing information changes.

Figure 4.81 *Combination of eBGP and iBGP sessions in our internet*

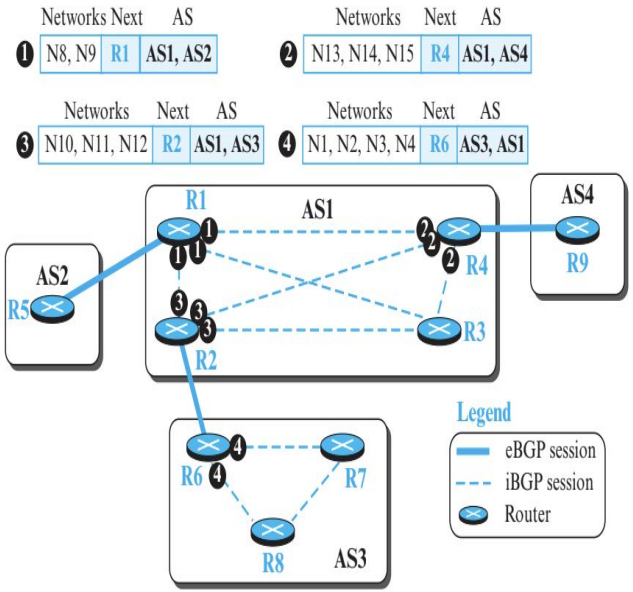


Figure 4.82 *Finalized BGP path tables*

Networks	Next	Path
N8, N9	R5	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R1

Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R6	AS1, AS3
N13, N14, N15	R1	AS1, AS4

Path table for R2

Networks	Next	Path
N8, N9	R2	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R3

Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R1	AS1, AS3
N13, N14, N15	R9	AS1, AS4

Path table for R4

Networks	Next	Path
N1, N2, N3, N4	R1	AS2, AS1
N10, N11, N12	R1	AS2, AS1, AS3
N13, N14, N15	R1	AS2, AS1, AS4

Path table for R5

Networks	Next	Path
N1, N2, N3, N4	R2	AS3, AS1
N8, N9	R2	AS3, AS1, AS2
N13, N14, N15	R2	AS3, AS1, AS4

Path table for R6

Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R7

Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R8

Networks	Next	Path
N1, N2, N3, N4	R4	AS4, AS1
N8, N9	R4	AS4, AS1, AS2
N10, N11, N12	R4	AS4, AS1, AS3

Path table for R9

BGP Update Messages

- A BGP update message contains information about the destination networks that can be reached through a particular router. Each update includes the destination networks, the next-hop router, and the Autonomous System path.
- As update messages are exchanged through eBGP and iBGP sessions, every router gradually learns the reachable networks in all Autonomous Systems.

Path Tables

- After receiving all BGP update messages, each router constructs a **path table**.
- The path table stores the destination networks, the next-hop router, and the AS path used to reach each destination.
- Using this information, the router selects the most appropriate path for forwarding packets.

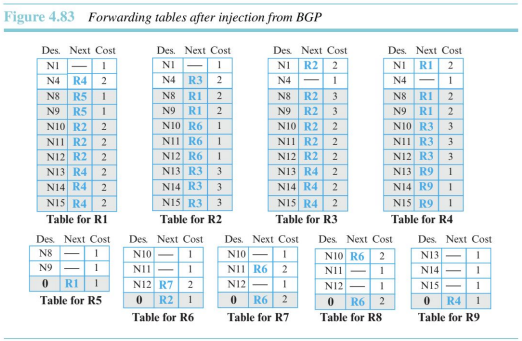


Figure 4.83 *Forwarding tables after injection from BGP*

Table for R1

Des.	Next	Cost
N1	—	1
N4	R4	2
N8	R5	1
N9	R5	1
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R2

Des.	Next	Cost
N1	—	1
N4	R3	2
N8	R1	2
N9	R1	2
N10	R6	1
N11	R6	1
N12	R6	1
N13	R3	3
N14	R3	3
N15	R3	3

Table for R3

Des.	Next	Cost
N1	R2	2
N4	—	1
N8	R2	3
N9	R2	3
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R4

Des.	Next	Cost
N1	R1	2
N4	—	1
N8	R1	2
N9	R1	2
N10	R3	3
N11	R3	3
N12	R3	3
N13	R9	1
N14	R9	1
N15	R9	1

Table for R5

Des.	Next	Cost
N8	—	1
N9	—	1
0	R1	1

Table for R6

Des.	Next	Cost
N10	—	1
N11	—	1
N12	R7	2
0	R2	1

Table for R7

Des.	Next	Cost
N10	—	1
N11	R6	2
N12	—	1
0	R6	2

Table for R8

Des.	Next	Cost
N10	R6	2
N11	—	1
N12	—	1
0	R6	2

Table for R9

Des.	Next	Cost
N13	—	1
N14	—	1
N15	—	1
0	R4	1

Injection into Intradomain Routing

- The routing information learned through BGP is incorporated into the forwarding tables of the intradomain routing protocol, such as RIP or OSPF.
- In stub Autonomous Systems, a default route is usually added to forward packets destined for external networks through the border router.
- In transient Autonomous Systems, multiple external routes are added to the forwarding table to support packet forwarding between different Autonomous Systems.

Address Aggregation

- BGP supports **address aggregation**, which combines multiple network prefixes into a single summarized prefix whenever they share the same routing path.
- Address aggregation reduces the size of routing tables, minimizes routing updates, and improves the efficiency of Internet routing.

Multicast Routing

- Communication over the Internet is not limited to one-to-one communication.
- Many modern applications require the same information to be delivered to multiple users simultaneously.
- Based on the number of destinations, Internet communication can be classified into **unicasting, multicasting, and broadcasting**. Multicast routing is designed to efficiently deliver data from one source to a group of interested receivers while minimizing bandwidth usage.

Unicasting

- In **unicasting**, one source sends data to one destination.
- The communication is **one-to-one**, where the source and destination are identified by unicast IP addresses.
- Each router forwards the packet through only one interface toward the destination, and only one copy of the packet exists throughout its journey. Common applications of unicasting include web browsing, file downloading, email communication, and remote login.

Multicasting

- In **multicasting**, one source sends data to a group of receivers. The communication is **one-to-many**, where the source uses a unicast address, but the destination is a multicast group address. Only the hosts that are members of the multicast group receive the data. Instead of sending separate copies from the source, multicast routers duplicate the packet only when necessary to reach different branches of the network. As a result, only one copy of the packet travels over each network link until duplication is required. This makes multicast communication highly efficient and reduces unnecessary network traffic.

Figure 4.87 *Unicasting*

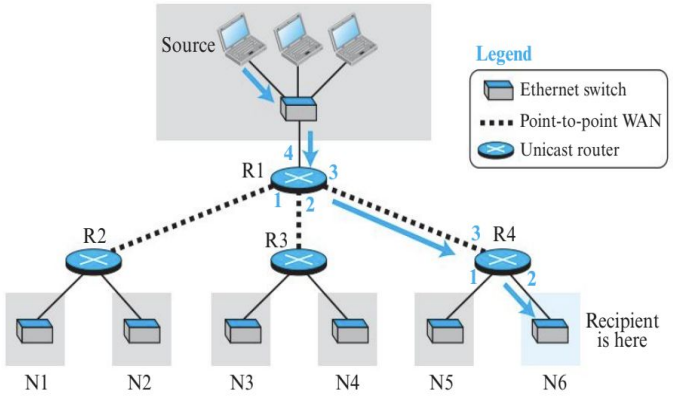


Figure 4.88 *Multicasting*

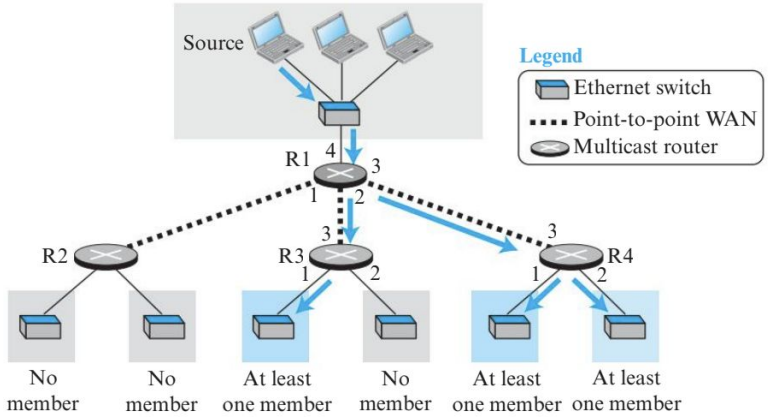
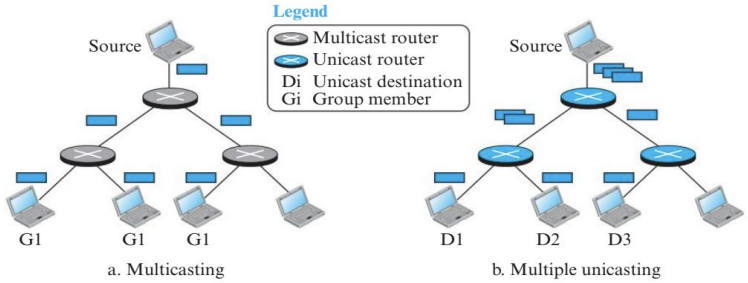


Figure 4.89 *Multicasting versus multiple unicasting*



Multicasting versus Multiple Unicasting

- Multicasting should not be confused with **multiple unicasting**. In multicasting, the source generates only one packet, and routers create additional copies whenever the transmission path branches toward different receivers. Therefore, only one copy of the packet travels between any two routers.
- In multiple unicasting, the source creates a separate packet for every destination. Each packet contains a different destination address, causing multiple copies of the same data to travel over the network. This increases bandwidth consumption and network traffic.
- Multicasting is therefore more efficient than multiple unicasting because it reduces duplicate transmissions and makes better use of available bandwidth. It also ensures that all receivers obtain the data almost simultaneously, whereas multiple unicasting may introduce delays when a large number of destinations are involved.

Applications of Multicasting

- Multicasting is widely used in applications where identical information must be delivered to multiple users at the same time.
- It is commonly used for **distributed databases**, where a query is sent to multiple database servers and the server containing the requested information responds.
- It is also used for **information dissemination**, such as software updates, news distribution, and financial data broadcasting. Other important applications include **teleconferencing**, where audio and video are delivered simultaneously to all participants, and **distance learning**, where online lectures are transmitted efficiently to a group of students.

Broadcasting

- **Broadcasting** is a **one-to-all** communication method in which a packet is sent to every host in a network.
- Every device within the broadcast domain receives the packet regardless of whether it needs the information.
- Because Internet-wide broadcasting would generate excessive traffic and consume a large amount of bandwidth, it is not supported across the Internet.
- However, controlled broadcasting is used within local networks and is also employed by some network applications during service discovery and communication setup.

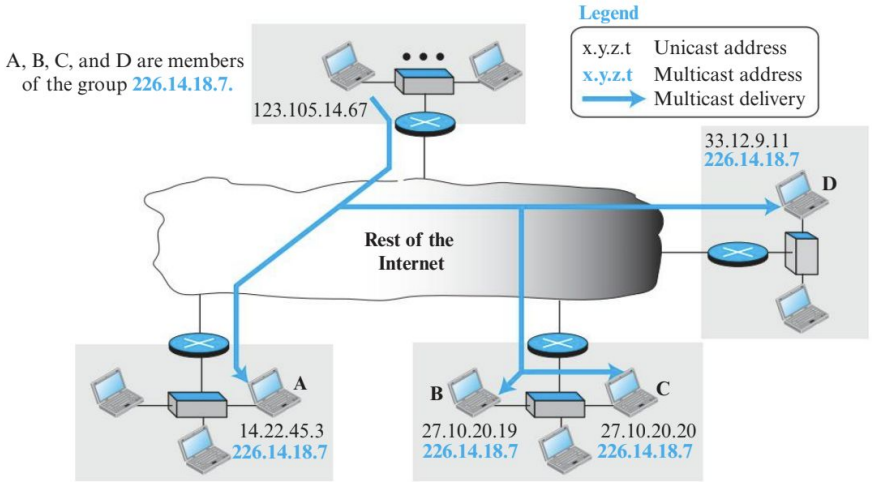
Multicasting Basics

- Before studying multicast routing protocols, it is important to understand some basic concepts of multicasting. These include **multicast addresses**, **multicast address selection**, **group membership**, and **multicast trees**, which enable efficient delivery of data from one sender to multiple receivers.

Multicast Addresses

- In unicast communication, a packet is sent from one source to one destination using the destination's unicast IP address. In multicast communication, however, a single sender transmits data to a group of receivers. Since a packet can contain only one destination address, it is not possible to include the addresses of all receivers in the packet. Therefore, multicast communication uses a **multicast address**, which represents an entire group of receivers instead of a single host.

Figure 4.90 *Needs for multicast addresses*



- The **source address** of a multicast packet is still a **unicast address**, identifying the sender, while the **destination address**is a **multicast group address**. Any host that joins the multicast group receives packets sent to that multicast address. A host may belong to multiple multicast groups. Therefore, in addition to its own unicast address, it can have several multicast addresses corresponding to the groups it has joined.

IPv4 Multicast Addresses

IPv4 reserves a special block of addresses for multicast communication. The multicast address range is **224.0.0.0 to 239.255.255.255**, represented by the prefix **224.0.0.0/4**. These addresses identify multicast groups rather than individual hosts.

The IPv4 multicast address space is divided into different blocks for specific purposes.

- **Local Network Control Block (224.0.0.0/24):** Used for communication within a local network. Packets in this range are **not forwarded by routers**. Common addresses include:
  - **224.0.0.1** – All hosts and routers on the local network.
  - **224.0.0.2** – All routers on the local network.
- **Internetwork Control Block (224.0.1.0/24):** Used for multicast communication across the Internet. Packets in this range **can be forwarded by routers**.
- **Source-Specific Multicast (SSM) Block (232.0.0.0/8):** Used for **Source-Specific Multicast**, where receivers receive data only from a specific sender.
- **GLOP Block (233.0.0.0/8):** Used within an **Autonomous System (AS)**. A range of multicast addresses is generated using the AS number.
- **Administratively Scoped Block (239.0.0.0/8):** Used within a specific organization or administrative domain. These multicast packets are restricted to that organization and are not forwarded outside it.

Selecting a Multicast Address

- The multicast address assigned to a group depends on the scope of communication.
- For a **limited group** within a single organization or Autonomous System, an address from the **Administratively Scoped Block** or the organization's allocated multicast range can be assigned. This allows communication only within that organization.
- For a **larger group** that spans multiple Autonomous Systems or the Internet, an address from the **Source-Specific Multicast (SSM) Block** is generally used. In SSM, routing depends on both the multicast group address and the source address, ensuring unique and efficient delivery of multicast packets.

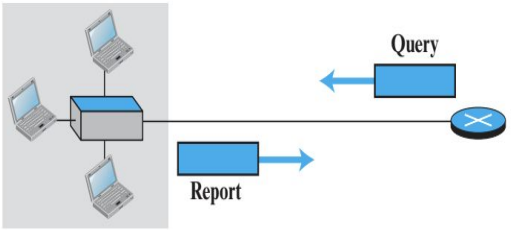
Collecting Information About Multicast Groups

- For multicast routing to work efficiently, routers must know **which hosts belong to each multicast group**. Whenever a host joins or leaves a multicast group, this information is communicated to the local multicast router using a **group management protocol** (such as **IGMP** in IPv4).
- Using this membership information, routers create multicast forwarding tables and forward multicast packets **only to networks that contain active group members**. This avoids unnecessary packet transmission and conserves network bandwidth.

Internet Group Management Protocol (IGMP)

- The **Internet Group Management Protocol (IGMP)** is used to manage **multicast group membership** in an IPv4 network. It enables hosts and multicast routers to exchange information about which multicast groups have active members on a local network. IGMP is a **network layer protocol** and is considered an auxiliary protocol of IP, similar to ICMP. IGMP messages are encapsulated within IP datagrams.
- IGMP allows a multicast router to determine whether there are hosts interested in receiving data for a particular multicast group. This information helps the router forward multicast packets only to networks where group members exist, reducing unnecessary network traffic.

Figure 4.93 *IGMP operation*



IGMP Messages

IGMP Version 3 uses two types of messages:

Query Message

A **Query** message is sent by the multicast router to check whether there are hosts interested in receiving data from a multicast group. The router periodically sends query messages to discover active multicast group members.

Report Message

A **Report** message is sent by a host in response to a query or whenever it joins a multicast group. The report informs the router that the host wants to receive multicast packets for a particular group.

If no host responds to a query within the specified time, the router assumes that there are no active members for that multicast group and removes the group from its multicast forwarding table.

Multicast Forwarding

- In multicast communication, routers forward packets differently from unicast communication.
- Instead of forwarding a packet through only one interface, a multicast router may forward copies of the same packet through **multiple interfaces so** that all members of the multicast group receive the data.
- Unlike unicast routing, multicast forwarding decisions are based on **both the source address and the destination (multicast group) address**.
- The destination address identifies the multicast group, while the source address helps the router determine the correct forwarding path and prevents duplicate packet transmission.
- When a multicast packet arrives at a router, the router forwards it only through interfaces that lead to networks containing multicast group members.
- The packet is **not forwarded back through the interface from which it was received**, as this would create duplicate packets and unnecessary network traffic.

Figure 4.94 *Destination in unicasting and multicasting*



a. Destination in unicasting is one

b. Destination in mulicasting is more than one

Figure 4.95 *Forwarding depends on the destination and the source*



a. Packet sent out of two interfaces

b. Packet sent out of one interface

Two Approaches to Multicasting

In multicast routing, routers must create **routing trees** to efficiently deliver packets from one source to multiple destinations. Unlike unicast routing, where forwarding depends only on the destination address, multicast routing depends on **both the source address and the multicast group address**. Based on the method used to build routing trees, multicast routing follows **two approaches**:

- **Source-Based Tree Approach**
- **Group-Shared Tree Approach**

Source-Based Tree Approach

In the **Source-Based Tree Approach**, a **separate routing tree is created for every source-group combination**. Each tree has the **source router as the root** and the routers containing multicast group members as the leaves.

If there are **m multicast groups** and **n sources**, each router must maintain **m × n routing trees**.

Although this approach requires more routing information and more memory, it provides the **shortest and most efficient path** from the source to every group member because each tree is optimized for a particular source.

Intradomain Multicast Routing Protocols

- Several multicast routing protocols are used **within an Autonomous System (AS)** to deliver multicast packets efficiently. These protocols differ in the way they create multicast routing trees and forward multicast packets. The commonly used intradomain multicast routing protocols are **Distance Vector Multicast Routing Protocol (DVMRP)**, **Multicast Open Shortest Path First (MOSPF)**, and **Protocol Independent Multicast (PIM)**.

Distance Vector Multicast Routing Protocol (DVMRP)

- **Distance Vector Multicast Routing Protocol (DVMRP)** is an extension of the **Routing Information Protocol (RIP)** for multicast routing. It follows the **Source-Based Tree** approach, where a separate multicast tree is created for each source-group combination.

DVMRP creates the multicast tree in three stages:

Reverse Path Forwarding (RPF)

- Reverse Path Forwarding (RPF) is used to prevent routing loops and duplicate multicast packets.
- When a router receives a multicast packet, it checks whether the packet arrived through the interface that lies on the shortest path back to the source. This information is obtained from the unicast forwarding table.
- If the packet arrives through the correct interface, it is accepted and forwarded; otherwise, it is discarded.

Reverse Path Broadcasting (RPB)

- After accepting the packet, DVMRP performs **Reverse Path Broadcasting (RPB)**. A broadcast tree is created with the **source as the root** and all networks as the leaves.
- To avoid duplicate packets reaching the same network, only one router is selected as the **parent router** for each network. The parent router alone forwards multicast packets to that network.

Reverse Path Multicasting (RPM)

- Reverse Path Broadcasting delivers packets to all networks, including those without multicast group members.
- To improve efficiency, **Reverse Path Multicasting (RPM)** removes unnecessary branches from the broadcast tree.
- Routers use **IGMP** to determine whether their attached networks contain active multicast group members.
- Branches leading to networks without members are **pruned**, resulting in an efficient multicast tree.

Interdomain Multicast Routing Protocols

The multicast routing protocols **DVMRP**, **MOSPF**, and **PIM** are designed for multicast communication **within a single Autonomous System (AS)**. When multicast group members are located in **different Autonomous Systems**, **interdomain multicast routing protocols** are required to exchange multicast routing information between ASs.

Multicast Border Gateway Protocol (MBGP)

Multicast Border Gateway Protocol (MBGP) is an extension of the **Border Gateway Protocol (BGP)** used for interdomain multicast routing. It exchanges multicast routing information between **border routers** of different Autonomous Systems.

Unlike BGP, which maintains a single routing path, MBGP maintains **two separate routing paths**:

- **Unicast path** – Used for unicast communication.
- **Multicast path** – Used for multicast communication.

MBGP follows the **Group-Shared Tree** approach. Each Autonomous System selects one router as the **Rendezvous Point (RP)**, which is responsible for receiving multicast packets and forwarding them to multicast group members within that AS.

Limitation of MBGP

- A major limitation of MBGP is that the **Rendezvous Point (RP)** in one Autonomous System may not know about multicast sources located in other Autonomous Systems. As a result, multicast packet delivery may become inefficient.**Multicast Source Discovery Protocol (MSDP)**
- The **Multicast Source Discovery Protocol (MSDP)** is used together with MBGP to overcome this limitation.
- In MSDP, each Autonomous System selects a **Source Representative Router**, which informs the Rendezvous Points (RPs) in other Autonomous Systems about the multicast sources available within its own AS.
- This allows all RPs to discover multicast sources across different Autonomous Systems and establish efficient multicast communication.

Border Gateway Multicast Protocol (BGMP)

Border Gateway Multicast Protocol (BGMP) is another interdomain multicast routing protocol proposed as a replacement for MBGP.

BGMP also uses the **Group-Shared Tree** approach. However, instead of having a separate Rendezvous Point in each Autonomous System, BGMP constructs **one shared multicast tree** for each multicast group.

The **root of the multicast tree** is located in one Autonomous System, while the branches extend to multicast group members in different Autonomous Systems.

Before multicast transmission begins, all multicast sources must know the location of the root and **tunnel** their multicast packets to it. The root then distributes the packets to all multicast group members.

Challenges of BGMP

- Selecting the Autonomous System that will act as the **root** of the multicast tree.
- Informing all multicast sources about the location of the root router.

Next Generation Internet Protocol (IPv6)

- The **Next Generation Internet Protocol (IPv6)** was developed to overcome the limitations of **IPv4**, especially the shortage of IP addresses. IPv6, also known as **Internet Protocol Version 6 (IPv6)** or **IP Next Generation (IPng)**, was introduced in the early 1990s.
- The most significant improvement in IPv6 is the use of a **128-bit address**, which provides a much larger address space than the **32-bit IPv4 address**. In addition to increasing the number of available addresses, IPv6 also introduces a simplified packet header, improved performance, better security, and support for modern Internet applications.
- IPv6 was designed not only to solve the address exhaustion problem but also to improve routing efficiency, support multimedia communication, and provide better scalability for future Internet growth.

Features of IPv6

Larger Address Space

IPv6 uses a **128-bit address**, compared to the **32-bit address** used in IPv4. This provides an extremely large number of unique IP addresses, allowing every Internet-connected device to have its own unique address.

Better Header Format

IPv6 has a **simplified header**. Optional fields are removed from the base header and placed in **extension headers**. This makes packet processing faster and improves routing efficiency.

New Options

IPv6 introduces **extension headers**, which provide additional functionalities whenever required without increasing the size of the basic header.

Extensibility

IPv6 is designed to support future technologies. New features can be added through extension headers without changing the basic protocol.

Resource Allocation

IPv6 replaces the IPv4 **Type of Service (ToS)** field with two new fields:

- **Traffic Class** – Identifies the priority of a packet.
- **Flow Label** – Identifies packets belonging to the same communication flow.

These fields help provide **Quality of Service (QoS)** for applications such as video conferencing, voice calls, and multimedia streaming.

Improved Security

IPv6 includes built-in support for **authentication** and **encryption**, ensuring data confidentiality, integrity, and secure communication.

Adoption of IPv6

Although IPv6 was developed to replace IPv4, its adoption has been gradual. The transition has been delayed because IPv4 has been extended using techniques such as:

- **Classless Addressing (CIDR)**
- **Dynamic Host Configuration Protocol (DHCP)**
- **Network Address Translation (NAT)**

These techniques temporarily reduced the shortage of IPv4 addresses. However, the rapid growth of the Internet, cloud computing, IoT devices, mobile networks, and multimedia applications is increasing the need for IPv6.

IPv6 Packet Format

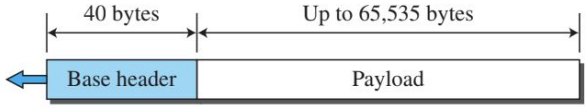
An **IPv6 packet** consists of **two main parts**:

- **Base Header**
- **Payload**

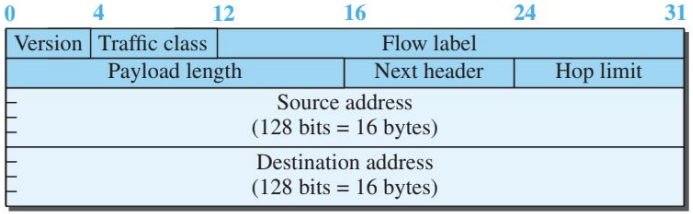
The **Base Header** has a **fixed size of 40 bytes**, while the **Payload** can be **up to 65,535 bytes**.

Unlike IPv4, the IPv6 header has a **fixed length**, making packet processing faster and more efficient.

Figure 4.101 *IPv6 datagram*



a. IPv6 packet



b. Base header

Pv6 Base Header Fields

Version

The **Version** field is **4 bits** long and identifies the IP version. For IPv6, its value is **6**.

Traffic Class

The **Traffic Class** field is **8 bits** long. It is used to identify packets with different priority levels and delivery requirements. It replaces the **Type of Service (ToS)** field used in IPv4 and helps provide **Quality of Service (QoS)**.

Flow Label

The **Flow Label** field is **20 bits** long. It identifies packets belonging to the same communication flow so that routers can provide special handling for applications such as video streaming, voice calls, and real-time communication.

Payload Length

The **Payload Length** field is **16 bits (2 bytes)** long. It specifies the size of the payload only and **does not include the 40-byte base header**.

Unlike IPv4, IPv6 does not require a separate header length field because the base header size is always fixed.

Next Header

The **Next Header** field is **8 bits** long. It identifies what follows the base header.

It may indicate:

- An **Extension Header**, or
- An upper-layer protocol such as **TCP**, **UDP**, or **ICMPv6**.

It performs a role similar to the **Protocol** field in IPv4.

Hop Limit

The **Hop Limit** field is **8 bits** long. It specifies the maximum number of routers (hops) through which a packet can travel. Each router decreases the value by one. When it becomes zero, the packet is discarded.

It performs the same function as the **TTL (Time To Live)** field in IPv4.

Source Address

The **Source Address** field is **128 bits (16 bytes)** long. It contains the IPv6 address of the sender.

Destination Address

The **Destination Address** field is **128 bits (16 bytes)** long. It contains the IPv6 address of the receiver.

IPv6 Payload

The **Payload** contains:

- **Zero or more Extension Headers**
- **Upper-layer protocol data** (TCP, UDP, ICMPv6, etc.)

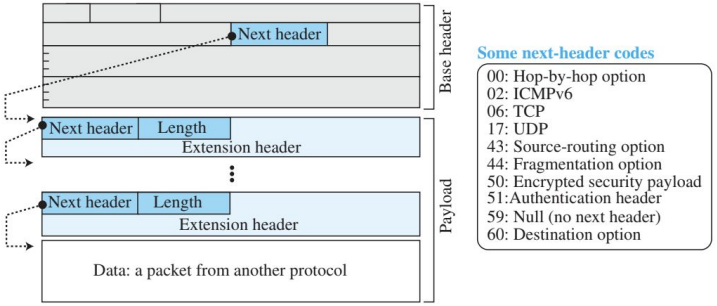
Unlike IPv4, optional information is **not stored in the base header**. Instead, IPv6 stores optional information in **Extension Headers**, making the base header simpler and improving routing speed.

Each Extension Header contains:

- **Next Header** – Specifies the type of the next extension header or the upper-layer protocol.
- **Header Length** – Indicates the size of the extension header.
- **Option-specific Information** – Contains information related to that extension.

The final **Next Header** field identifies the transport layer protocol such as **TCP**, **UDP**, or **ICMPv6**.

Figure 4.102 *Payload in an IPv6 datagram*



Concept of Flow and Priority in IPv6

- IPv4 was designed as a **connectionless protocol**, where each packet is routed independently. IPv6 introduces the **Flow Label** field, allowing packets belonging to the same communication session to be identified and handled together. This enables IPv6 to support **connection-oriented communication** and improve the performance of real-time applications.
- A **flow** is a sequence of packets that share the same characteristics, such as following the same path, requiring the same Quality of Service (QoS), using the same security settings, or belonging to the same application.
- Routers that support flow labels maintain a **Flow Label Table**. When a packet arrives, the router checks the **Flow Label**in the packet and looks up the corresponding entry in the table. Based on this entry, the router forwards the packet using the required services without performing the complete routing process every time. This speeds up packet forwarding.
- The **Flow Label** can also support **real-time applications** such as voice calls, video conferencing, and multimedia streaming. These applications require guaranteed resources like higher bandwidth, larger buffers, and lower delay. Resource reservation for such applications is achieved with the help of protocols such as **RTP (Real-Time Transport Protocol)** and **RSVP (Resource Reservation Protocol)**.

IPv6 ADDRESSING – KEY CONCEPTS

1. IPv6 ADDRESS STRUCTURE

128 bits (16 bytes)

16 bits	16 bits	16 bits	16 bits	16 bits	16 bits	16 bits	16 bits

An IPv6 address has 128 bits, written as 8 groups of 4 hexadecimal digits separated by colons.

2. IPv6 ADDRESS NOTATIONS

Binary (in computer)	0010 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 1111 0000 0001
Colon Hex (full)	2001:0DB8:0000:0000:0000:FF00:0042:8329
Abbreviation (omit leading zeros)	2001:DB8:0:0:0:FF00:42:8329
Zero Compression	2001:DB8::FF00:42:8329 (only one "::" allowed)
Mixed Notation (IPv4 embedded)	::130.24.24.18 (IPv4 in last 32 bits)

3. ADDRESS TYPES

Unicast
Address of a single interface.



Packet goes to one specific host.

Anycast
Address of a group. Packet goes to the most reachable member.



Packet delivered to one member only.

Multicast
Address of a group. Packet is delivered to all members.



Packet delivered to all members.

IPv6 does not use broadcasting. It treats broadcasting as a special case of multicasting.

4. ADDRESS SPACE SIZE



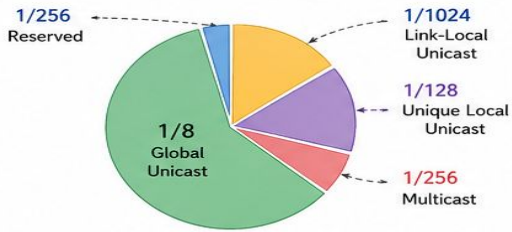
- Total IPv6 address space = 2^{128} (= 3.40×10^{38} addresses)
- If only $\frac{1}{64}$ of the space is for people on Earth,
- and population ≈ 2^{34} (≈ 16 billion)

⇒ Each person can have 2^{88} addresses!

Address depletion in IPv6 is impossible.

5. IPv6 ADDRESS SPACE ALLOCATION

Block (Prefix)	Purpose	Binary Start	First Address (Hex)	Size	Fraction of Address Space
0000::/8	Reserved	0000 0000	0000::	2^{120}	1/256
2000::/3	Global Unicast	001	2000::	2^{125}	1/8
FE80::/10	Link-Local Unicast	1111 1110 10	FE80::	2^{118}	1/1024
FC00::/7	Unique Local Unicast	1111 110	FC00::	2^{121}	1/128
FF00::/8	Multicast	1111 1111	FF00::	2^{120}	1/256



(Most of the address space is for Global Unicast)

IPv6 ADDRESSING – COMPLETE OVERVIEW

1. IPv6 ADDRESS OVERVIEW

IPv6 address length = 128 bits (16 bytes)

128 bits

16 bits	16 bits	16 bits	16 bits	16 bits	16 bits	16 bits	16 bits

- Four times larger than IPv4 (32 bits)
- Solves address exhaustion problem
- Supports massive growth of Internet and future technologies

2. IPv6 ADDRESS NOTATIONS

Binary (in computer)	0010 0000 0000 0001 0000 0DB8 0000 0000 0000 0000 0000 0000 0000 0042 8329
Colon Hex (full)	2001:0DB8:0000:0000:0000:FF00:0042:8329
Abbreviation (omit leading zeros)	2001:DB8:0:0:0:FF00:42:8329
Zero Compression (only one "::")	2001:DB8::FF00:42:8329
Mixed Notation (IPv4 embedded)	::130.24.24.18 (IPv4 in last 32 bits)

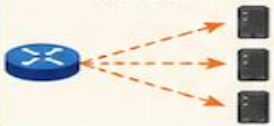
Rules:
- Leading zeros in a section can be omitted.
- Consecutive zero sections can be replaced with "::" (only once).

3. TYPES OF IPv6 ADDRESSES

Unicast
(One-to-One)
Address of a single interface.
Packet is delivered to that one interface.



Anycast
(One-to-Nearest)
Address of a group. Packet is delivered to the most reachable member.



Multicast
(One-to-Many)
Address of a group. Packet is delivered to all members.



IPv6 does not use broadcasting.
Broadcasting is treated as a special case of multicasting.

4. IPv6 ADDRESS SPACE SIZE



- Total IPv6 address space = 2^{128} (= 3.40×10^{38} addresses)
- If only $\frac{1}{64}$ of the space is for people on Earth,
- and population ≈ 2^{34} (≈ 16 billion)

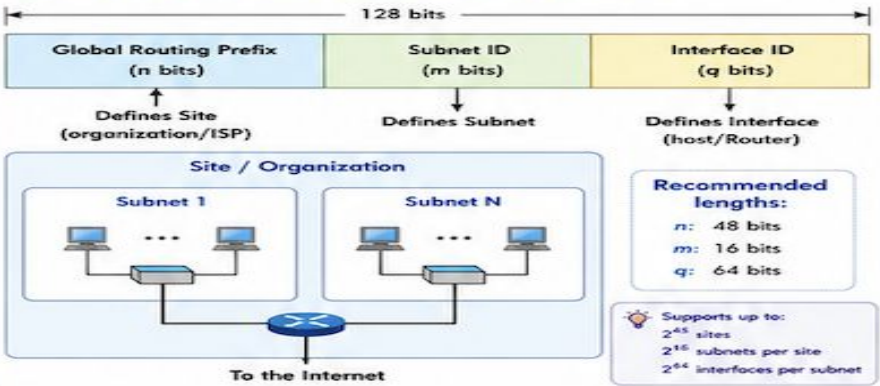
  ⇒ Each person can have 2^{88} addresses!

Address depletion in IPv6 is impossible!

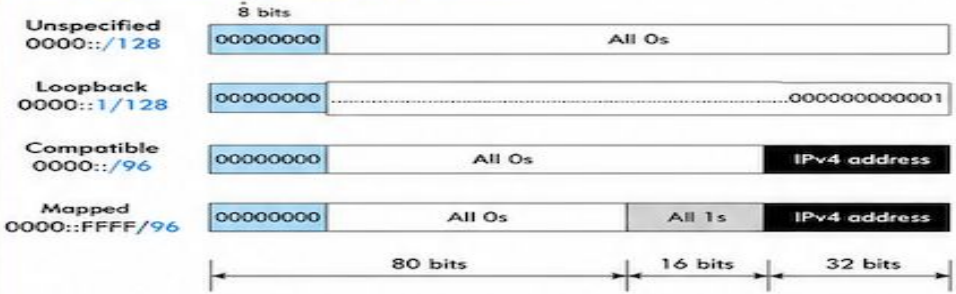
5. IPv6 ADDRESS SPACE ALLOCATION

Block Prefix	CIDR	Block Assignment	Size	Fraction of Address Space
0000::/8	0000::/8	Special addresses	2^{120}	1/256
001	2000::/3	Global unicast	2^{125}	1/8
1111 110	FC00::/7	Unique local unicast	2^{121}	1/128
1111 1110 10	FE80::/10	Link local unicast	2^{118}	1/1024
1111 1111	FF00::/8	Multicast addresses	2^{120}	1/256

6. GLOBAL UNICAST ADDRESS (2000::/3)



7. SPECIAL ADDRESSES (0000::/8)



- **Compatible:** Used when communicating with IPv6 host.
- **Mapped:** Used to communicate with IPv4 host.

8. OTHER ASSIGNED BLOCKS

A. UNIQUE LOCAL UNICAST (FC00::/7)



- Used for private addressing within an organization.
- Not routed on the public Internet.
- 40-bit random number ⇒ very low chance of duplication.

L = Local/Global (0 or 1)

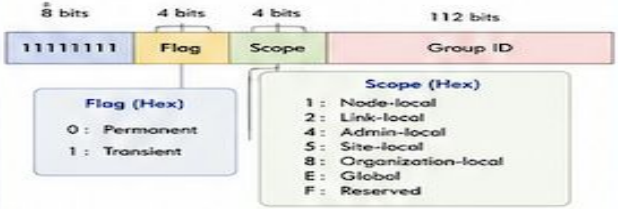
B. LINK LOCAL UNICAST (FE80::/10)



- Used for communication within a link (local network).
- Not routed by routers.
- Interface ID can be EUI-64 or other 64-bit value.

Typically used for:
- Neighbor Discovery (ND)
- Router discovery
- Initial communication

C. MULTICAST ADDRESS (FF00::/8)



- Used to send packets to a group of hosts.
- All members of the group receive a copy.

ICMPv6 (Internet Control Message Protocol Version 6)

- ICMPv6 is the control and error-reporting protocol used in IPv6. It performs the same basic functions as ICMP in IPv4, such as reporting network errors and providing diagnostic information, but it is more powerful and includes additional responsibilities.
- In IPv6, some protocols that existed separately in IPv4, such as **ARP (Address Resolution Protocol)** and **IGMP (Internet Group Management Protocol)**, have been integrated into ICMPv6.
- This makes ICMPv6 a central protocol for error reporting, diagnostics, neighbor discovery, and multicast group management.

Types of ICMPv6 Messages

ICMPv6 messages are divided into **four main categories**.

1. Error Reporting Messages

These messages inform the sender whenever a problem occurs during packet delivery. They perform functions similar to ICMPv4, but with one important addition: since routers do not fragment packets in IPv6, a new **Packet Too Big** message is used.

Figure 4.109 *Comparison of network layer in version 4 and version 6*

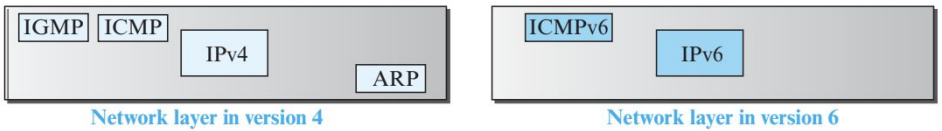
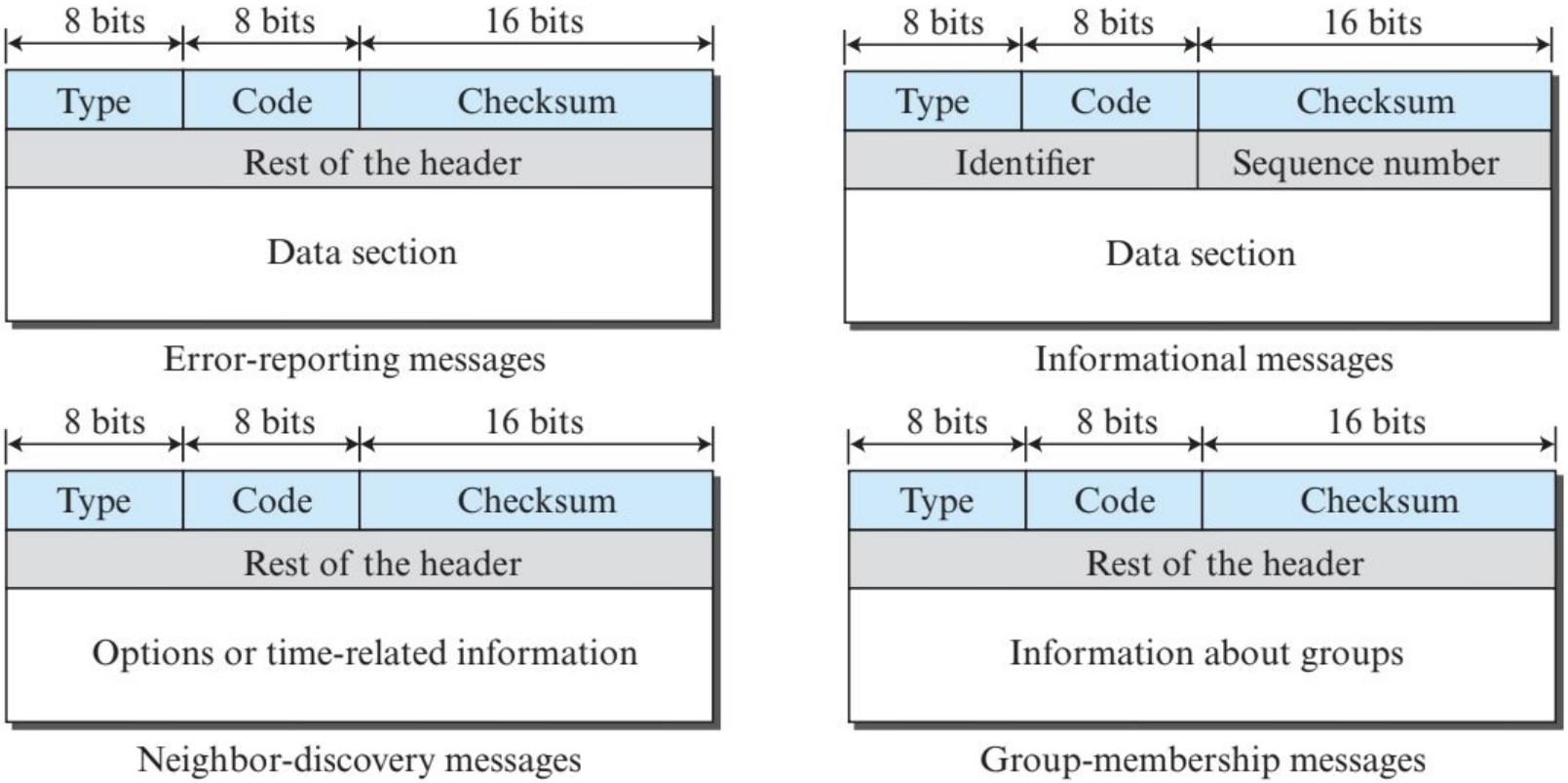


Figure 4.110 *ICMPv6 messages*



Error Messages

- **Destination Unreachable** – Destination cannot be reached.
- **Packet Too Big** – Packet size exceeds the network MTU.
- **Time Exceeded** – Hop limit becomes zero before reaching the destination.
- **Parameter Problem** – Invalid field found in the IPv6 header.

Changes from IPv4

- **Source Quench** message has been removed.
- **Packet Too Big** message has been added.
- **Redirect** message has been moved to Neighbor Discovery.

2. Informational Messages

These messages are mainly used for testing connectivity between two devices.

- **Echo Request** – Sent to check whether a destination is reachable.
- **Echo Reply** – Returned by the destination as a response.

These messages are used by the **ping** command.

Changes from IPv4

- Timestamp Request and Timestamp Reply messages have been removed because they were rarely used.

. Neighbor Discovery Messages (NDP)

Neighbor Discovery Protocol (NDP) is built into ICMPv6. It replaces the functions performed by ARP and several router discovery mechanisms in IPv4.

Neighbor Discovery helps devices:

- Discover default routers.
- Find the link-layer (MAC) address of neighboring devices.
- Determine the IPv6 address of neighboring devices.
- Redirect packets to a better router.

Neighbor Discovery Messages

Message	Purpose
Router Solicitation (RS)	Host requests nearby router information
Router Advertisement (RA)	Router advertises its presence and network information
Neighbor Solicitation (NS)	Finds the MAC address of a neighbor
Neighbor Advertisement (NA)	Replies with the MAC address
Redirect	Suggests a better router/path
Inverse Neighbor Solicitation	Finds IPv6 address using MAC address
Inverse Neighbor Advertisement	Responds with IPv6 address

Neighbor Discovery Messages

Message	Purpose
Router Solicitation (RS)	Host requests nearby router information
Router Advertisement (RA)	Router advertises its presence and network information
Neighbor Solicitation (NS)	Finds the MAC address of a neighbor
Neighbor Advertisement (NA)	Replies with the MAC address
Redirect	Suggests a better router/path
Inverse Neighbor Solicitation	Finds IPv6 address using MAC address
Inverse Neighbor Advertisement	Responds with IPv6 address

4. Group Membership Messages

These messages are used in IPv6 multicast communication. They perform the same function as IGMP in IPv4.

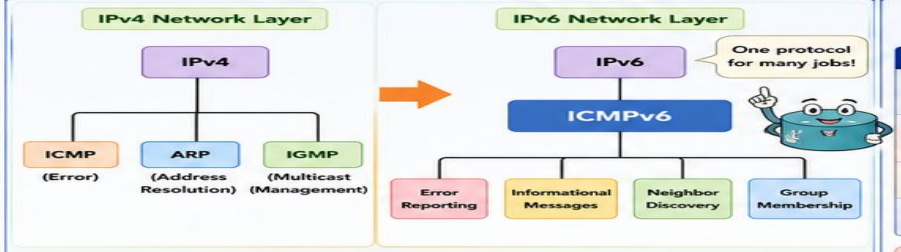
Messages

- **Membership Query** – Router asks which multicast groups are active.
- **Membership Report** – Hosts inform the router about the multicast groups they have joined.

ICMPv6 (Internet Control Message Protocol Version 6)

ICMPv6 is the control and error-reporting protocol used in IPv6.
In IPv6, ICMPv6 combines the functions of ICMPv4, ARP and IGMP.

1. ICMPv6 IN IPV6 NETWORK LAYER



ICMPv6 = ICMP + ARP + IGMP + New Messages

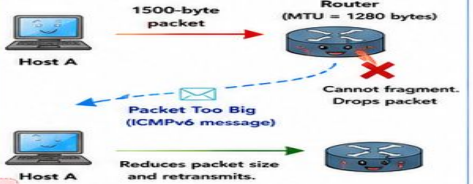
2. ERROR REPORTING MESSAGES

ICMPv6 informs the sender when a problem occurs during packet delivery.

Message	Purpose
Destination Unreachable	Destination cannot be reached.
Packet Too Big	Packet size > MTU. Router cannot fragment.
Time Exceeded	Hop limit becomes zero.
Parameter Problem	Invalid field in the IPv6 header.

- Source Quench message – removed (rarely used)
- Packet Too Big message – added (new in IPv6)
- Redirect message – moved to Neighbor Discovery

Packet Too Big Example



3. INFORMATIONAL MESSAGES

Used to test connectivity between two devices.



- Used by the ping command.
- Timestamp Request and Reply – removed (rarely used).

4. NEIGHBOR DISCOVERY MESSAGES (NDP)

Helps devices discover routers, other hosts and resolve addresses.

Router Discovery

Find default router and network info.



Replaces DHCP discovery in IPv4.

Address Resolution

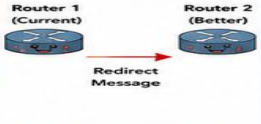
Find MAC address using IPv6 address.



Replaces ARP in IPv4.

Redirection

Suggests a better router to send packets.



Helps improve routing efficiency.

Inverse Discovery

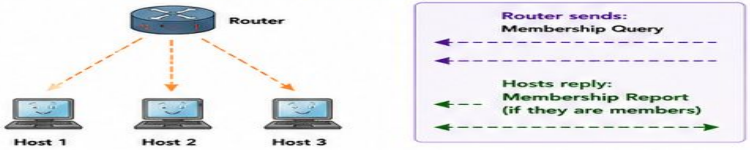
Find IPv6 address using MAC address.



Finds IPv6 address from MAC address.

5. GROUP MEMBERSHIP MESSAGES

Used in IPv6 multicast communication (replaces IGMP).



Router knows which hosts belong to the multicast group.

6. ICMPv4 vs ICMPv6 COMPARISON

Feature	ICMPv4	ICMPv6
Error Reporting	✓	✓
Ping (Echo Req/Reply)	✓	✓
Address Resolution	ARP (separate)	Neighbor Discovery (NDP)
Router Discovery	Separate methods (DHCP, etc.)	Neighbor Discovery (NDP)
Multicast Group Management	IGMP (separate)	ICMPv6 Group Membership
Packet Too Big Message	✗	✓
Router Fragmentation	Allowed	Not Allowed

Key Takeaways

- ICMPv6 is a powerful control protocol in IPv6.
- Combines ICMP, ARP and IGMP functions.
- Adds new messages for better network discovery and control.
- Essential for error reporting, diagnostics and multicast.

Quality of Service (QoS)

- The Internet was originally designed to provide **best-effort service**, where the network attempts to deliver data without guaranteeing performance such as delay, bandwidth, or reliability.
- This approach is suitable for applications like **file transfer** and **e-mail**, which are not sensitive to transmission delays.
- Such applications are called **elastic applications** because they can adapt to changes in network conditions and available bandwidth.
- They are also referred to as **Available Bit Rate (ABR)** applications since they can speed up or slow down depending on the network capacity.
- However, modern multimedia applications such as **voice calls, video conferencing, and online streaming** generate **real-time traffic**, which is highly sensitive to delay and requires predictable network performance. These applications cannot tolerate excessive delays or variations in packet arrival times.
- To support such applications, **Quality of Service (QoS)** mechanisms are used. QoS is a collection of networking techniques that ensure the network provides **predictable and guaranteed performance** in terms of delay, reliability, bandwidth, and jitter, enabling applications to function efficiently.

Data-Flow Characteristics

To provide Quality of Service (QoS), the network must understand the requirements of different applications. Every data flow is characterized by four important parameters: **Reliability, Delay, Jitter, and Bandwidth**. The importance of each characteristic depends on the type of application.

1. Reliability

- **Reliability** refers to the ability of the network to deliver all packets correctly to the destination without loss or corruption. If packets or acknowledgments are lost, they must be retransmitted, increasing communication time.
- Applications such as **e-mail, file transfer, and web browsing** require high reliability because losing data is unacceptable.
- On the other hand, applications like **voice calls and video conferencing** can tolerate occasional packet loss, making reliability less critical for real-time communication.

2. Delay

- **Delay** is the time taken for a packet to travel from the source to the destination. Different applications have different delay requirements.
- **Real-time applications** such as **telephony, audio conferencing, video conferencing, and remote login** require very low delay to provide smooth communication.
- In contrast, applications such as **e-mail and file transfer** can tolerate higher delays because immediate delivery is not essential.

3. Jitter

- **Jitter** is the variation in the delay experienced by packets belonging to the same data flow.
- If all packets experience the same delay, communication remains smooth even if the delay is relatively large.
- However, if packet delays vary significantly, audio and video playback become distorted, resulting in poor quality.
- Therefore, applications such as **voice and video conferencing** require **low jitter** to ensure continuous and synchronized communication.

4. Bandwidth

- **Bandwidth** is the amount of data that can be transmitted over the network in one second, usually measured in **bits per second (bps)**.
- Different applications require different bandwidths.
- **Video conferencing, online streaming, and high-definition multimedia** require very high bandwidth because they transmit large amounts of data continuously.
- In contrast, **e-mail and text messaging** require very little bandwidth since the amount of transmitted data is comparatively small.

Application	*Reliability*	*Delay*	*Jitter*	*Bandwidth*
FTP	High	Low	Low	Medium
HTTP	High	Medium	Low	Medium
Audio-on-demand	Low	Low	High	Medium
Video-on-demand	Low	Low	High	High
Voice over IP	Low	High	High	Low
Video over IP	Low	High	High	High

Flow Classes

- Based on the required **Quality of Service (QoS)** characteristics, network data flows can be grouped into different **flow classes**. Each class has different requirements for **reliability, delay, jitter, and bandwidth**. Although the Internet does not have a formal classification of flow classes, different applications have different QoS needs. For example, the **File Transfer Protocol (FTP)** requires **high reliability** and **moderate bandwidth**, while delay and jitter are not significant concerns because file transfers can tolerate waiting.

Unlike the Internet, **Asynchronous Transfer Mode (ATM)** defines **five standard flow classes**, each designed for specific application requirements.

1. Constant Bit Rate (CBR)

- **Constant Bit Rate (CBR)** is used for applications that require a **fixed and continuous data transmission rate**, similar to traditional circuit switching. These applications are highly sensitive to **delay and jitter**, making consistent bandwidth essential. CBR is commonly used for **telephone communication, video conferencing, and television broadcasting**, where uninterrupted and smooth data delivery is required.

2. Variable Bit Rate – Non-Real Time (VBR-NRT)

- **Variable Bit Rate – Non-Real Time (VBR-NRT)** is designed for applications whose transmission rate changes depending on the amount of available data. These applications are **not sensitive to delay or jitter**, allowing data to be transmitted whenever it is available. A typical example is **multimedia e-mail**, where timely delivery is less critical than successful delivery.

3. Variable Bit Rate – Real Time (VBR-RT)

- Variable Bit Rate – Real Time (VBR-RT) also allows the transmission rate to vary, but it is intended for real-time applications that are sensitive to delay and jitter. It is mainly used for applications such as interactive compressed video and live multimedia communication, where smooth playback is more important than maintaining a constant transmission rate.

4. Available Bit Rate (ABR)

- Available Bit Rate (ABR) provides rate-based flow control, where the transmission speed is adjusted according to the available network bandwidth. It is suitable for applications such as file transfer, e-mail, and general data communication, which can adapt to changing network conditions and do not require guaranteed bandwidth.

5. Unspecified Bit Rate (UBR)

- Unspecified Bit Rate (UBR) is a best-effort service that does not guarantee bandwidth, delay, or reliability. Data is transmitted whenever network resources are available. UBR is widely used for TCP/IP traffic, including applications such as web browsing and other Internet services that can tolerate variable network performance.

Flow Control to Improve QoS

- Although the Internet does not formally define flow classes, the **Type of Service (ToS)** field in the IP datagram header can be used to indicate the level of service required by different applications. Based on this information, routers can apply different mechanisms to improve the **Quality of Service (QoS)** by managing packet transmission efficiently. One of the most important QoS mechanisms is **packet scheduling**, which determines the order in which packets are processed and transmitted.

Scheduling

- **Scheduling** is the process of deciding **which packet should be transmitted first** when multiple packets are waiting in a router's queue. Since packets may experience **delay, jitter, packet loss, or limited bandwidth** at routers, an efficient scheduling algorithm is essential for providing better QoS. A good scheduling technique ensures that packets from different applications are treated fairly according to their service requirements.

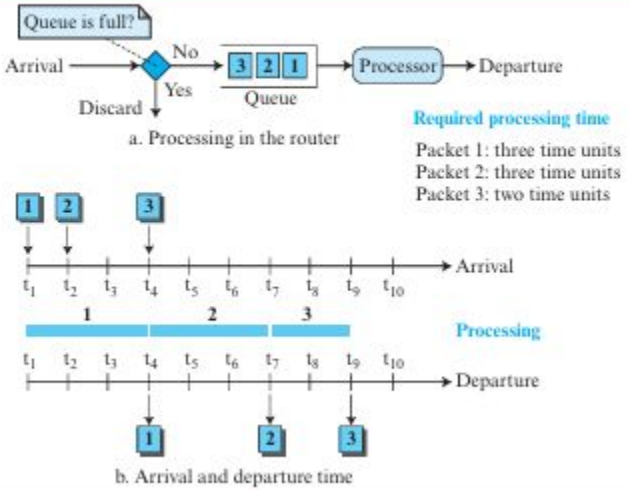
Some commonly used scheduling techniques are:

- **FIFO (First-In, First-Out) Queuing**
- **Priority Queuing**
- **Weighted Fair Queuing (WFQ)**

FIFO (First-In, First-Out) Queuing

- **FIFO Queuing** is the simplest and most commonly used scheduling technique in the Internet.
- In this method, packets are placed in a **single queue** and are processed in the **same order in which they arrive**.
- The packet that enters the queue first is transmitted first, regardless of its application or priority.
- If the **arrival rate of packets becomes greater than the processing rate**, the queue gradually fills up. Once the queue becomes full, **newly arriving packets are discarded**, resulting in packet loss.
- FIFO guarantees only one thing: **packets leave the queue in the same order they entered**.

Figure 8.59 *FIFO queue*



Working of FIFO Queuing

1. Packets from different applications arrive at the router.
2. All packets are placed in a **single queue**.
3. The router processes packets one by one in the order of arrival.
4. Large packets require more processing time than smaller packets.
5. If the queue becomes full, new packets are dropped.

Limitations of FIFO Queuing

FIFO **does not distinguish between different types of network traffic**. All packets are treated equally, whether they belong to:

- FTP (File Transfer Protocol)
- Voice over IP (VoIP)
- Video Conferencing
- E-mail
- Web Browsing

As a result:

- **Real-time applications** such as **VoIP and video conferencing** may suffer from **high delay and jitter**.
- **No priority** is given to delay-sensitive traffic.
- The **bandwidth allocated** to each application depends only on the number of packets arriving at the router.
- FIFO cannot provide **service differentiation**, making it unsuitable for applications that require guaranteed QoS.

Advantages of FIFO Queuing

- Very simple to implement.
- Requires low processing overhead.
- Ensures packets are transmitted in the order they arrive.
- Suitable for best-effort networks.

Disadvantages of FIFO Queuing

- No priority for important or real-time traffic.
- High delay and jitter under heavy network load.
- Packet loss occurs when the queue is full.
- Cannot guarantee Quality of Service (QoS).
- Treats all applications equally, regardless of their requirements.

Priority Queuing (PQ)

Definition

Priority Queuing (PQ) is a scheduling technique in which packets are classified into different **priority levels**, and each priority level has its own queue. The router always serves the **highest-priority queue first**. Lower-priority queues are served only after all higher-priority queues become empty.

Working

1. Incoming packets are assigned to priority classes (High, Medium, Low).
2. Each class has a separate queue.
3. The scheduler always checks the highest-priority queue first.
4. It transmits **all packets** from the highest-priority queue before moving to the next lower-priority queue.

Example

Suppose there are two queues:

- **High Priority Queue:** Video call packets
- **Low Priority Queue:** Email packets

Transmission order:

High Queue : V1 V2 V3
Low Queue : E1 E2 E3

Output:
V1 → V2 → V3 → E1 → E2 → E3

Even if **E1** arrived before **V3**, **V3** is transmitted first because it has higher priority.

Advantages

- Very low delay for high-priority traffic.
- Suitable for real-time applications such as:
    - Voice over IP (VoIP)
    - Video conferencing
    - Live streaming

Disadvantage (Starvation)

If high-priority packets keep arriving continuously, lower-priority packets may never get transmitted.

This problem is called **Starvation**.

Example:

High Queue : H1 H2 H3 H4 H5 ...
Low Queue : L1 L2

Output:
H1 H2 H3 H4 H5 ...

L1 and L2 keep waiting.

Weighted Fair Queuing (WFQ)

- **Weighted Fair Queuing (WFQ)** is an improved scheduling technique that gives **every queue a chance to transmit**, but **higher-priority queues receive a larger share of bandwidth**.

Instead of serving one queue completely, the scheduler visits each queue in a **round-robin** manner according to its assigned **weight**.

Working

1. Packets are classified into different queues.
2. Each queue is assigned a weight.
3. During each scheduling cycle, packets are transmitted according to these weights.

Example

Weights:

- Queue 1 = **3**
- Queue 2 = **2**
- Queue 3 = **1**

Scheduling order:

Queue 1 : A1 A2 A3
Queue 2 : B1 B2
Queue 3 : C1

Output:
A1 A2 A3 → B1 B2 → C1
Repeat...

Every queue gets service, but higher-weight queues transmit more packets.

Bandwidth Allocation Example

If router throughput is R, then:

Queue	Weight	Bandwidth Share
High	3	R/2
Medium	2	R/3
Low	1	R/6

(Approximate allocation when packet sizes are equal.)

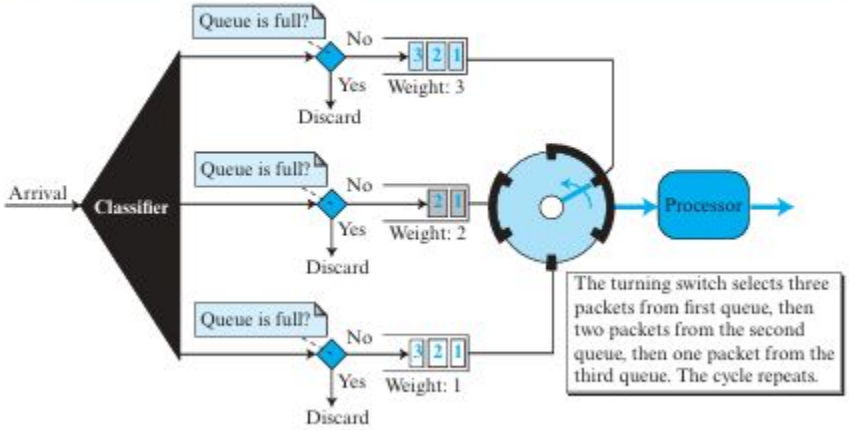
Advantages

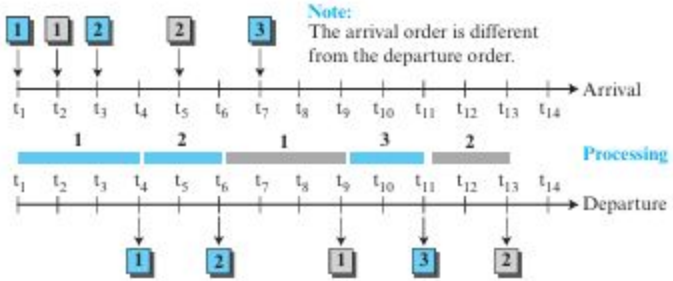
- Prevents starvation.
- Provides fairness among all traffic classes.
- High-priority traffic still receives more bandwidth.
- Suitable for mixed traffic (voice, video, web browsing, email).

Limitation

- If packet sizes differ significantly, the bandwidth allocation may not be perfectly fair because larger packets consume more transmission time

Figure 8.61 *Weighted fair queuing*





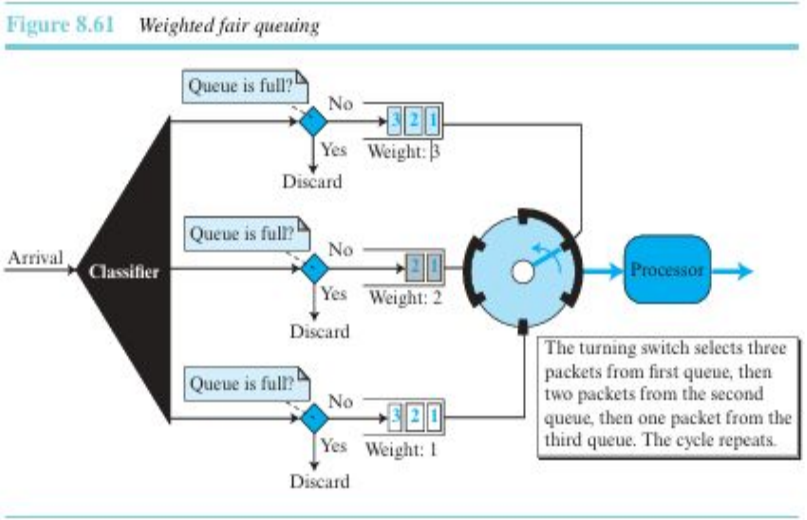
b. Arrival and departure time

Difference Between Priority Queuing and Weighted Fair Queuing

Feature	Priority Queuing (PQ)	Weighted Fair Queuing (WFQ)
Service order	Highest-priority queue first	All queues served in weighted round-robin
Fairness	Not fair	Fair
Starvation	Possible	Prevented
Delay for high-priority traffic	Very low	Low
Lower-priority traffic	May never be served	Always gets some service
Scheduling	Strict priority	Weight-based round robin
Best suited for	Pure real-time traffic	Mixed network traffic

Traffic Shaping and Traffic Policing

- Traffic shaping and traffic policing are techniques used to control the amount and rate of network traffic, thereby improving Quality of Service (QoS) and preventing network congestion.
- **Traffic shaping** is applied when traffic leaves a network to regulate the transmission rate, whereas **traffic policing** is applied when traffic enters a network to ensure that incoming traffic conforms to the agreed bandwidth limits.
- The two commonly used mechanisms for implementing traffic shaping and policing are the **Leaky Bucket** and **Token Bucket** algorithms.

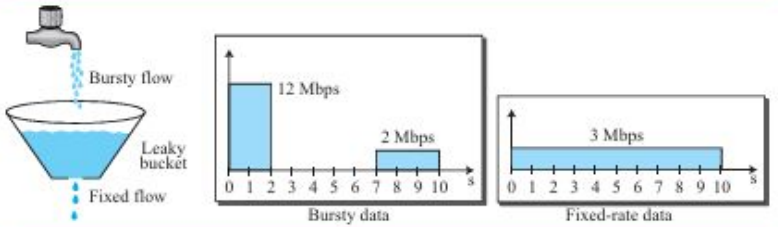


Leaky Bucket Algorithm

- The **Leaky Bucket algorithm** smooths bursty traffic by converting it into a constant-rate output.
- It is based on the analogy of a bucket with a small hole at the bottom. Water can be poured into the bucket at any rate, but it leaks out at a constant rate as long as water is present.
- If water enters faster than it can leak out, the bucket eventually becomes full and any additional water overflows. Similarly, in a computer network, incoming packets are temporarily stored in a **FIFO (First-In, First-Out) queue**, and packets are transmitted at a **fixed, predetermined rate**, regardless of how bursty the incoming traffic is. If the queue becomes full, newly arriving packets are **discarded**, thereby preventing excessive traffic from entering the network.
- The Leaky Bucket algorithm is particularly useful for enforcing the bandwidth committed to a host. For example, if a network allocates **3 Mbps** bandwidth to a host, but the host transmits data at **12 Mbps for 2 seconds** (24 Mb of data), remains idle for **5 seconds**, and later transmits at **2 Mbps for 3 seconds** (6 Mb of data), the total traffic generated is **30 Mb in 10 seconds**.
- Instead of allowing this burst to overload the network, the Leaky Bucket releases the stored data continuously at the committed rate of **3 Mbps**, ensuring that the average transmission rate remains within the allocated bandwidth. This smoothing effect helps prevent congestion and improves network performance.

- In a simple implementation, packets are stored in a FIFO queue. For **fixed-size packets**, a fixed number of packets are transmitted at every clock tick. For **variable-length packets**, transmission is based on the number of bytes or bits rather than the number of packets.
- At each clock tick, a counter is initialized with a fixed value representing the allowable number of bytes. Packets are transmitted only if their size is less than or equal to the remaining counter value, and the counter is reduced accordingly. Once the remaining counter becomes smaller than the next packet size, transmission stops until the next clock tick, when the counter is reset.
- The Leaky Bucket algorithm effectively **averages the data rate**, ensuring a smooth and predictable traffic flow.
- However, it does not preserve traffic bursts, as all outgoing traffic is transmitted at a constant rate. If the bucket (queue) becomes full due to excessive incoming traffic, **packets are dropped**, making the algorithm suitable for applications that require steady traffic transmission rather than burst tolerance.

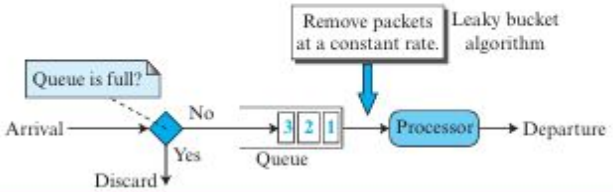
Figure 8.62 *Leaky bucket*



Algorithm for Leaky Bucket (Variable-Length Packets)

1. Initialize a counter to **n** at each clock tick, where **n** represents the maximum number of bytes (or bits) that can be transmitted during one clock interval.
2. Compare the counter value with the size of the packet at the front of the FIFO queue.
3. If the counter value is greater than or equal to the packet size:
    - Transmit the packet.
    - Decrease the counter by the packet size.
    - Repeat this step for the next packet until the remaining counter value is smaller than the size of the next packet.
4. If the remaining counter value is smaller than the size of the next packet, stop transmission and wait for the next clock tick.
5. At the next clock tick, reset the counter to **n** and repeat the process.

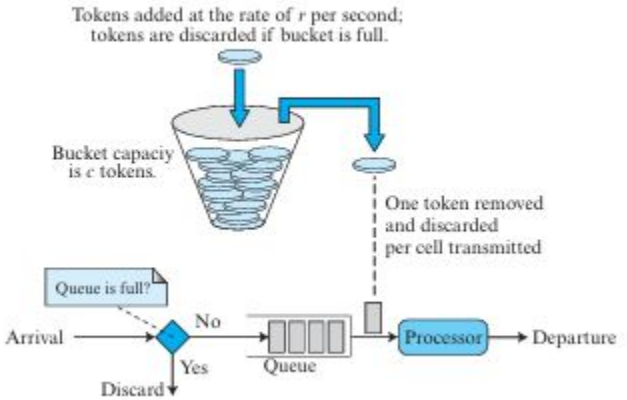
Figure 8.63 *Leaky bucket implementation*



Token Bucket Algorithm

- The **Token Bucket algorithm** is an improved traffic shaping and policing technique that overcomes the major limitation of the **Leaky Bucket algorithm**.
- Unlike the Leaky Bucket, which does not give any benefit to an idle host, the Token Bucket allows an idle host to **accumulate credits** in the form of **tokens**.
- These accumulated tokens can later be used to transmit bursty traffic at a higher rate without violating the average bandwidth allocated to the host. Thus, the Token Bucket supports both **bursty traffic** and **average rate control**, making it more flexible and efficient for modern network applications.

Figure 8.64 *Token bucket*



In this algorithm, the bucket has a capacity of **c tokens**, and tokens are generated and added to the bucket at a constant rate of **r tokens per second**. If the bucket becomes full, any newly generated tokens are discarded. To transmit data, **one token is removed from the bucket for every packet (or cell) sent**. If sufficient tokens are available, packets are transmitted immediately, allowing burst transmissions. However, if there are no tokens in the bucket, packets must wait until new tokens are generated. In this way, the algorithm permits occasional bursts while ensuring that the long-term average transmission rate remains within the committed bandwidth.

The maximum number of packets that can be transmitted during any time interval **t** is given by:

$$\boxed{\text{Maximum number of packets} = rt + c}$$

where:

- **r** = Token generation rate (tokens/second)
- **t** = Time interval (seconds)
- **c** = Bucket capacity (maximum number of stored tokens)

The maximum average transmission rate is:

$$\boxed{\text{Maximum average rate} = \frac{rt + c}{t} \text{ packets/second}}$$

Algorithm for Token Bucket

1. Initialize the bucket with a maximum capacity of **c tokens**.
2. Generate and add tokens to the bucket at a constant rate of **r tokens per second**.
3. If the bucket is full, discard any newly generated tokens.
4. When a packet arrives, check whether at least one token is available.
5. If a token is available, remove one token and transmit the packet immediately.
6. If no token is available, the packet waits until a new token is generated.
7. Repeat the process for all incoming packets.

Combining Token Bucket and Leaky Bucket

- The **Token Bucket** and **Leaky Bucket** algorithms are often combined to provide efficient traffic shaping. The **Token Bucket** allows an idle host to accumulate tokens, enabling it to send bursty traffic when required. The output is then passed to the **Leaky Bucket**, which smooths the bursty traffic by transmitting packets at a constant rate. Thus, the combination supports burst transmission while regulating traffic and reducing congestion. For proper operation, the **Leaky Bucket rate should be higher than the token generation rate**.

Resource Reservation

- **Resource Reservation** is the process of reserving network resources such as **bandwidth, buffer space, and CPU time** before data transmission. This ensures the required Quality of Service (QoS) by reducing delay, packet loss, and congestion. The **Integrated Services (IntServ)** QoS model uses resource reservation to guarantee network performance.

Admission Control

- **Admission Control** is a mechanism in which a **router or switch** decides whether to **accept or reject** a new data flow based on its available resources. It checks **bandwidth, buffer size, CPU speed, and existing commitments** before admitting a flow. If sufficient resources are available, the flow is accepted; otherwise, it is rejected to avoid congestion. In **ATM networks**, this mechanism is called **Connection Admission Control (CAC)**.

Integrated Services (IntServ)

- **Integrated Services (IntServ)** is a **flow-based Quality of Service (QoS) model** developed by the **IETF** to provide guaranteed QoS for applications such as **real-time audio and video**. Unlike the traditional Internet, which provides only **best-effort service**, IntServ reserves network resources such as **bandwidth and buffer space** for each data flow before transmission begins. Since IntServ is flow-based, all resource reservations must be completed before the data flow starts.

IntServ is based on **three main schemes**:

- **Packet Classification:** Packets are classified according to the type of service they require.
- **Scheduling:** Packets are forwarded based on their flow characteristics to meet QoS requirements.
- **Admission Control:** Routers check whether sufficient resources are available before accepting a new flow. If resources are insufficient, the flow is rejected.

Since **IP is a connectionless protocol**, IntServ requires the **Resource Reservation Protocol (RSVP)** to establish resource reservations before communication begins.

Flow Specification (FlowSpec)

A flow is defined using a **Flow Specification (FlowSpec)**, which consists of two parts:

- **Rspec (Resource Specification):** Specifies the resources required by the flow, such as **bandwidth and buffer space**.
- **Tspec (Traffic Specification):** Describes the traffic characteristics of the flow.

Admission

After receiving the Flow Specification, the router checks its **available bandwidth, buffer space, CPU resources, and existing commitments**. The router **admits** the flow if sufficient resources are available; otherwise, it **rejects** the request to maintain QoS.

Service Classes in IntServ

1. Guaranteed Service

- Designed for **real-time applications** such as **VoIP and video conferencing**.
- Provides **guaranteed minimum end-to-end delay**.
- Ensures packets are delivered within a specified time and are **not discarded** if the traffic remains within the **Tspec** limits.
- It is a **quantitative service**, where delay and data rate are explicitly specified.

2. Controlled-Load Service

- Designed for applications such as **file transfer, e-mail, and Internet access**.
- Allows **small delays** but aims to provide **low packet loss** and avoid network congestion.
- It is a **qualitative service**, offering performance similar to a lightly loaded network.

Resource Reservation Protocol (RSVP)

The **Resource Reservation Protocol (RSVP)** is a protocol that works **on top of IP** to provide **connection-oriented resource reservation** required by IntServ. RSVP reserves network resources before data transmission begins.

Features of RSVP

- Works over the **connectionless IP protocol**.
- Supports both **multicast** and **unicast** communication.
- Uses **receiver-based reservation**, where the **receiver** requests and reserves network resources instead of the sender.
- Establishes and releases resource reservations as required by the application.

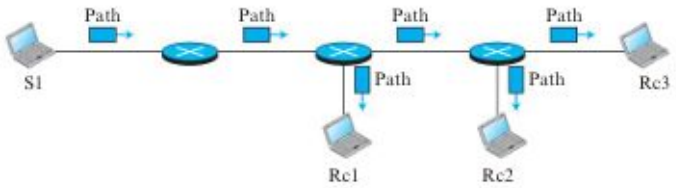
Resource Reservation Protocol (RSVP)

The **Resource Reservation Protocol (RSVP)** is a protocol that works **on top of IP** to support the **Integrated Services (IntServ)** model. Since **IP is a connectionless protocol**, RSVP provides the **connection-oriented resource reservation** required for QoS. Although RSVP is mainly used with IntServ, it is an **independent protocol** and can also be used with other QoS models in the future.

Features of RSVP

- **Supports Multicast and Unicast:** RSVP is primarily designed for **multicast communication**, but it also supports **unicast**, which is considered a special case of multicast with only one receiver.
- **Receiver-Based Reservation:** Unlike most protocols where the sender reserves resources, **the receiver initiates the reservation**. This matches the operation of multicast routing, where receivers decide to join or leave a multicast group.

Figure 8.65 *Path messages*



RSVP Messages

RSVP mainly uses **two types of messages**:

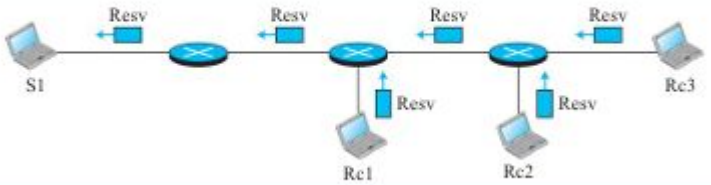
1. Path Message

The **Path** message is sent **from the sender to all receivers** along the multicast path. As it travels through the network, it records the path information required for resource reservation. When the multicast path branches, a **new Path message** is created for each branch.

2. Resv (Reservation) Message

- After receiving the Path message, each receiver sends a **Resv** message **towards the sender (upstream)**.
- As the Resv message passes through RSVP-enabled routers, resources such as **bandwidth and buffer space** are reserved. If a router does not support RSVP, it forwards the packets using the **best-effort service**.

Figure 8.66 *Resv messages*



Reservation Merging

- In multicast communication, RSVP **merges reservation requests** from multiple receivers instead of reserving resources separately for each one. The router reserves resources based on the **largest bandwidth request**. For example, if one receiver requests **2 Mbps** and another requests **1 Mbps**, the router reserves **2 Mbps**, which is sufficient for both receivers.

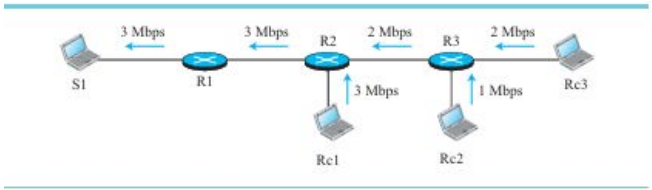
Reservation Styles

RSVP defines **three reservation styles**:

- **Wildcard Filter (WF):** A **single reservation** is shared by **all senders** and is based on the **largest bandwidth request**. Used when different senders are **not active simultaneously**.
- **Fixed Filter (FF):** A **separate reservation** is created for **each sender (flow)**. Used when multiple senders are **likely to transmit at the same time**.
- **Shared Explicit (SE):** A **single reservation** is shared by a **specified group of senders**.

Soft State

- RSVP maintains reservation information as a **soft state**, meaning the reservation must be **periodically refreshed**. If the reservation is not refreshed, it is automatically removed. The **default refresh interval is 30 seconds**. This differs from protocols such as **ATM**, which use a **hard state**, where the reservation remains until it is explicitly deleted.



Differentiated Services (DiffServ)

- **Differentiated Services (DiffServ)** is a **class-based Quality of Service (QoS) model** developed by the **IETF** to overcome the limitations of **Integrated Services (IntServ)**.
- In DiffServ, **packets are classified into different priority classes** by the host or boundary router, and routers forward packets based on their **class** rather than maintaining information about individual flows. This improves **scalability** and simplifies router operation.

Key Features

- Processing is moved from the **core network to the edge network**.
- Provides **per-class service** instead of **per-flow service**.
- Suitable for large IP networks due to better scalability.

DS (Differentiated Services) Field

- Each IP packet contains a **DS (Differentiated Services) field**, which replaces the **ToS field (IPv4)** or **Priority field (IPv6)**.
- The DS field has two subfields:
- **DSCP (6 bits):** Defines the **Per-Hop Behavior (PHB)** for packet forwarding.
- **CU (2 bits):** Currently Unused.
- Routers use the **DSCP value** to determine how each packet should be handled.

Figure 8.68 *DS field*



Per-Hop Behaviors (PHBs)

DiffServ defines three main **Per-Hop Behaviors (PHBs)**:

- **DE PHB (Default PHB):** Provides **best-effort delivery**.
- **EF PHB (Expedited Forwarding):** Provides **low delay, low packet loss, and guaranteed bandwidth**, making it suitable for real-time applications.
- **AF PHB (Assured Forwarding):** Provides **high delivery assurance**, but packets may be dropped if traffic exceeds the agreed profile.

Traffic Conditioner

A **Traffic Conditioner** ensures that traffic follows the negotiated profile using four components:

- **Meter:** Checks whether incoming traffic matches the traffic profile.
- **Marker:** Marks or **down-marks** packets based on the meter's result.
- **Shaper:** Delays or reshapes traffic to make it conform to the profile.
- **Dropper:** Discards packets that seriously violate the traffic profile.

